



life.augmented



ST67W611 Wi-Fi 6 + Bluetooth® LE + Thread

Santa Clara Wireless Team

Agenda

X-Cube- ST67W61 Package

Introduction to MLC

How to use X-CUBE-ST67W61
within STM32CubeMX
(Walk-through)

Demo: MLC Workflow in MEMS Studio

Device Setup and Flashing

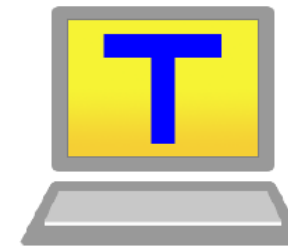
MQTT Hands-On
Integrating MLC into ST67 MQTT Demo

Walk through of the MQTT
application

Documentation

Prerequisites

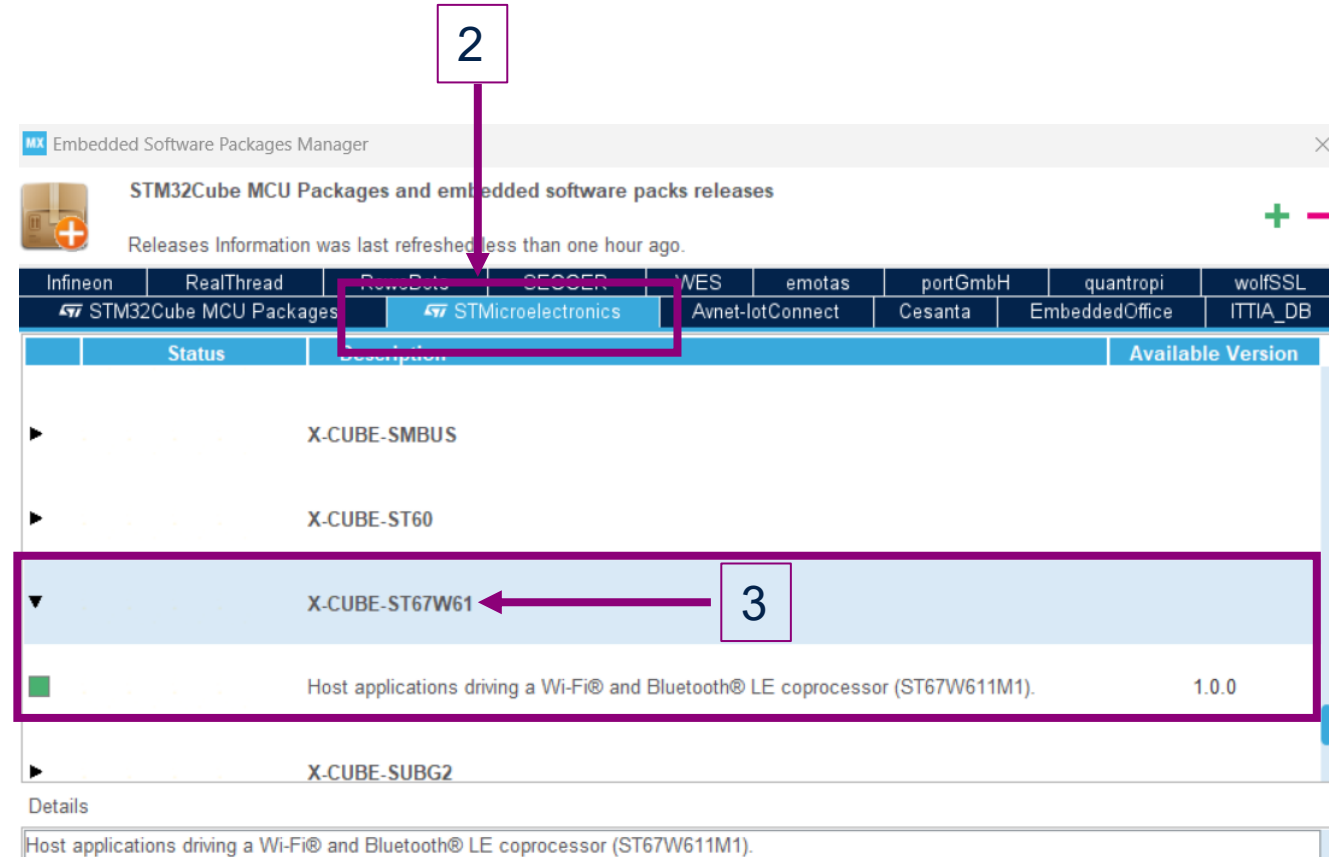
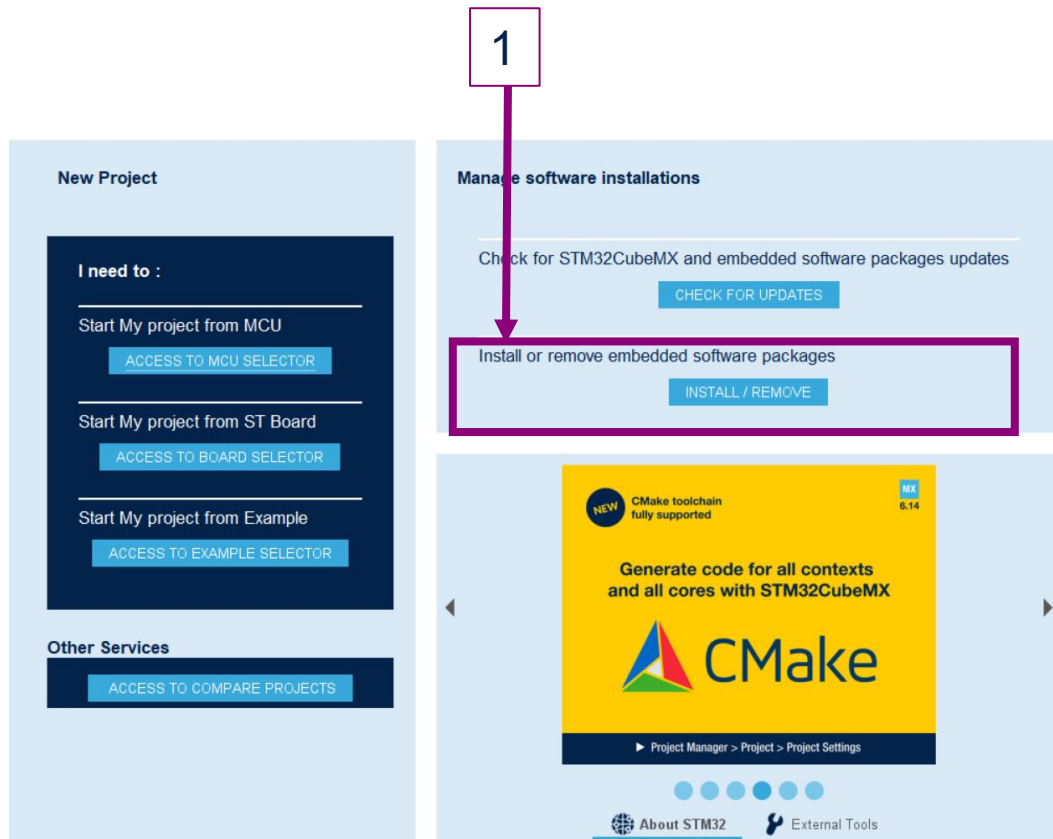
- [X_CUBE_ST67W61_V1.0.0](#)
- [STM32CubeIDE v1.16.1+](#)
- [STM32CubeProgrammer v2.18.0+](#)
- [STM32CubeMX v6.14.1+](#)
- [HyperTerminal – TeraTerm v.5.4.0](#)
- [Python v.3.12.7+](#)
- [STBLE Toolbox](#)
- IoT MQTT Panel App



Installing X-CUBE-ST67W61

Open STM32CubeMX with admin rights

1. Go to “Install/Remove” software packages
2. Under STMicroelectronics tab
3. Install the X-CUBE-ST67W61



STM32U5 and X-CUBE-FREERTOS

MX Embedded Software Packages Manager

STM32Cube MCU Packages and embedded software packs releases

Releases Information was last refreshed 4 hours ago.

Infinion	RealThread	RoweBots	SEGGER	WES	emotas	portGmbH	quantropi	wolfSSL
STM32Cube MCU Packages	STMicroelectronics	Avnet-IotConnect	Cesanta	EmbeddedOffice	ITTIA_DB			
Description	Installed Version	Available Version						
▼ STM32U5								
☒ STM32Cube MCU Package for STM32U5 Series	1.8.0	1.8.0						
☐ STM32Cube MCU Package for STM32U5 Series (Size : 390 MB)		1.7.0						
☐ STM32Cube MCU Package for STM32U5 Series (Size : 351 MB)		1.6.0						
☐ STM32Cube MCU Package for STM32U5 Series (Size : 306 MB)		1.5.0						

Download latest STM32U5

Download latest X-CUBE-FREERTOS

MX Embedded Software Packages Manager

STM32Cube MCU Packages and embedded software packs releases

Releases Information was last refreshed 4 hours ago.

Infinion	RealThread	RoweBots	SEGGER	WES	emotas	portGmbH	quantropi	wolfSSL
STM32Cube MCU Packages	STMicroelectronics	Avnet-IotConnect	Cesanta	EmbeddedOffice	ITTIA_DB			
Status	Description	Available Version						
▼ X-CUBE-FREERTOS								
☒	FreeRTOS STM32Cube expansion package for STM32H5/U5/WBA/C0/U0/N6/U3 series	1.3.1						
☐	FreeRTOS STM32Cube expansion package for STM32H5/U5/WBA/C0/U0/N6/U3 series (Siz	1.3.0						
☐	FreeRTOS STM32Cube expansion package for STM32H5/U5/WBA/C0/U0/N6/U3 series (Siz	1.2.0						
☐	FreeRTOS STM32Cube expansion package for STM32H5/U5/WBA/C0/U0/N6/U3 series (Siz	1.1.0						



life.augmented

X-Cube-ST67W61

X-CUBE-ST67W61 package overview

Board support package

CMSIS libraries

Device HAL drivers

APIs include files

Templates for middleware and driver config files
Integration between generic API and driver

W61 driver

AT commands / responses
SPI driver

UART Output logs

Memory usage statistics
CLI Console

Variety of **Nucleo** hosts

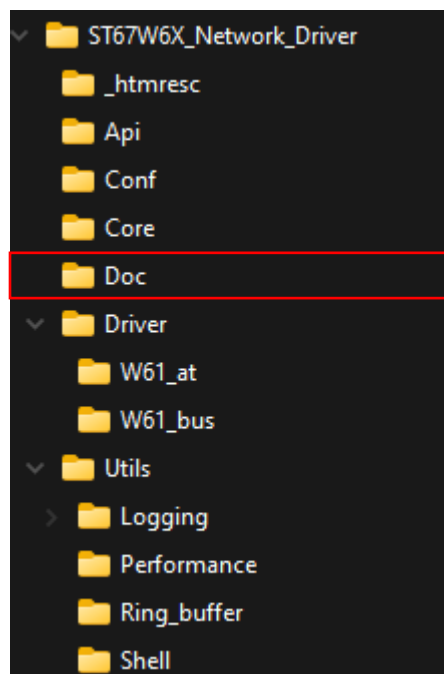
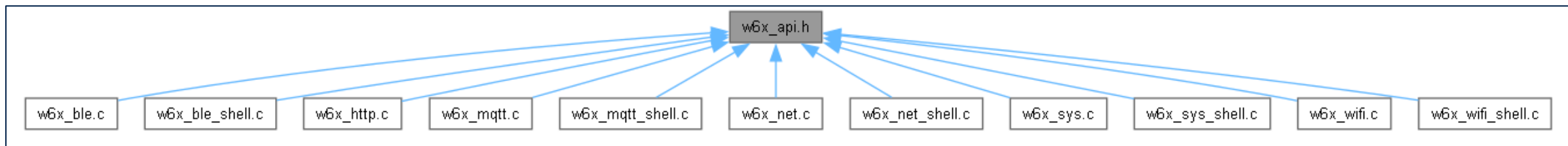
STM32CubeIDE and **EWARM** Project Files

X-CUBE-ST67W1 middleware

	Scope / description
W61_bus	This is the SPI driver for communication between the host and the ST67W61.
W61_at	It does the formatting of the AT commands to be sent and it implements a receive tasks to decode the received messages from the W61 and dispatch them to the correspondent module.
W6x service API	The Core directory implements an adaptation layer between the W61 AT driver and the API proposed to the application. The APIs are "Zephyr-like" for the Wi-Fi® and socket API for the network operations.
Utils	Console handling, debug, trace, performance measurements.

All the above submodules make use of FreeRTOS™

API Documentation



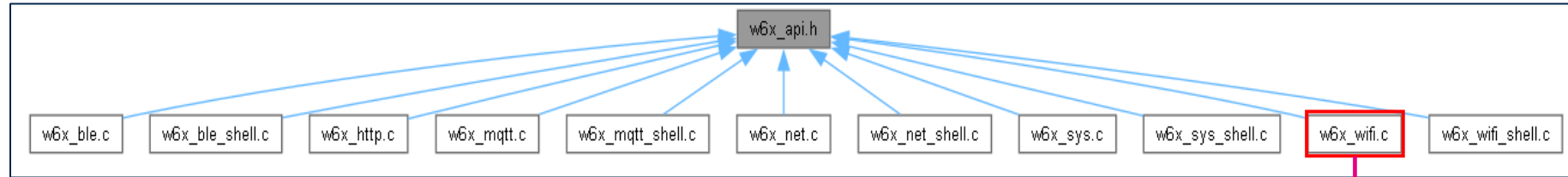
Name

ST67W6X_Network_Driver.chm

Found in \Middlewares\ST\ST67W6X_Network_Driver\Doc

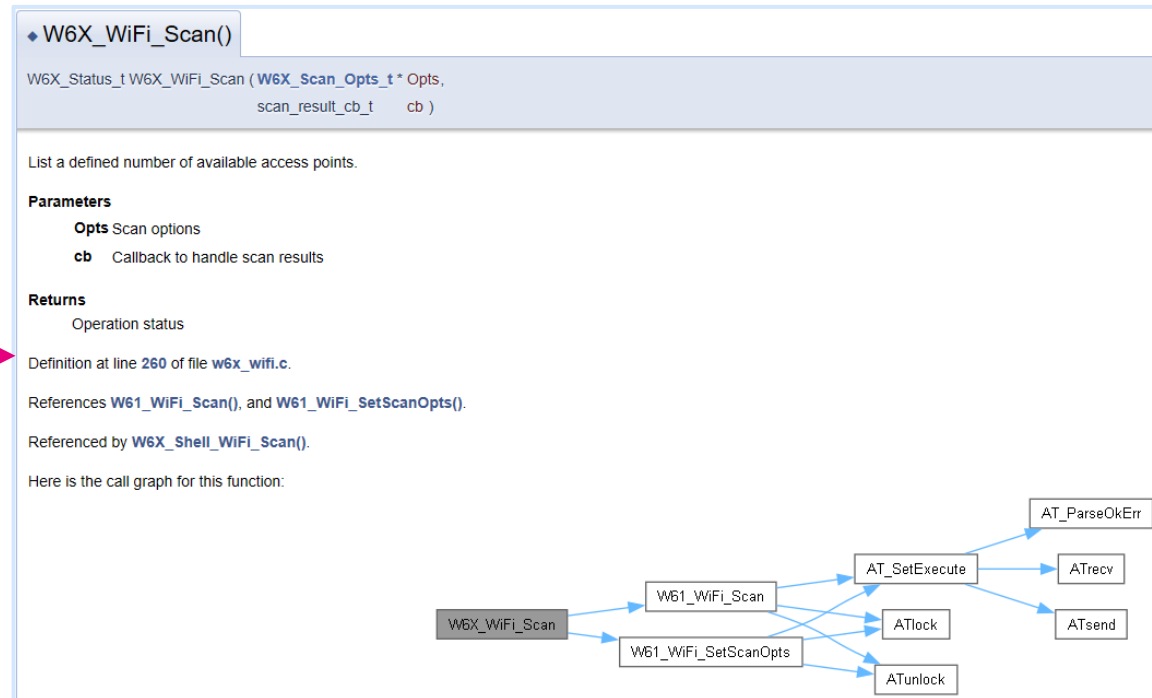
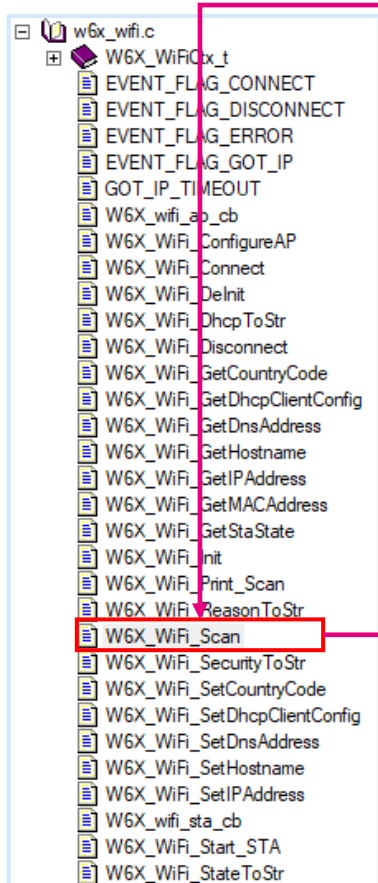
- API Definitions
- Overview of API calls
- Correlated AT Commands

API Documentation cont...



Selecting **w6x_wifi.c** will provide the different API categories

Selecting the API, **W6X_WiFi_Scan()**, will provide information and the call graph



X-CUBE-ST67W61 applications

	Scope / description
Bluetooth® LE commissioning	Uses Bluetooth® LE to provide Wi-Fi® SSID and password after scanning available AP
Bluetooth® LE p2p client	Point-to-Point communication demo using Bluetooth® LE (GATT Client) Scan and connect to a p2pServer Write messages & receive notifications from the p2pServer
Bluetooth® LE p2p server	Point-to-Point communication demo using Bluetooth® LE (GATT server) Advertises and wait for a connection from either: - p2pClient app - <i>ST BLE Toolbox</i> smartphone application
Echo	TCP Echo feature over Wi-Fi - Good starting point for generic user development
CLI	ST67 evaluation via CLI and for WTS tests the Wi-Fi solution. - Wi-Fi API: scan, connect, disconnect, etc - NET socket API and iperf - Bluetooth® LE API
Echo FOTA	FOTA (firmware update over-the-air) feature demo over Wi-Fi - Using HTTP
HTTP server	HTTP server over Network API The server, running on the host board, has been validated with 1 client, using HTTP requests to send and receive data to and from the server.
HTTPS client	HTTPS Client over Network API demo - Basic GET requests to retrieve worldwide weather reports
Wi-Fi® commissioning	Wi-Fi® commissioning project using an HTTP server, hosted by a device configured as a soft access point (SoftAP) and station (STA).
MQTT Demo	Starting point for MQTT user development - Sends IKS sensor data to an MQTT broker

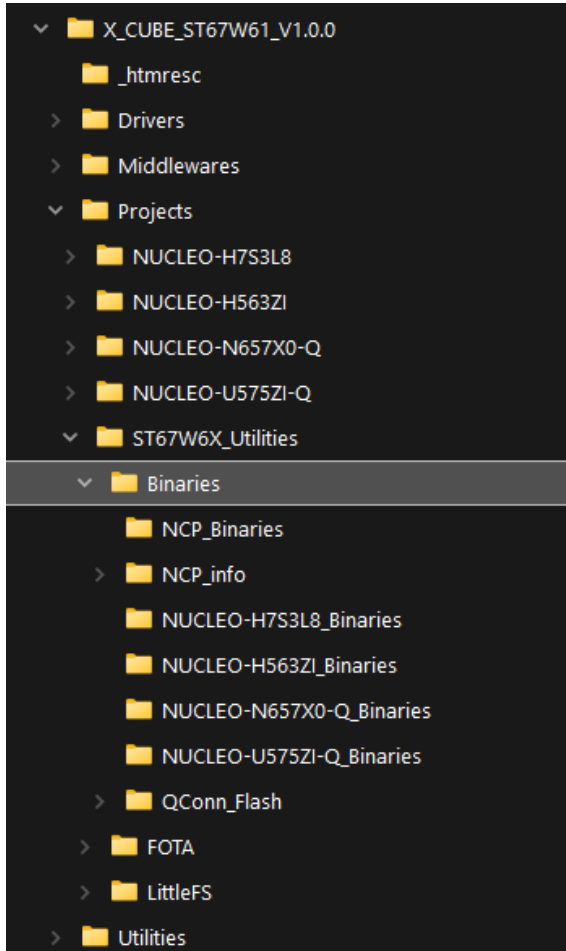
- Applications
 - ST67W6X
 - ST67W6X_BLE_Commissioning
 - ST67W6X_BLE_p2pClient
 - ST67W6X_BLE_p2pServer
 - ST67W6X_CLI
 - ST67W6X_Echo
 - ST67W6X_FOTA
 - ST67W6X_HTTP_Server
 - ST67W6X_HTTPS_Client
 - ST67W6X_WiFi_Commissioning
- Demonstrations
 - ST67W6X
 - ST67W6X_MQTT

X-CUBE-ST67W1 NCP tools

Scripts for flashing the latest mission or manufacturing firmware:

..\x-cube-st67w61-v1.0.0\Projects\ST67W6X_Uilities\Binaries

Tool	Description
NCP_get_chip_info.bat	Script to retrieve ST67 MAC address and locking information even if no firmware is loaded onto the ST67 device.
NCP_update_mfg.bat	Script to flash the ST67 module with the manufacturing application.
NCP_update_mission_profile.bat	Script to flash the ST67 module with the mission profile application.



ST67W611M Security Overview

Two modes, each with a header containing a digital signature and versioning info

A. Mission mode:

- **Bootloader binary**
 - Authenticated by the ROM code during initial boot process
 - Versioning info to prevent rollback
- **Mission profile binary**
 - Wi-Fi® & Bluetooth® LE binary used for operational tasks
 - Authenticated by bootloader to ensure integrity & authenticity before execution

B. Manufacturing mode:

- **Bootloader binary**
 - Authenticated by the ROM code during initial boot process
 - Versioning info to prevent rollback
- **MFG firmware binary**
 - Used for test and production configuration
 - Authenticated by bootloader to ensure integrity & authenticity before execution



life.augmented



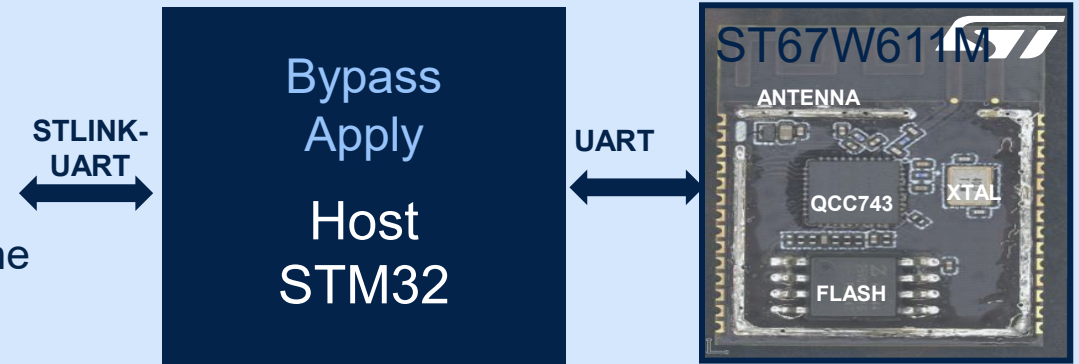
Flashing ST67 & Device Setup

ST67W611M Modes of Operation

Manufacturing and Mission Modes

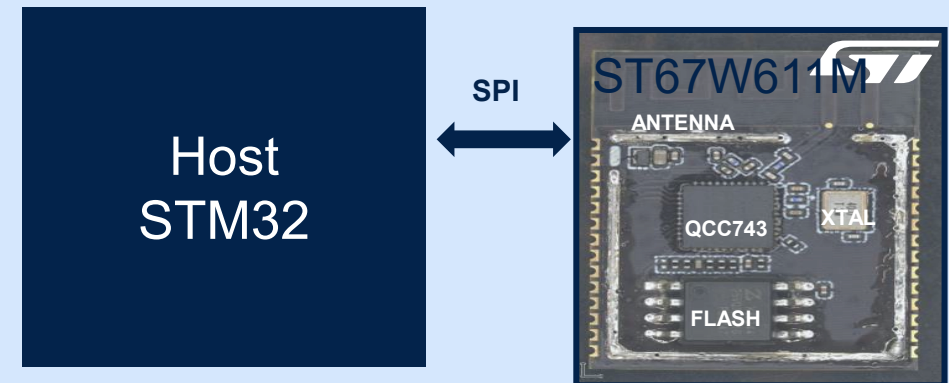
- **Manufacturing mode**

- Host-less Application
- Dedicated to RF performances monitoring and calibration.
- QC provides graphical tools to control the device through the **UART** interface



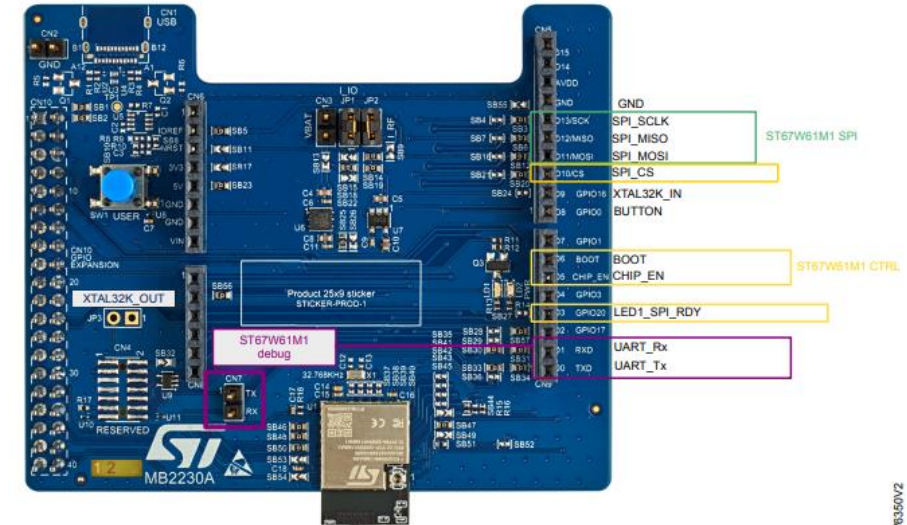
- **Mission mode (Functional)**

- Implementing the AT command for the different supported protocol (WIFI / BLE / MQTT /..) through the **SPI** interface.
- The ROM boot loader loads and executes the program from SPI Flash to boot the system.

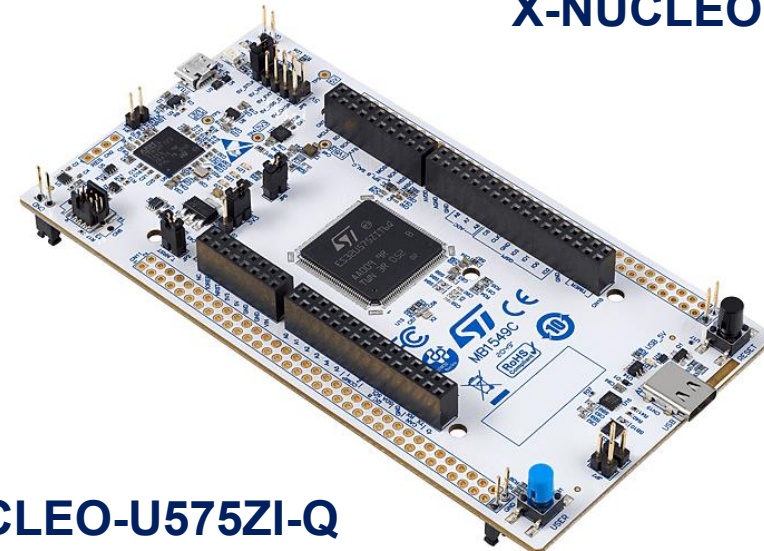


STM32U575 and ST67W611M configuration

- 1x NUCLEO-U575ZI-Q board
- 1x X-NUCLEO-67W61M1 board
- Boards connected through Arduino® connectors
 - Key signals: SPI (SCK, MISO, MOSI, CS), SPI_RDY, CHIP_EN, BOOT, UART_TX/RX
- Power: MicroUSB connection to NUCLEO-U575ZI-Q

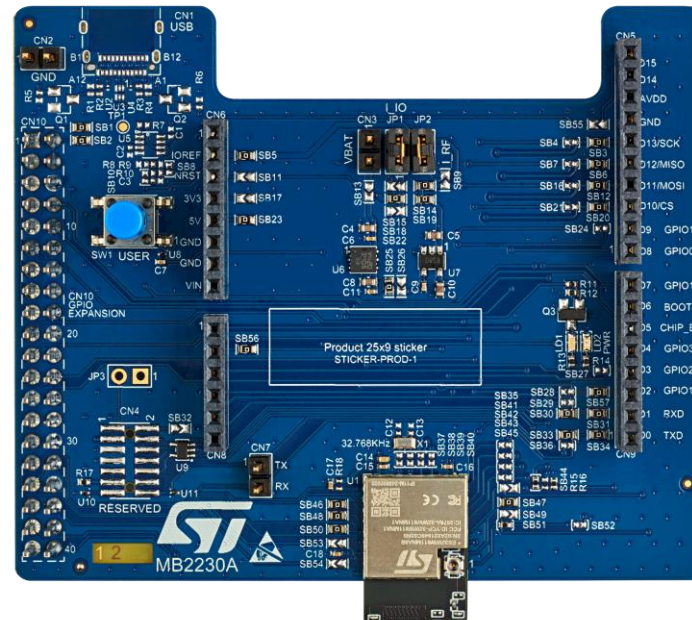


X-NUCLEO-67W61M1



NUCLEO-U575ZI-Q

Nucleo-U575ZI and X-Nucleo-67W61M1 Configuration



When stacking the Nucleo and X-Nucleo boards the Arduino connectors must be aligned.

The MicroUSB port is used when programming the STM32U575 or ST67W611M1.

The MicroUSB port is also used to read the UART messages from the STM32U575 in a serial terminal.

Flashing ST67W611M1

Mission Mode (Functional Mode) Setup

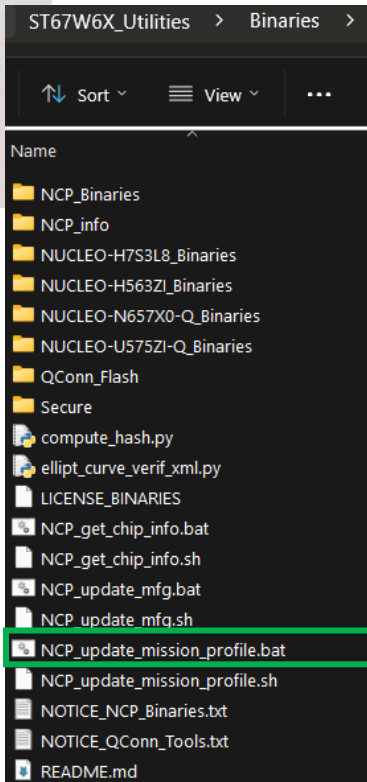
Prerequisites:

- Installed STM32CubeProgrammer
 - Installed in default directory (C:\Program Files\STMicroelectronics\STM32Cube\STM32CubeProgrammer)
- Installed X-CUBE-ST67W61
 - Installed on C: drive (C:\x-cube-st67w61-v1.0.0)**

Using batch scripts located:

C:\x-cube-st67w61-v1.0.0\Projects\ST67W6X_Uilities\Binaries

1. Nucleo-U575ZI-Q and X-Nucleo-67W61M1 stacked together
2. USB cable connected to microUSB port on Nucleo-U575ZI-Q and Windows PC.
3. Double-click the batch files to execute
 1. NCP_update_mission_profile.bat will flash the ST67W61M with the spi_wifi binary and the host processor with CLI application.

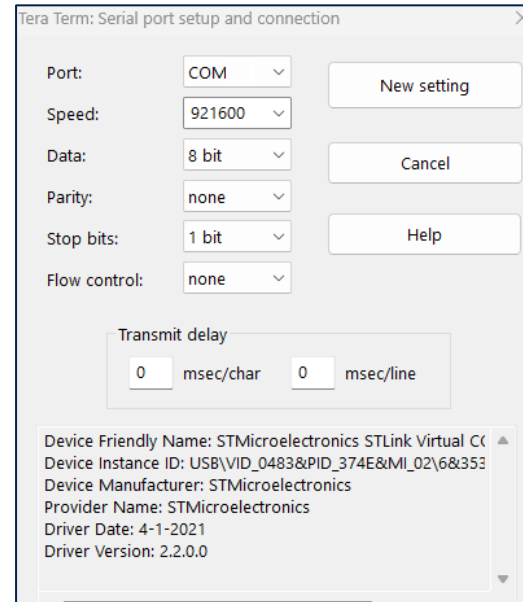
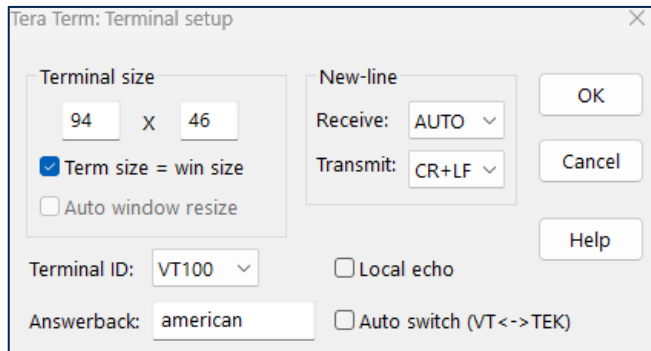


Flashing ST67W611M1

Mission Mode (Functional Mode) Verification

Flashing success can be verified by connecting to the Nucleo-U575ZI-Q with a serial terminal.

Configure the serial terminal with the following parameters and connect to the corresponding COM port for the Nucleo-U575ZI-Q.



With the serial terminal program configured, press the reset button the Nucleo-U575ZI-Q and verify application and SDK version.

```
>#### Welcome to ST67W6X CLI Application ####
# build: 22:39:24 May 28 2025
----- Host info -----
Host FW Version:      1.0.0
----- ST67W6X info -----
ST67W6X MW Version:   1.0.0
AT Version:           1.0.0.1
SDK Version:          2.0.75
MAC Version:          1.6.38
Build Date:           May 20 2025 23:01:38
Module ID:            C6AFDBD111400004
BOM ID:               1
Manufacturing Year:   2024
Manufacturing Week:   47
Battery Voltage:      3.334 V
Trim Wi-Fi hp:        6,6,6,6,6,6,5,5,6,6,6,7,7,7
Trim Wi-Fi lp:        7,7,8,8,8,9,9,9,10,10,10,11,11,11
Trim BLE:             5,4,5,6,7
Trim XTAL:            37
MAC Address:          40:82:7b:00:33:26
Anti-rollback Bootloader: 0
Anti-rollback App:    0
-----
mount success
Wi-Fi init is done
Net init is done
MQTT init is done
Starting FOTA task
ready
Application started from bank 1
```



life.augmented

How to use X-CUBE-ST67W61 within STM32CubeMX

Software Development Tools



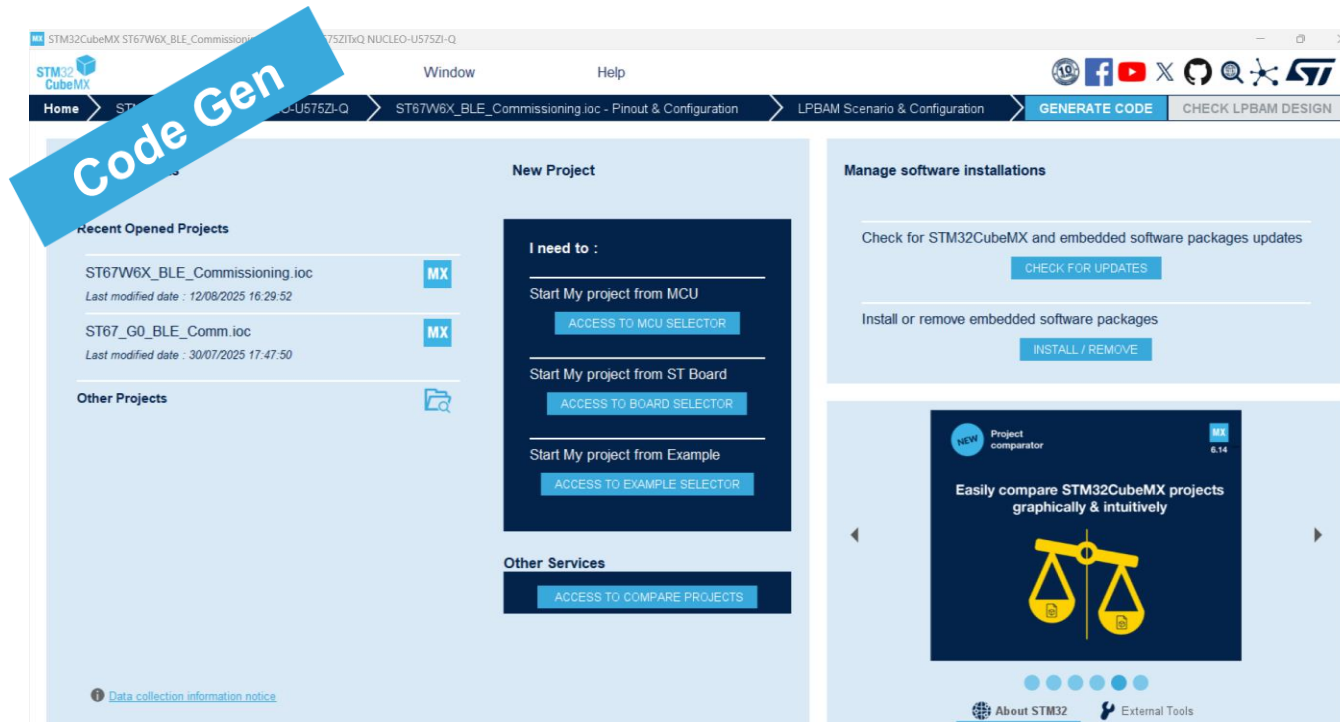
“STM32CubeIDE”
development environment

↳ **“STM32CubeMX”**
initialization code generator

↳ **“X-CUBE-ST67W61”**
application code and drivers for Wi-Fi

STM32CubeMX

an All-in-1 Development Tool



Very powerful configuration and code generation

Support all STM32 MCU and MPU, with integrated powerful Finder

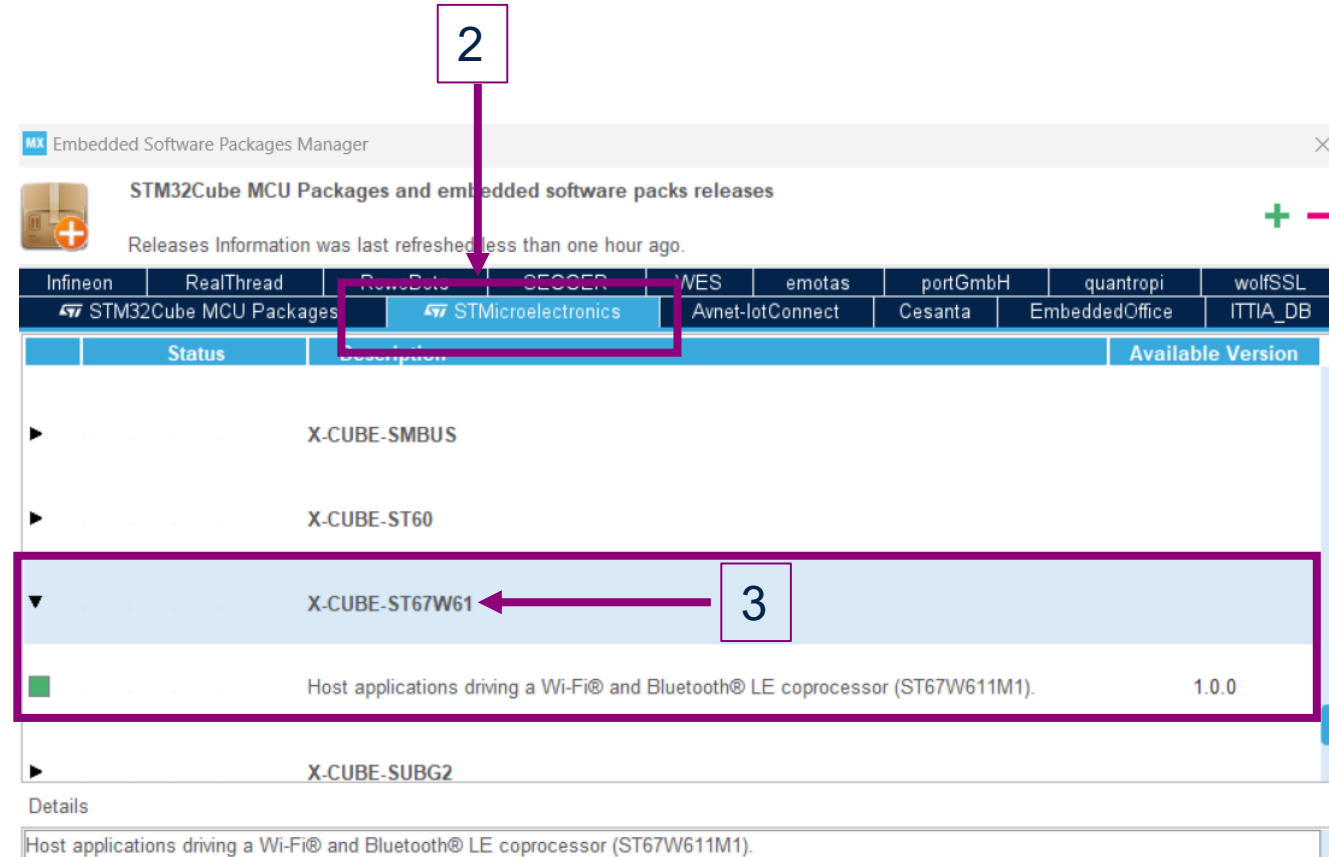
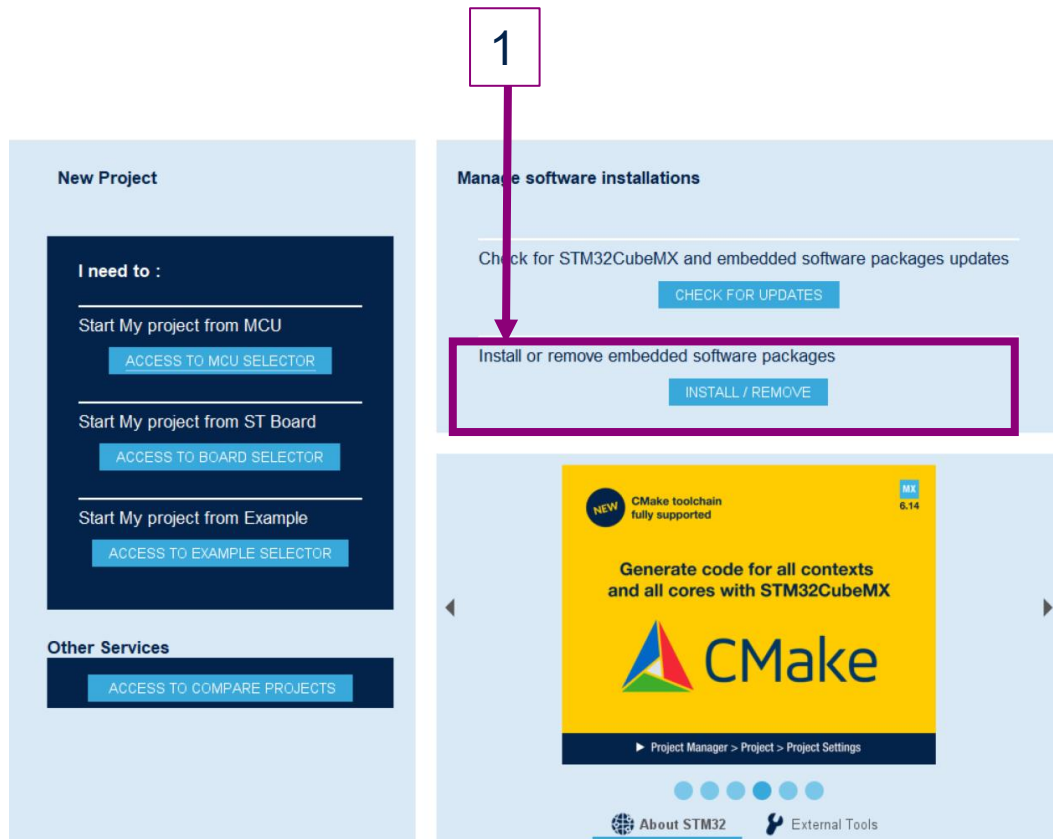
Pinouts, clock tree, peripherals and middleware configuration.

Expandable to support wireless connectivity, sensors and much more!

Installing X-CUBE-ST67W61

Open STM32CubeMX with admin rights

1. Go to “Install/Remove” software packages
2. Under STMicroelectronics tab
3. Install the X-CUBE-ST67W61



STM32U5 and X-CUBE-FREERTOS

MX Embedded Software Packages Manager

STM32Cube MCU Packages and embedded software packs releases

Releases Information was last refreshed 4 hours ago.

Infinion	RealThread	RoweBots	SEGGER	WES	emotas	portGmbH	quantropi	wolfSSL
STM32Cube MCU Packages	STMicroelectronics	Avnet-IotConnect	Cesanta	EmbeddedOffice	ITTIA_DB			
Description	Installed Version	Available Version						
▼ STM32U5								
☒ STM32Cube MCU Package for STM32U5 Series	1.8.0	1.8.0						
☐ STM32Cube MCU Package for STM32U5 Series (Size : 390 MB)		1.7.0						
☐ STM32Cube MCU Package for STM32U5 Series (Size : 351 MB)		1.6.0						
☐ STM32Cube MCU Package for STM32U5 Series (Size : 306 MB)		1.5.0						

Download latest STM32U5

Download latest X-CUBE-FREERTOS

MX Embedded Software Packages Manager

STM32Cube MCU Packages and embedded software packs releases

Releases Information was last refreshed 4 hours ago.

Infinion	RealThread	RoweBots	SEGGER	WES	emotas	portGmbH	quantropi	wolfSSL
STM32Cube MCU Packages	STMicroelectronics	Avnet-IotConnect	Cesanta	EmbeddedOffice	ITTIA_DB			
Status	Description	Available Version						
▼ X-CUBE-FREERTOS								
☒	FreeRTOS STM32Cube expansion package for STM32H5/U5/WBA/C0/U0/N6/U3 series	1.3.1						
☐	FreeRTOS STM32Cube expansion package for STM32H5/U5/WBA/C0/U0/N6/U3 series (Siz	1.3.0						
☐	FreeRTOS STM32Cube expansion package for STM32H5/U5/WBA/C0/U0/N6/U3 series (Siz	1.2.0						
☐	FreeRTOS STM32Cube expansion package for STM32H5/U5/WBA/C0/U0/N6/U3 series (Siz	1.1.0						

Three possible cases to generate the code from the Cube MX:

- **Case 1:** How to generate a project starting from an existing .ioc
- **Case 2:** How to export a project to a pinout compatible MCU
- **Case 3:** How to generate a project starting from scratch

In this section: We would be walking through to generate **BLE Commissioning** code using the following:

- Case 1 with NUCLEO-U575ZI-Q as host processor
- Case 3 with NUCLEO-G0B1RE as host processor

BLE Commissioning App

- This application aims to demonstrate **how to provision Wi-Fi credentials via Bluetooth® LE** to establish a Wi-Fi connection to an access point.



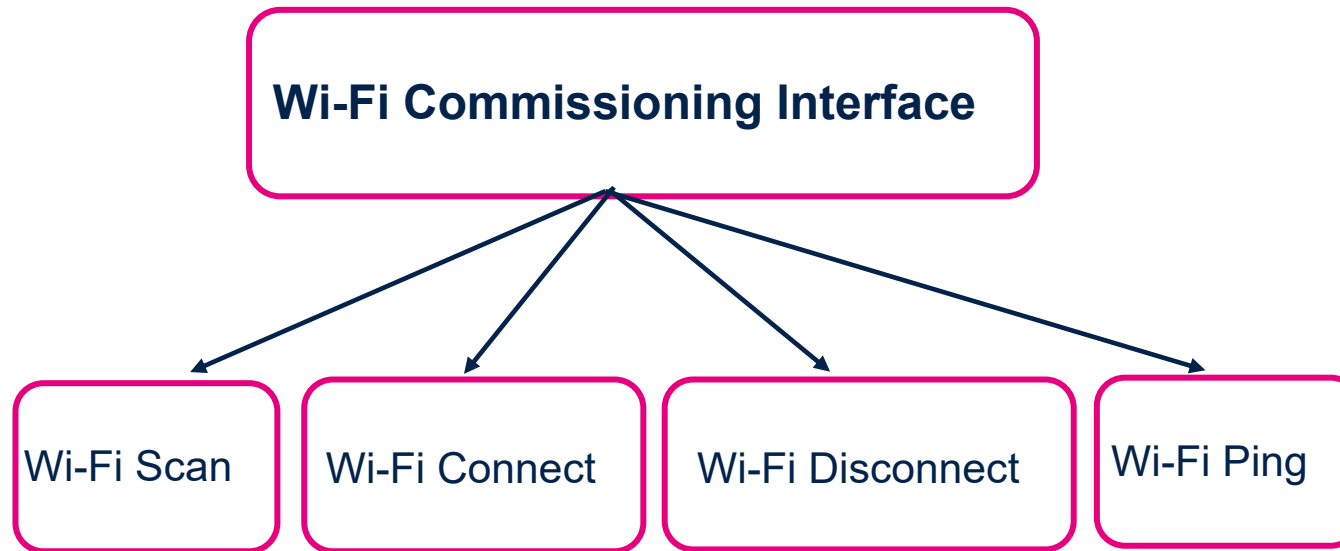
BLE Commissioning App cont..

The application acts as a peripheral device embedding the commissioning profile and its four characteristics.

At startup, the application starts to advertise.

ST Web Bluetooth® Interface must be used to scan and connect to the commissioning application: (Web-Interface available from the browser) / ST BLE Toolbox (Mobile Application)

Once connected, a Wi-Fi commissioning interface is available on web Bluetooth® page/ ST BLE Toolbox



Commissioning profile overview

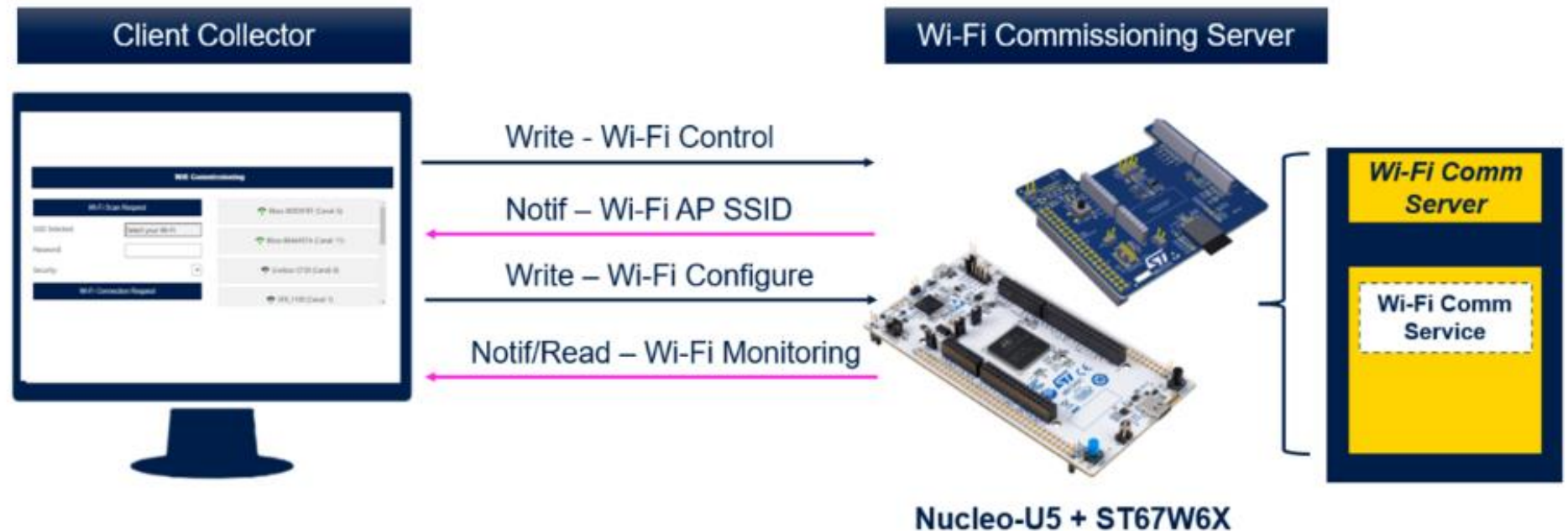
The Wi-Fi commissioning application demonstrates point to point communication using proprietary service and characteristics. It acts as a Peripheral device with one GATT service and four characteristics.

Wi-Fi commissioning server:

- Contains the Wi-Fi commissioning service, which exposes four characteristics: **Wi-Fi Control (write)**, **Wi-Fi Configure (write)**, **Wi-Fi Access Point List (notification)**, **Monitoring (Notification)** to control and monitor a Wi-Fi connection.
- Is the GATT server

Client Collector: (STBLE Toolbox/ ST Web Bluetooth page)

- Accesses the information exposed by the Wi-Fi commissioning application, **configures and controls the Wi-Fi connection** with the write characteristics, **monitors the Wi-Fi connection status** by receiving notifications from it
- Is the GATT client



The table below describes the structure of the commissioning service:

Bluetooth® LE commissioning service specification		
Service	Characteristic	Mode
Wi-Fi commissioning		
	Wi-Fi Control	Write with Response/Read
	Wi-Fi Configure	Write with Response/Read
	Wi-Fi AP List	Notify
	Monitoring	Notify

Wi-Fi commissioning - Wi-Fi control	
Byte Index	0
Name	Action
Value	0x01: Wi-Fi Start Scan 0x03: Wi-Fi Connect 0x04: Wi-Fi Disconnect 0x05: Wi-Fi Ping

Wi-Fi control characteristic:
Used to drive the Wi-Fi by launching scans, connecting or disconnecting from a network

Wi-Fi commissioning - Wi-Fi configure

Byte Index	0	1
Name	Type	Data[]
Value	0x01: AP-SSID 0x02: PWD	"ASCII" "ASCII"

Wi-Fi configure characteristic:

Used to setup the Wi-Fi parameters before establishing a connection

Wi-Fi Commissioning - Wi-Fi AP List

Byte Index	0	1	2 to 3	4 to 7	8 to 40
Name	SSID Length	Channel	Signal level	Security Flag	SSID
Value	0x00 ... 0x20	0x00 ... 0xFF	0x00 ... 0xFFFF	Security_Flag[]	Data[]

Wi-Fi access point list characteristic:

Used to notify information about scanned access points

Wi-Fi commissioning - Wi-Fi monitoring

Byte Index	0	1 up to 239
Name	Type	Data[]
Value	0x03: Connecting 0x04: Connection established 0x05: Ping response 0x06: Error	X SSID Refer to table below 0x01: Connection Timeout

Monitoring characteristic:

Used to notify information about Wi-Fi network.



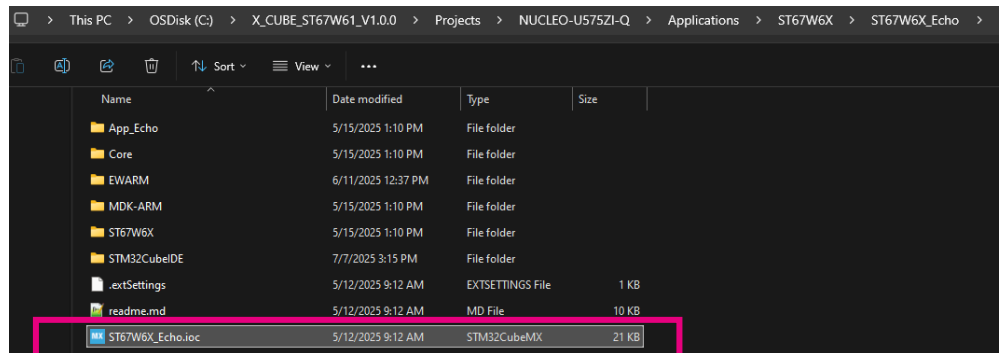
life.augmented

Generating BLE Commissioning
app using NUCLEO-U575ZI-Q as
host processor (Hands-on)

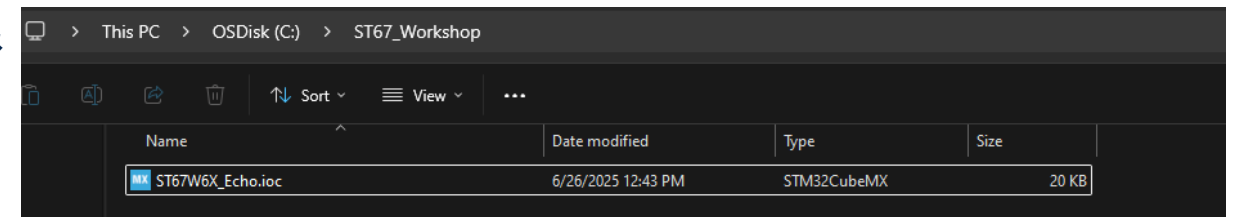
Case 1: Generating BLE Commissioning app using NUCLEO-U575ZI-Q as host processor

We would start with the ST67W6X_Echo.ioc file as a starting point and change the configuration to generate the BLE Commissioning application from the STM32CubeMX.

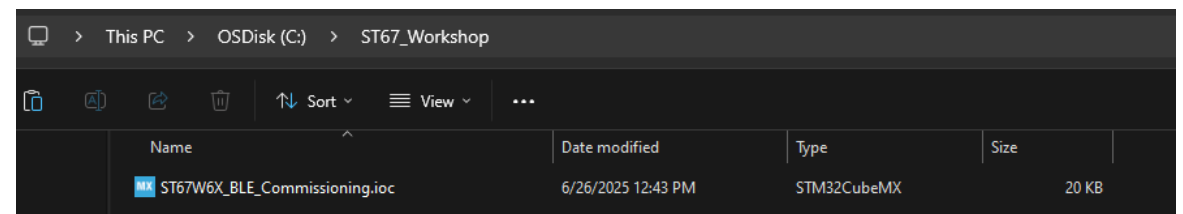
1. Browse through C:\X_CUBE_ST67W61_V1.0.0\Projects\NUCLEO-U575ZI Q\Applications\ST67W6X\ST67W6X_Echo
2. Copy ST67W6X_Echo.ioc
3. Create a new directory C:\ST67_Workshop
4. Paste and rename to ST67W6X_BLE_Commissioning.ioc



Copy & Paste



Rename

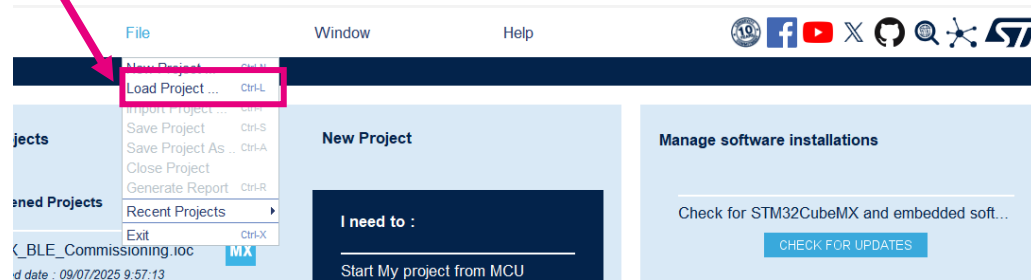


Loading ioc file in STM32CubeMX

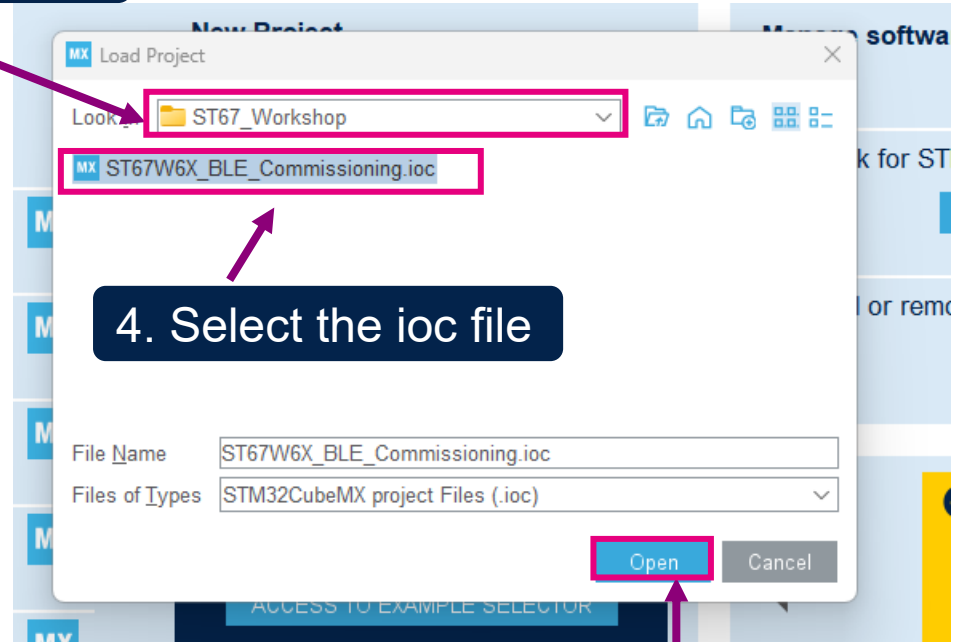
3. Go the ST67_Workshop

1. Run STM32CubeMX with admin rights

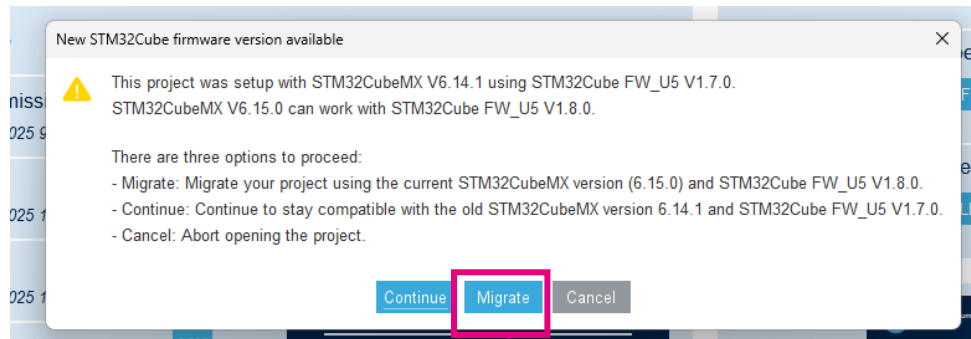
2. Load Project



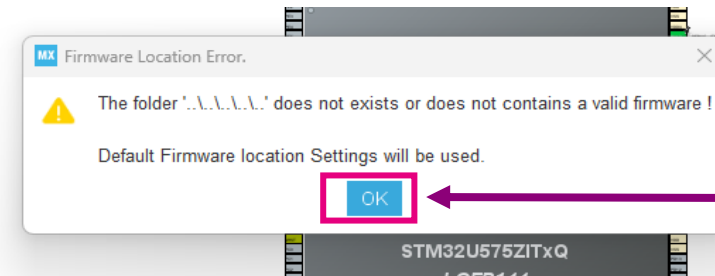
4. Select the ioc file



5. Click open



6. Select Migrate



7. Click Ok

Configuration

2. Select BLE Commissioning app

MX Software Packs Component Selector

Packs

Pack / Bundle / Component	Status	Version	Selection
✓ STMicroelectronics.X-CUBE-ST67W61	✓	1.0.0	
✓ Device Applications	✓	1.0.0	
Application	✓	1.0.0	BLE_Commissioning_v
FreeRTOS_Tickless	✓	1.0.0	
✓ Network ST67W6X_Network_Driver	✓	1.0.0	BLE_Commissioning_v
ServiceShell		1.0.0	
ServiceAPI	✓	1.0.0	
Driver / W61_at_and_bus	✓	1.0.0	
✓ Utilities	✓		
Logging	✓	1.0.0	☒
Shell		1.0.0	☐
Statistics		1.0.0	☐
✓ Debug TraceRecorder		4.10.2	
TraceRecorder		4.10.2	☐
✓ Data Exchange cJSON		1.7.18	
cJSON		1.7.18	☐
✓ File System LittleFS		2.10.1	
LittleFS		2.10.1	☐
✓ Utility Utilities	✓	1.4.2	
LPM / Tiny LPM	✓	1.4.2	☒
> STMicroelectronics.X-CUBE-SUBG2		5.0.0	Install
> STMicroelectronics.X-CUBE-TCPP		4.2.0	Install
> STMicroelectronics.X-CUBE-TOF1		3.4.3	Install
> STMicroelectronics.X-CUBE-TOUCHGFX		4.25.0	Install

1. Select Components

Commissioning.Ioc - Pinout & Configuration

LPBAM Scenario & Configuration

Clock Configuration

Pinout

Software Packs

Select Components

Manage Software Packs

Pinout view

PE2

PE3

PE4

LED_BLUE

PEUS_JTDS-SMO

Configuration

3. Under Middleware and Software Packs

4. X-CUBE-ST67W61

5. Rename W6X_BLE_HOSTNAME

6. Disable W6X_POWER_SAVE_AUTO

STM32CubeIDE Configuration Interface for X-CUBE-ST67W61.1.0.0 Mode and Configuration

Middleware and Software Packs

- AIROC-Wi-Fi-Bluetooth-STM32
- FILEX
- FP-SNS-FLIGHT1
- FP-SNS-MOTENV1
- FP-SNS-MOTENVWB1
- FP-SNS-SMARTAG2
- FP-SNS-STAIOTCFT
- FP-SNS-STBOX1
- I-CUBE-CANOPEN
- I-CUBE-Cesium
- I-CUBE-FS-RTOS
- I-CUBE-ITTIADB
- I-CUBE-Mongoose
- I-CUBE-embOS
- I-CUBE-wolfMQTT
- I-CUBE-wolfSSH
- I-CUBE-wolfSSL
- I-CUBE-wolfTPM
- I-Cube-SoM-uGOAL
- LEVELX
- NETXDUO
- THREADX
- TOUCHSENSING
- USBPD
- USBX
- X-CUBE-AI
- X-CUBE-ALGOBUILD
- X-CUBE-ALS
- X-CUBE-BLE1
- X-CUBE-BLE2
- X-CUBE-BLEMGR
- X-CUBE-DISPLAY
- X-CUBE-DPower
- X-CUBE-EEPROMA1
- X-CUBE-FREERTOS**
- X-CUBE-GNSS1
- X-CUBE-IPS
- X-CUBE-ISPU
- X-CUBE-IOTC-DA16k-PMOD
- X-CUBE-MEMS1
- X-CUBE-NFC4
- X-CUBE-NFC6
- X-CUBE-NFC7
- X-CUBE-NFC9
- X-CUBE-NFC10
- X-CUBE-NFC12
- X-CUBE-PM33A1
- X-CUBE-RT-Thread_Nano
- X-CUBE-SAFE1
- X-CUBE-SFXS2LP1
- X-CUBE-SMBUS
- X-CUBE-ST67W61

Configuration

Reset Configuration

W6X Modules | Parameter Settings | User Constants | Platform Settings

Configure the below parameters :

Search (Ctrl+F)

Basic Parameters

- DEBUGGER_ON: ON
- LOG_OUTPUT_MODE: LOG_OUTPUT_UART
- LOW_POWER_MODE: LOW_POWER_SLEEP_ENABLE
- FREERTOS_TASK_NOTIF_ARRAY_LEN: 8

Logging-Shell

- LOG_LEVEL: LOG_DEBUG

W6X Parameters

- W6X_BLE_HOSTNAME: TEST_BLE
- W6X_WIFI_HOSTNAME: ST67W61_WIFI
- W6X_POWER_SAVE_AUTO: No
- W6X_CLOCK_MODE: Internal RC oscillator
- W6X_WIFI_AUTOCONNECT: No
- W6X_WIFI_DHCP_MODE: DHCP client STA
- W6X_WIFI_COUNTRY_CODE: 00
- W6X_WIFI_ADAPTIVE_COUNTRY_CODE: No
- W6X_NET_RECV_TIMEOUT (ms): 10000
- W6X_NET_SEND_TIMEOUT (ms): 10000
- W6X_NET_RECV_BUFFER_SIZE: 9216
- W6X_WIFI_DNS_MANUAL: No
- W6X_WIFI_DNS_IP_1: {208, 67, 222, 222}
- W6X_WIFI_DNS_IP_2: {8, 8, 8, 8}
- W6X_WIFI_DNS_IP_3: {0, 0, 0, 0}

W61 drv Parameters

- W61_WIFI_MAX_DETECTED_AP: 50
- W61_BLE_MAX_CONN_NBR: 1
- W61_BLE_MAX_DETECTED_PERIPHERAL: 10
- W61_BLE_MAX_SERVICE_NBR: 5
- W61_BLE_MAX_CHAR_NBR: 5

Project Settings

home > STM32U575ZITxQ - NUCLEO-U575ZL-Q > ST67W6X_BLE_Commissioning.loc - Project Manager > LPBAM Scenario & Configuration > GENERATE CODE

Pinout & Configuration | Clock Configuration | Project Manager

Project

Code Generator

Advanced Settings

Project Settings

Project Name: ST67W6X_BLE_Commissioning

Project Location: C:\ST67_Workshop\ Browse

Application Structure: Advanced ☐ Do not generate the main()

Toolchain Folder Location: C:\ST67_Workshop\ST67W6X_BLE_Commissioning\

Toolchain / IDE: STM32CubeIDE ☒ Generate Under Root

Linker Settings

Minimum Heap Size: 0x200

Minimum Stack Size: 0x400

Thread-safe Settings

CortexM33

☐ Enable multi-threaded support

Thread-safe Locking Strategy: Default - Mapping suitable strategy depending on RTOS selection.

Mcu and Firmware Package

Mcu Reference: STM32U575ZITxQ

Firmware Package Name and Version: STM32Cube_FW_U5_V1.8.0

☒ Use Default Firmware Location

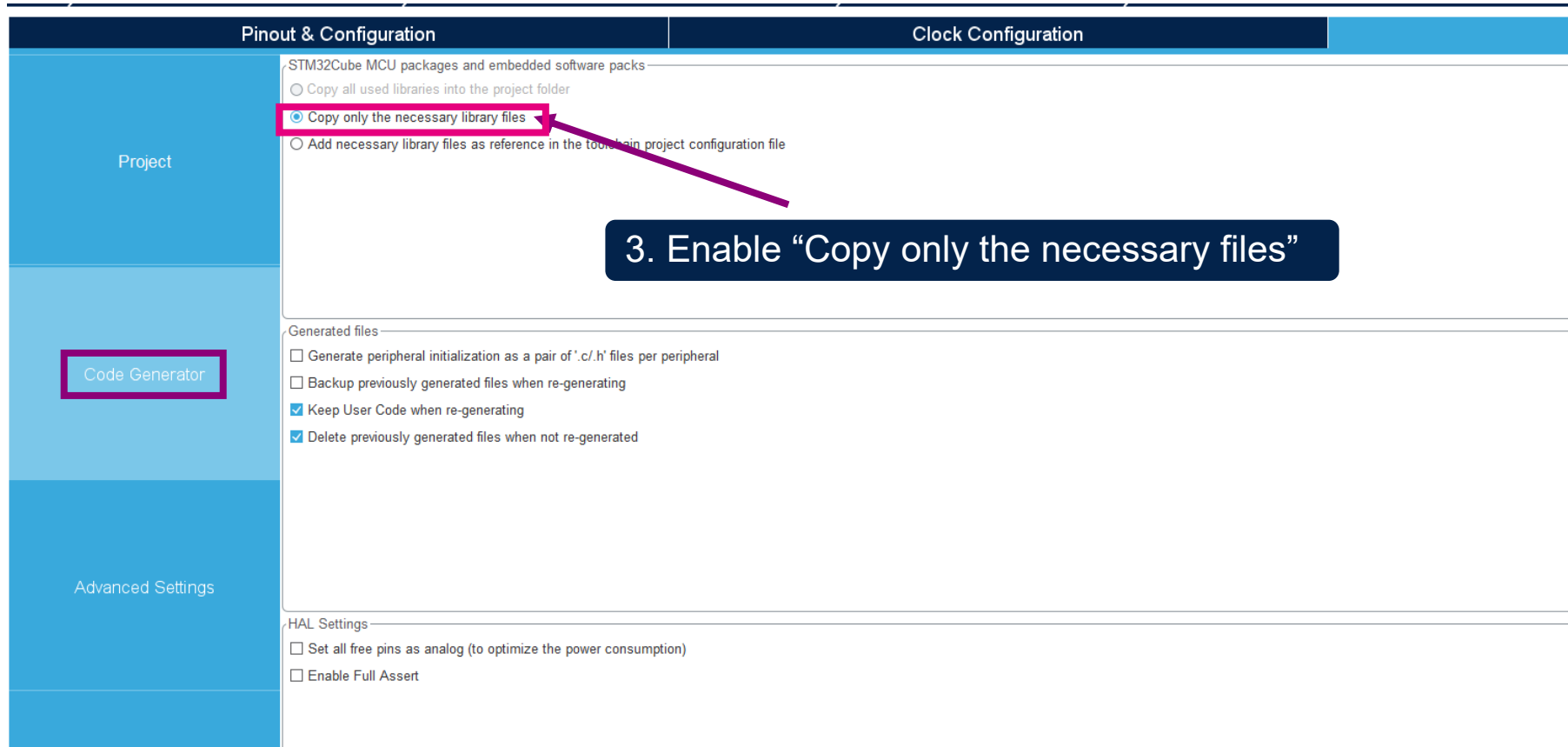
Firmware Relative Path: C:\Users\kulkarni\STMicroelectronics\Library\STM32Cube_FW_U5_V1.8.0 Browse

Here, we are enabling the Flat directory structure: (i.e. Driver and MW directories copied into your project directory)

1. Enable "Generate Under Root"

2. Enable "Use Default Firmware Location"

Project Settings cont..



Pinout & Configuration

Clock Configuration

Project

Code Generator

Advanced Settings

STM32Cube MCU packages and embedded software packs

- ☐ Copy all used libraries into the project folder
- ☒ Copy only the necessary library files
- ☐ Add necessary library files as reference in the toolchain project configuration file

Generated files

- ☐ Generate peripheral initialization as a pair of '.c/.h' files per peripheral
- ☐ Backup previously generated files when re-generating
- ☒ Keep User Code when re-generating
- ☒ Delete previously generated files when not re-generated

HAL Settings

- ☐ Set all free pins as analog (to optimize the power consumption)
- ☐ Enable Full Assert

3. Enable "Copy only the necessary files"

Generate Code

Home > STM32U575ZITxQ - NUCLEO-U575ZI-Q > ST67W6X_BLE_Commissioning.ioc - Project Manager > LPBAM Scenario & Configuration > **GENERATE CODE** > CHECK LPE

Pinout & Configuration | Clock Configuration | Project Manager | Tools

Project

Code Generator

Advanced Settings

Driver Selector

Search (Ctrl+F)

CORTEX_M33_NS
PWR
RCC
GPIO
> GPDMA
ICACHE
> SPI
> USART
> LPTIM
STMicroelectronics.X-CUBE-ST67W61.1.0.0

HAL
HAL
HAL
HAL
HAL
HAL
HAL
HAL
HAL
HAL

Generated Function Calls

Generate Code	Rank	Function Name	Peripheral Instance Name	☐ Do Not Generate Function Call	☐ Visibility (Static)
☒	1	SystemClock_Config	RCC	☐	☐
☒	2	MX_GPIO_Init	GPIO	☐	☒
☒	3	MX_GPDMA1_Init	GPDMA1	☐	☒
☒	4	MX_ICACHE_Init	ICACHE	☐	☒
☒	5	MX_SPI1_Init	SPI1	☐	☒
☒	6	MX_USART1_UART_Init	USART1	☐	☒
☒	7	MX_LPTIM1_Init	LPTIM1	☐	☒
☒	10	MX_App_BLE_Commissioning_Init	STMicroelectronics.X-CUBE-ST67W61.1.0.0	☒	☐

5. Generate Code

4. Enable this option
(here: we are not generating this function call)

STM32CubeIDE

Name	Date modified	Type	Size
App_BLE_Commissioning	7/10/2025 12:05 PM	File folder	
Core	7/10/2025 12:06 PM	File folder	
Drivers	7/10/2025 12:06 PM	File folder	
Middlewares	7/10/2025 12:06 PM	File folder	
ST67W6X	7/10/2025 12:05 PM	File folder	
Utilities	7/10/2025 12:06 PM	File folder	
.cproject	7/10/2025 12:06 PM	CPROJECT File	35 KB
.mxproject	7/10/2025 12:06 PM	MXPROJECT File	17 KB
.project	7/10/2025 12:06 PM	PROJECT File	2 KB
ST67W6X_BLE_Commissioning.ioc	7/10/2025 12:05 PM	STM32CubeMX	21 KB
STM32U575ZITXQ_FLASH.ld	7/10/2025 12:06 PM	LD File	5 KB
STM32U575ZITXQ_RAM.ld	7/10/2025 12:06 PM	LD File	5 KB

After, generating the code, you would view similar folder structure under ST67_Workshop

Load the project on the STM32CubeIDE

```
152 /* USER CODE END RTOS_THREADS */
153
154 /* USER CODE BEGIN RTOS_EVENTS */
155 /* add events, ... */
156 /* USER CODE END RTOS_EVENTS */
157
158 }
159
160 /* USER CODE BEGIN Header_StartDefaultTask */
161 /**
162  * Brief Function Implementing the defaultTask thread.
163  * @param argument: Not used
164  * @retval None
165  */
166 /* USER CODE END Header_StartDefaultTask */
167 void StartDefaultTask(void *argument)
168 {
169 /* USER CODE BEGIN defaultTask */
170     main_app();
171     /* Infinite loop */
172     for(;;)
173     {
174         osDelay(1);
175     }
176 /* USER CODE END defaultTask */
177
178 /* Private application code -----*/
179 /* USER CODE BEGIN Application */
180
181 /* USER CODE END Application */
182
```

Call main_app() function under StartDefaultTask() in app_freertos.c file

Time to build and flash

```
Problems Tasks Console X Properties
CDT Build Console [ST67W6X_BLE_Commissioning]

arm-none-eabi-gcc "-I./App_BLE_Commissioning/Target/freertos_tickless.c" -mcpu=cortex-m33 -std=gnu11 -g3 -DDEBUG -DconfigTASK_NOTIFICATION_ARR
arm-none-eabi-gcc "-I./App_BLE_Commissioning/Target/logshell_ctrl.c" -mcpu=cortex-m33 -std=gnu11 -g3 -DDEBUG -DconfigTASK_NOTIFICATION_ARR
arm-none-eabi-gcc "-I./App_BLE_Commissioning/Target/stm32_lpm_if.c" -mcpu=cortex-m33 -std=gnu11 -g3 -DDEBUG -DconfigTASK_NOTIFICATION_ARR
arm-none-eabi-gcc "-I./App_BLE_Commissioning/App/main_app.c" -mcpu=cortex-m33 -std=gnu11 -g3 -DDEBUG -DconfigTASK_NOTIFICATION_ARRAY_ENTR
arm-none-eabi-gcc -o "ST67W6X_BLE_Commissioning.elf" @"objects.list" -mcpu=cortex-m33 -T"C:\ST67_Workshop\STM32U575ZITXQ_FLASH.ld" --s
Finished building target: ST67W6X_BLE_Commissioning.elf

arm-none-eabi-size ST67W6X_BLE_Commissioning.elf
arm-none-eabi-objdump -h -S ST67W6X_BLE_Commissioning.elf > "ST67W6X_BLE_Commissioning.list"
text data bss dec hex filename
143956 328 208016 352300 5602c ST67W6X_BLE_Commissioning.elf
Finished building: default.size.stdout

Finished building: ST67W6X_BLE_Commissioning.list

12:48:17 Build Finished. 0 errors, 1 warnings. (took 25s.856ms)
```

Console after building the application

```
COM77 - Tera Term VT
File Edit Setup Control Window Help
##### Welcome to ST67W6X Vi-Fi Commissioning over BLE Application #####
# build: 10:55:58 Jul 9 2025
----- Host info -----
Host FW Version: 1.0.0
ST67W6X MW Version: 1.0.0
OT Version: 1.0.0.1
SDK Version: 2.0.75
MAC Version: 1.6.38
Build Date: May 10 2025 10:15:04
Module ID:
BOM ID:
Manufacturing Year: 2000
Manufacturing Week: 00
Battery Voltage: 3.314 V
Trim Vi-Fi hp: 8.8.8.8.8.7.6.6.7.7.7.8.8.8.8
Trim Vi-Fi lp: 8.8.8.8.9.9.9.9.10.10.10.11.11.11
Trim BLE: 7.5.5.7.7
Trim XIAL: 32
MAC Address: 40:82:7b:00:0c:2d
Anti-rollback Bootloader: 0
Anti-rollback App: 0

Vi-Fi init is done
Net init is done
Ble init is done
BD Address: 40:82:7b:00:0c:2e
Configure BLE
BLE configuration is done

BLE Commissioning Service Creation
- BLE service created
- BLE Vi-Fi Control charac created
- BLE Vi-Fi Configure charac created
- BLE Vi-Fi Monitoring charac created
- BLE services and charac registered
BLE service and charac creation is done

Start BLE advertising
BLE advertising is started
BLE connected
-> BLE CONNECTED: Conn_Handle: 0
BLE MTU exchange
BLE notification enabled
-> BLE NOTIFICATION ENABLED [Service: 0, Charac: 21.
BLE write
-> BLE WRITE
-> Conn_Handle: 0, Service: 0, Charac: 0, length 1. Data:
0x01
Vi-Fi Control Charac
Vi-Fi Scan Enable
Vi-Fi Scan done.
MAC : [56:2d:a2:23:5e:ee] Channel: 6 WPA2 : RSSI: -42 : SSID: Shreya Iphone
MAC : [e4:a4:1c:a0:f9:00] Channel: 6 WPA-EAP : RSSI: -48 : SSID: STULAN2
MAC : [e4:a4:1c:a0:f9:01] Channel: 6 WPA-EAP : RSSI: -49 : SSID: STSMARTMOBILE
MAC : [e4:a4:1c:a0:f9:02] Channel: 6 OPEN : RSSI: -49 : SSID: STGUEST
MAC : [e4:a4:1c:a0:f9:03] Channel: 11 OPEN : RSSI: -62 : SSID: STGUEST
MAC : [e4:a4:1c:a0:f9:04] Channel: 11 WPA-EAP : RSSI: -63 : SSID: STULAN2
MAC : [e4:a4:1c:a0:f9:05] Channel: 11 WPA-EAP : RSSI: -63 : SSID: STSMARTMOBILE
MAC : [e4:a4:1c:a0:f9:06] Channel: 1 WPA-EAP : RSSI: -66 : SSID: STULAN2
MAC : [e4:a4:1c:a0:f9:07] Channel: 1 WPA-EAP : RSSI: -66 : SSID: STSMARTMOBILE
MAC : [e4:a4:1c:a0:f9:08] Channel: 1 OPEN : RSSI: -66 : SSID: STGUEST
MAC : [e4:a4:1c:a0:f9:09] Channel: 8 WPA2 : RSSI: -70 : SSID: Netgear 2.4G
MAC : [e4:a4:1c:a0:f9:0a] Channel: 6 WPA-EAP : RSSI: -72 : SSID: STULAN2
```

Tera Term: Serial port setup and connection

Port: COM77

Speed: 921600

Data: 8 bit

Parity: none

Stop bits: 1 bit

Flow control: none

Transmit delay
0 msec/char 0 msec/line

Device Friendly Name: STMicroelectronics STLink Virtual COM P
Device Instance ID: USB\VID_0483&PID_374E&MI_02\6&2812CE8
Device Manufacturer: STMicroelectronics
Provider Name: STMicroelectronics
Driver Date: 4-1-2021
Driver Version: 2.2.0.0

Tera Term logs after flashing the BLE Commissioning application



life.augmented

Porting to a new host processor
using STM32CubeMX

Porting over new host processor(1/3)

1. Select the Host MCU

Choose the target STM32 part number from the STM32CubeMX

2. Check schematics of Host MCU and ST67

Review pin mappings for SPI, GPIO, USART

3. Configure USART for Debugging

Used for debugging logs via STLink virtual port

4. Configure the SPI Interface for Host MCU and ST67 communication

Set SPI pins, SPI mode

5. Configure DMA Channels for SPI

Fast and reliable SPI data transfer, reducing CPU load during transmission



Porting over new host processor(2/3)

6. **Configure GPIO Pins** (CHIP_EN, SPI_RDY, SPI_CS, BOOT, User_Button)

Set GPIO Mode, GPIO output level, GPIO Maximum Output speed

7. **Configure SYS** (Time base Source)

8. **Configure Clock**

9. **Enable FreeRTOS Middleware**

Setup default task, heap size, stack size and required priorities

10. **Enable X-CUBE-ST67W61**

Setup Application, W61 Drivers, Utilities

11. **Configure NVIC Interrupt Settings**

SPI, DMA, USART, EXTI



Porting over new host processor(3/3)

12. Project Creation and Code Generation

Generate project files (STM32CubeIDE/IAR)

Open in IDE for application development

13. Application Entry: Call main_app()

Inside StartDefaultTask, invoke main_app()

This initializes ST67W6X Driver, ST67W6X Wi-Fi module, ST67W6X Network module, Application specific functions

14. Build and Flash the application

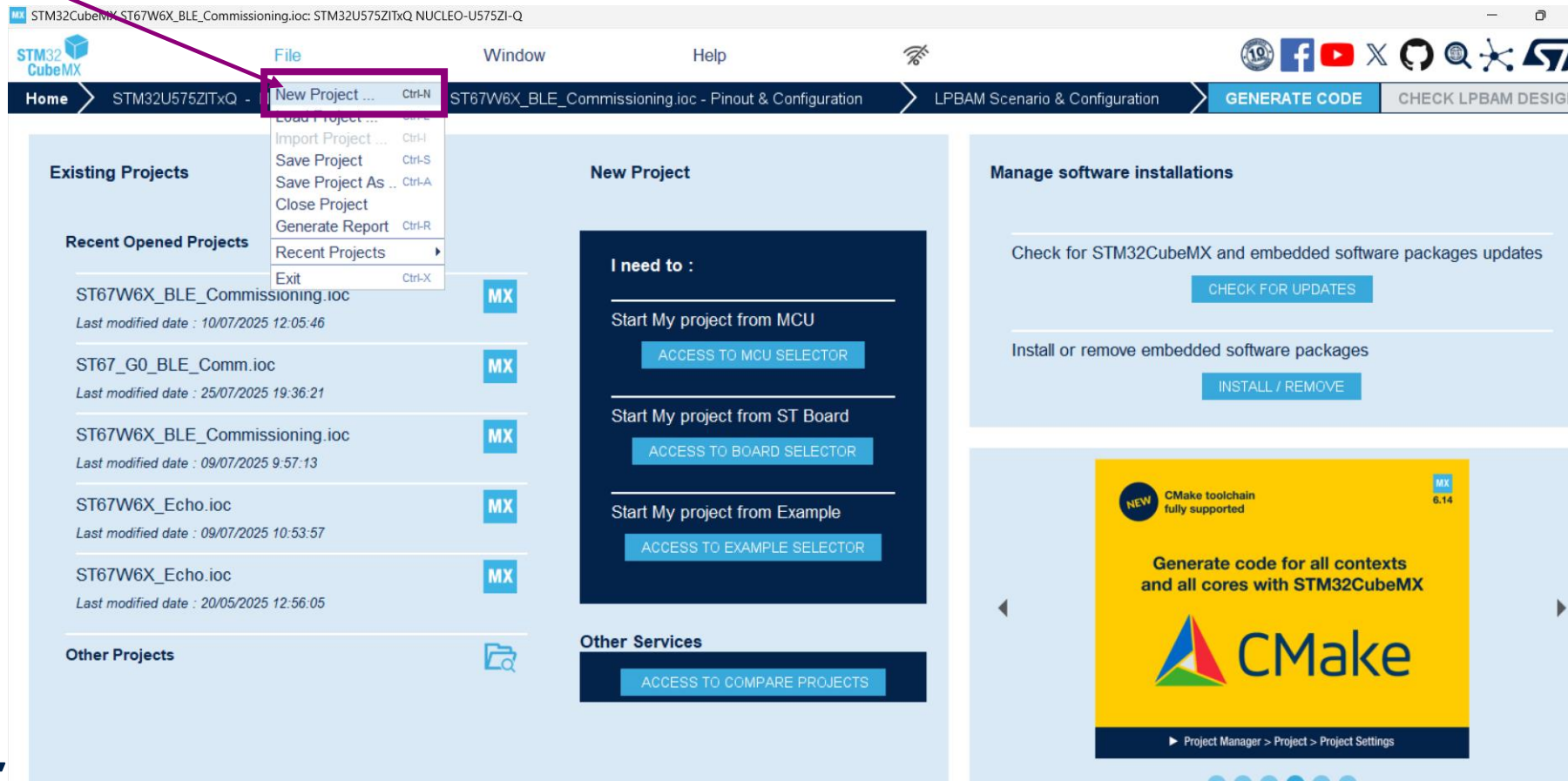


life.augmented

Generating BLE Commissioning
app using NUCLEO-G0B1RE as
host processor (Walk-through)

Case 3: Generating BLE Commissioning app using NUCLEO-G0B1RE as host processor

Open STM32CubeMX and create the new project.



Select the board selector

Type STM32G0

Select STM32G0

Select NUCLEO-G0B1RE

[Start Project](#)



*	Overview	Commercial Part No	Type	Marketing Status	Unit Price (USD)	Mounted Device
☆		NUCLEO-G031KB	Nucleo-32	Active	10.32	STM32G031KBT6
☆		NUCLEO-G070RB	Nucleo-64	Active	10.32	STM32G070RBT6
☆		NUCLEO-G071RB	Nucleo-64	Active	10.32	STM32G071RBT6
☆		NUCLEO-G0B1RE	Nucleo-64	Active	10.32	STM32G0B1RET6

MCU/MPU Selector

Board Selector

Example Selector

Cross Selector

Board Filters

Commercial Part Number

STM32G0

+

-

PRODUCT INFO

Type

Supplier

MCU / MPU Series

Check/Uncheck All (1)

STM32C0

STM32F0

STM32F1

STM32F2

STM32F3

STM32F4

STM32F7

STM32G0

STM32G4

STM32H5

STM32H7

STM32L0

STM32L1

STM32L4

STM32L4+

STM32L5

Features

Large Picture

Docs & Resources

Datasheet

Buy

Start Project

STM32G0 Series

NUCLEO-G0B1RE

ACTIVE

Product is in mass production

Part Number : NUCLEO-G0B1RE

Commercial Part Number : NUCLEO-G0B1RE

Unit Price (US\$) : 10.32

Mounted Device : [STM32G0B1RET6](#)



The STM32 Nucleo-64 board provides an affordable and flexible way for users to try out new concepts and build prototypes by choosing from the various combinations of performance and power consumption features provided by the STM32 microcontroller. For the compatible boards, the internal or external SMPS significantly reduces power consumption in Run mode.

ARDUINO® Uno V3 connectivity support and the ST morpho headers allow the easy integration of the functionality of the STM32 Nucleo open development platform with a wide choice of specialized shields.

The STM32 Nucleo-64 board does not require any separate probe as it integrates the

Board Project Options: NUCLEO-G0B1RE

Initialize all peripherals with their default Mode ?

Yes No

Boards List: 8 items

Export

	Overview	Commercial Part No	Type	Marketing Status	Unit Price (US\$)	Mounted Device
☆		NUCLEO-G070RB	Nucleo-64	Active	10.32	STM32G070RBT6
☆		NUCLEO-G071RB	Nucleo-64	Active	10.32	STM32G071RBT6
☆		NUCLEO-G0B1RE	Nucleo-64	Active	10.32	STM32G0B1RET6
☆		STM32G0316-DISCO	Discovery Kit	Active	9.89	STM32G0316M6

Select Yes



 life.augmented

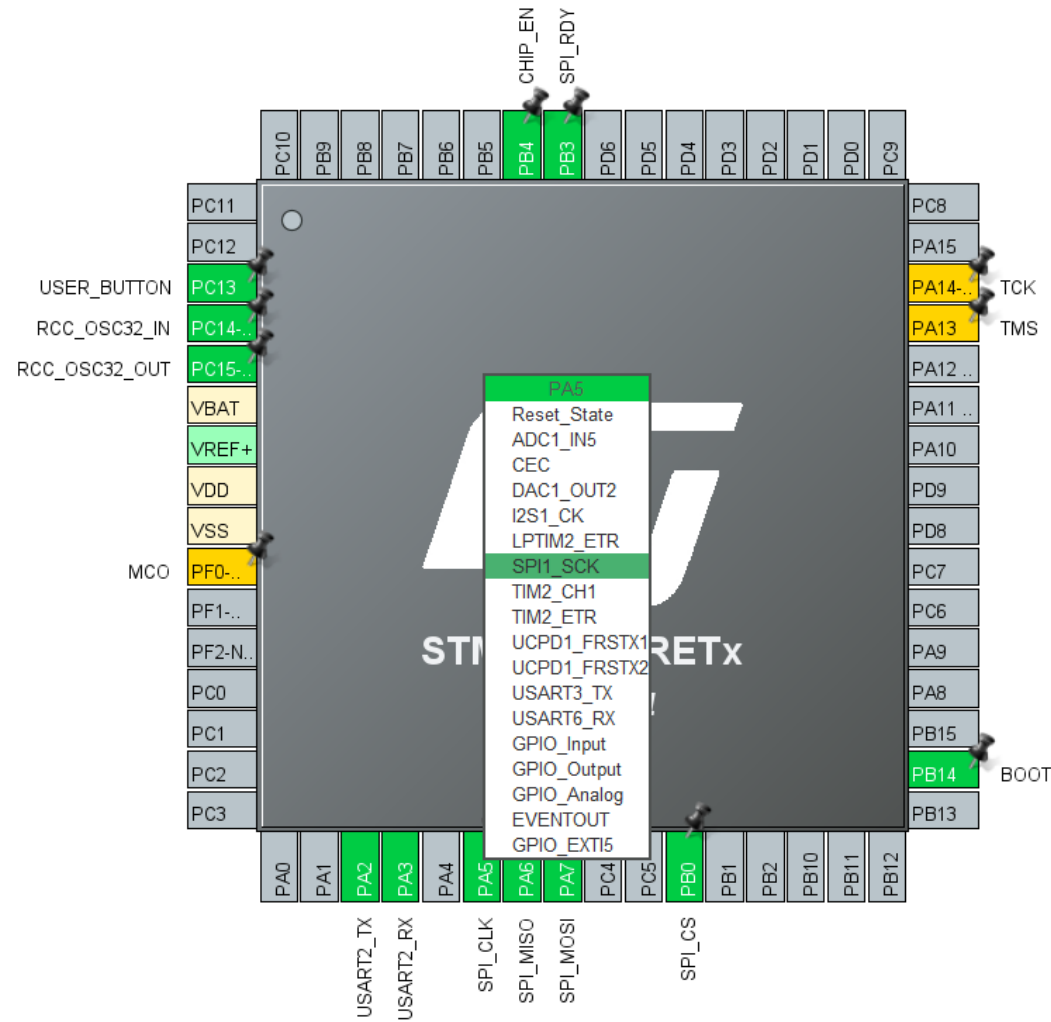
Pinout

When generating an application from scratch and if X-Nucleo-67W61M1 is used, the pinout must be set in accordance with the ST67W61M1 Arduino interfaces.

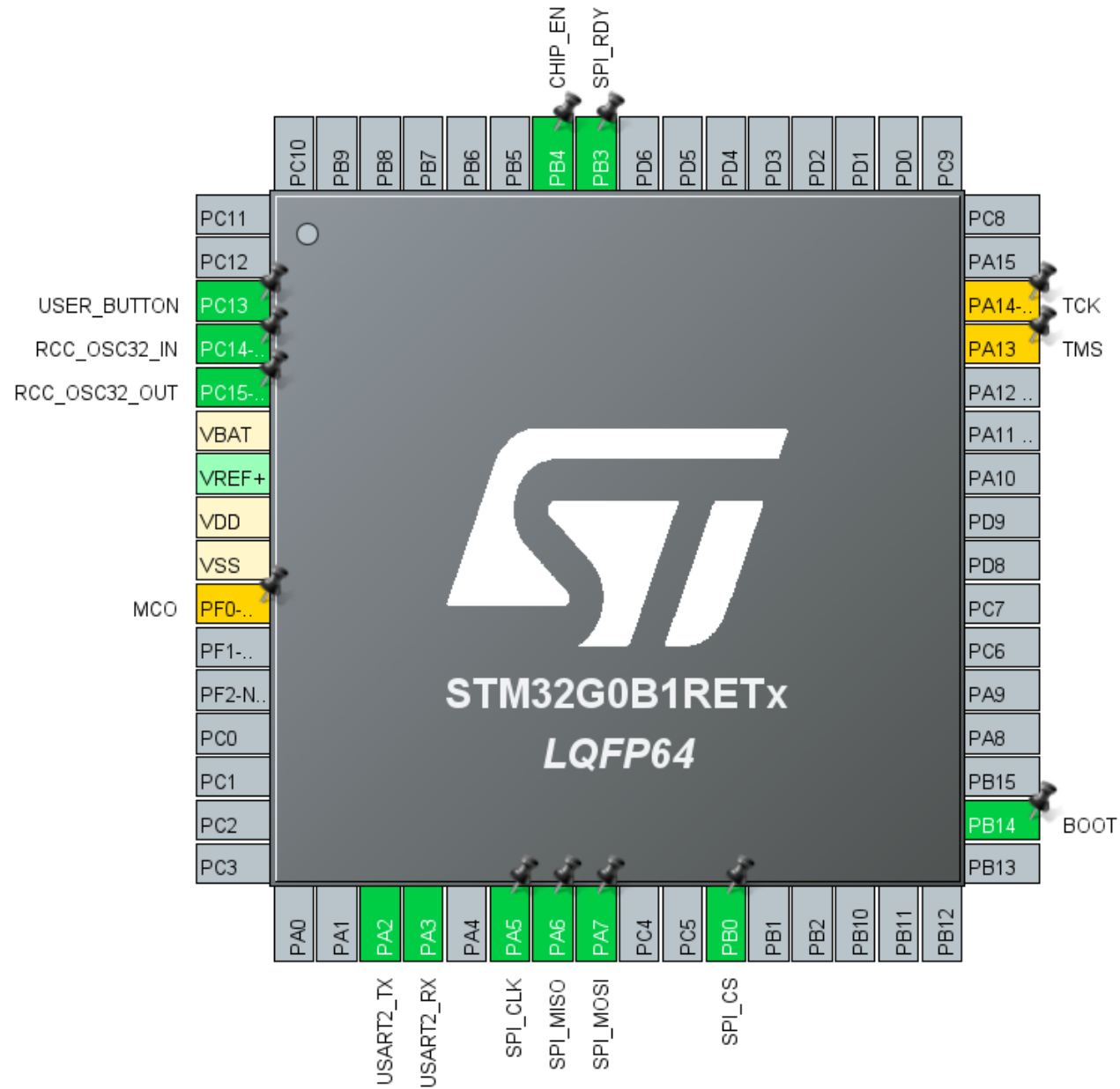
Pin function	Arduino Connector	STM32G0 Pin	GPIO mode	GPIO pull mode	GPIO speed
SPI_CLK	CN5.D13	PA5	AF PP	No Pull	Very high
SPI_MISO	CN5.D12	PA6	AF PP	No Pull	Very high
SPI_MOSI	CN5.D11	PA7	AF PP	No Pull	Very high
SPI_CS	CN5.D10	PB0	Output PP	No Pull	high
USER_BUTTON	-	PC13	EXTI Falling	Pull-up	N/A
CHIP_EN	CN9.D5	PB4	Output PP	No Pull	High
BOOT	CN9.D6	PB14	Output PP	No Pull	Low
SPI_RDY	CN9.D3	PB3	EXTI Falling/Rising	No Pull	N/A

Assigning functionality

For assigning, click on the pin and select functionality.
(e.g) PA5->SPI_SCK



Pinout View



After, assigning all the pins mentioned from the pin out table, this would be overall pinout view

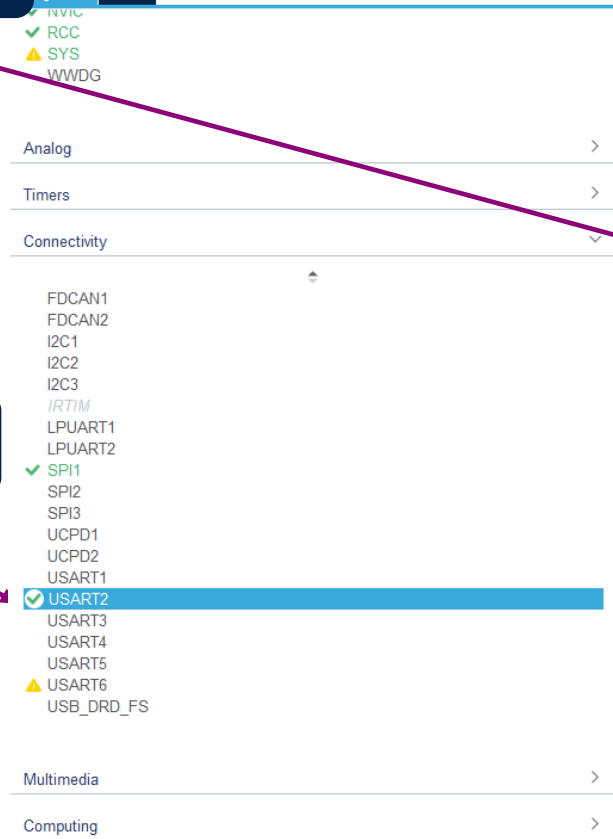
UART GPIO(USART 2)

Parameter Settings

PA2: USART2_TX
PA3: USART2_RX

Select USART2

GPIO Settings



Mode Asynchronous

Hardware Flow Control (RS232) Disable

☐ Hardware Flow Control (RS485)

Slave Select(NSS) Management Disable

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

Basic Parameters

Baud Rate 115200 Bits/s

Word Length 8 Bits (including Parity)

Parity None

Stop Bits 1

Advanced Parameters

Data Direction Receive and Transmit

Over Sampling 16 Samples

Single Sample Disable

ClockPrescaler 1

Fifo Mode Disable

Txfifo Threshold 1 eighth full configuration

Rxfifo Threshold 1 eighth full configuration

Advanced Features

Auto Baudrate Disable

TX Pin Active Level Inversion Disable

RX Pin Active Level Inversion Disable

Data Inversion Disable

TX and RX Pins Swapping Disable

Overrun Enable

DMA on RX Error Enable

MSB First Disable

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pins

Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull-down	Maximum out...	Fast Mode	User Label	Modified
PA2	USART2_TX	Low	Alternate Functio...	No pull-up and no pull-down	Low	n/a		☐
PA3	USART2_RX	Low	Alternate Functio...	No pull-up and no pull-down	Low	n/a		☐

PA5: SPI_SCK
PA6: SPI_MISO
PA7: SPI_MOSI

SPI GPIO(SPI1)

Select SPI1

Home > STM32G0B1RETx - NUCLEO-G0B1RE > ST67_G0_BLE_Comm.loc - Pinout & Configuration

Pinout & Configuration | Clock Configuration | Project Manager

Software Packs | Pinout

SPI1 Mode and Configuration

Mode

Mode: Full-Duplex Master

Hardware NSS Signal: Disable

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | DMA Settings | GPIO Settings

Search Signals

Search (Ctrl+F)

Show only Modified Pins

Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull...	Maximum output ...	Fast Mode	User Label	Modified
PA5	SPI1_SCK	Low	Alternate Functio...	No pull-up and no...	Very High	n/a	SPI_SCK	✓
PA6	SPI1_MISO	Low	Alternate Functio...	No pull-up and no...	Very High	n/a	SPI_MISO	✓
PA7	SPI1_MOSI	Low	Alternate Functio...	No pull-up and no...	Very High	n/a	SPI_MOSI	✓

Parameter Settings

SPI1 Mode and Configuration

Mode

Mode: Full-Duplex Master

Hardware NSS Signal: Disable

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | DMA Settings | GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

Basic Parameters

Frame Format: Motorola

Data Size: 8 Bits

First Bit: MSB First

Clock Parameters

Prescaler (for Baud Rate): 2

Baud Rate: 32.0 MBits/s

Clock Polarity (CPOL): Low

Clock Phase (CPHA): 1 Edge

Advanced Parameters

CRC Calculation: Disabled

NSSP Mode: Enabled

NSS Signal Type: Software

DMA Channels

Select DMA

The screenshot shows the STM32CubeMX software interface for configuring DMA channels. The left sidebar lists various system components, with 'DMA' selected. The main window displays the 'DMA Mode and Configuration' tab, which includes a table of DMA requests and a configuration section below.

DMA Request	Channel	Direction	Priority
SPI1_TX	DMA1 Channel 1	Memory To Peripheral	High
SPI1_RX	DMA1 Channel 2	Peripheral To Memory	Low

Below the table, there are buttons for 'Add' and 'Delete'. The 'DMA Request Settings' section includes a 'Mode' dropdown set to 'Normal', an 'Increment Address' checkbox, and 'Data Width' dropdowns set to 'Byte'. The 'DMA Request Synchronization Settings' section includes checkboxes for 'Enable synchronization' and 'Enable event', and dropdowns for 'Synchronization signal', 'Signal polarity', and 'Request number'.

SPI1_TX -> DMA1 CHANNEL 1->High
SPI1_RX -> DMA1 CHANNEL 2-> High

GPIO

Select GPIO

Categories

A-Z

System Core

✓ DMA

✓ GPIO

IWDG

✓ NVIC

✓ RCC

▲ SYS

WWDG

Analog

>

Timers

>

Connectivity

>

Multimedia

>

Computing

>

Middleware and Software Packs

>

Utilities

>

GPIO output level

GPIO Mode and Configuration

Mode

Configuration

Group By Peripherals

✓ GPIO

✓ Single Mapped Signals

✓ RCC

✓ SPI

✓ USART

✓ NVIC

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pins

Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-u...	Maximum outp...	Fast Mode	User Label	Modified
PB0	n/a	Low	Output Push Pull	No pull-up a...	High	n/a	SPI_CS	✓
PB3	n/a	n/a	External Interrupt Mode with Ri...	No pull-up a...	n/a	n/a	SPI_RDY	✓
PB4	n/a	Low	Output Push Pull	No pull-up a...	High	n/a	CHIP_EN	✓
PB14	n/a	Low	Output Push Pull	No pull-up a...	Low	n/a	BOOT	✓
PC13	n/a	n/a	External Interrupt Mode with F...	Pull-up	n/a	n/a	USER_BUTTON	✓

GPIO pin names

GPIO mode

GPIO maximum speed

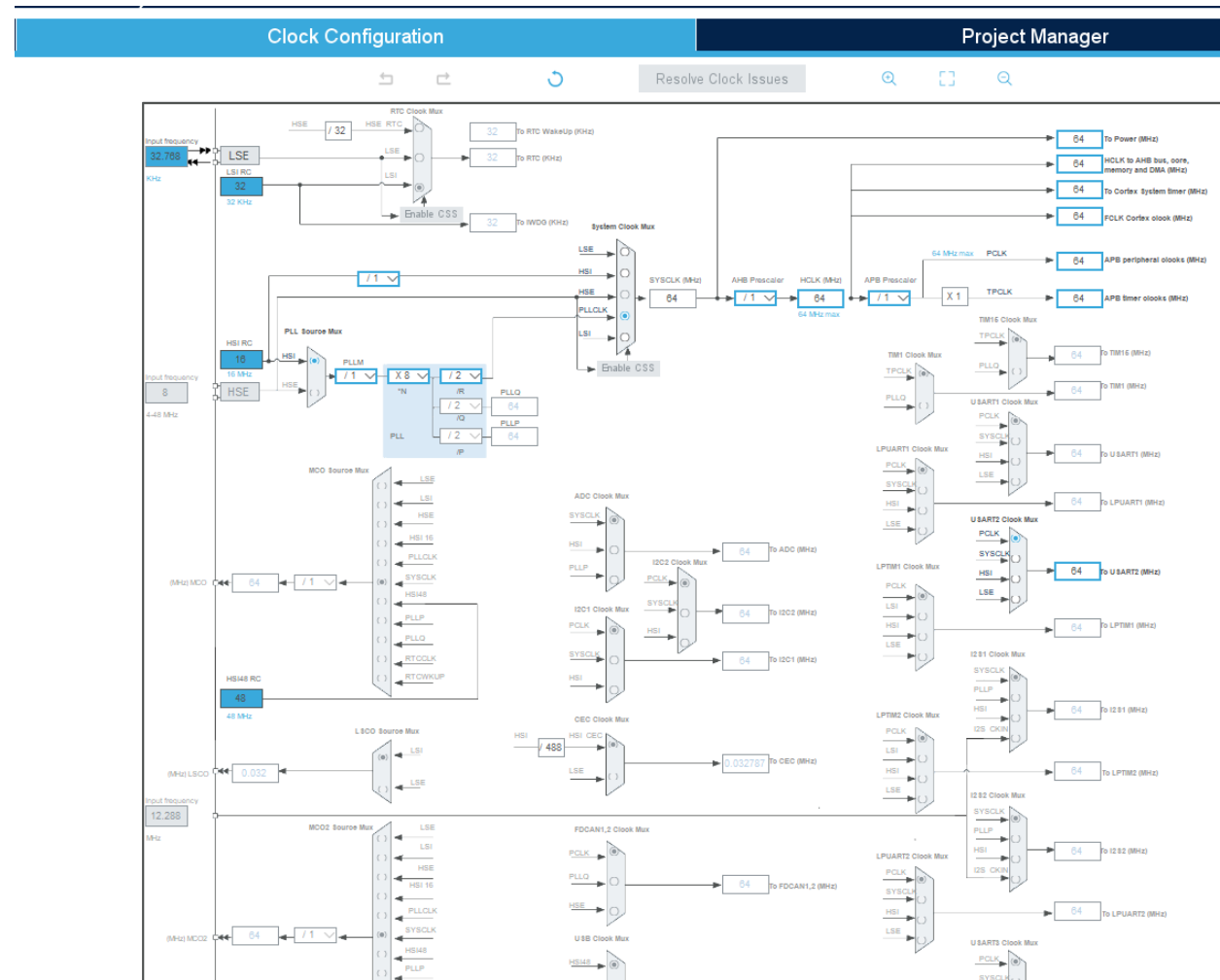
GPIO user level

Timer

The screenshot displays the 'SYS Mode and Configuration' window in the IDE. On the left, a sidebar shows a tree view of system components: System Core (with sub-items DMA, GPIO, IWDG, NVIC, RCC, SYS, and WWDG), Analog, Timers, Connectivity, Multimedia, and Computing. The 'SYS' component is selected. The main window, titled 'SYS Mode and Configuration', shows various system settings. The 'System Wake-Up 2' option is highlighted with a pink bar. Below it, the 'Timebase Source' dropdown is set to 'TIM1'. A purple arrow points from a text box at the bottom to this dropdown. Other settings include 'Serial Wire' (disabled), 'System Wake-Up 1' (disabled), 'System Wake-Up 4' (disabled), 'System Wake-Up 5' (disabled), 'System Wake-Up 6' (disabled), 'Power Voltage Detector In' (Disable), 'VREFBUF Mode' (Disable), 'Pin PA9 instead of pin PA11' (disabled), 'Pin PA10 instead of pin PA12' (disabled), and 'save power of non-active UCPD - deactive Dead Battery pull-up' (checked).

Sys mode configuration: TIM1 in place of systick

- In the "Clock Configuration" panel, system clock is set to the maximum frequency of 64 MHz, which increases the SPI clock speed to 32 MHz.



Middleware & Software Packs

FreeRTOS

- FreeRTOS component can be either be selected from Middleware (FREERTOS) or from the Software Pack (X-CUBE-FREERTOS) depending on the chosen STM32.
- By default, the stack size of the default task defined by STM32CubeMx tool is equal to 128 words. The application default task required **512 words**.

The screenshot shows the STM32CubeMX interface with the 'FREERTOS Mode and Configuration' window open. The 'Interface' is set to 'CMSIS_V2'. The 'Configuration' section shows various options checked, including 'User Constants', 'Tasks and Queues', 'Timers and Semaphores', 'Mutexes', 'Events', and 'FreeRTOS Heap Usage'. The 'Tasks' table is displayed with the following data:

Task Name	Priority	Stack Size (Words)	Entry Function	Code Generation	Parameter	Allocation	Buffer Name	Control Block Na...
defaultTask	osPriorityNormal	512	StartDefaultTask	Default	NULL	Dynamic	NULL	NULL

An 'Edit Task' dialog box is open, showing the configuration for the 'defaultTask'. The 'Stack Size (Words)' field is highlighted with a red box and contains the value '512'. Other fields include 'Task Name' (defaultTask), 'Priority' (osPriorityNormal), 'Entry Function' (StartDefaultTask), 'Code Generation Option' (Default), 'Parameter' (NULL), 'Allocation' (Dynamic), 'Buffer Name' (NULL), and 'Control Block Name' (NULL). The 'OK' and 'Cancel' buttons are at the bottom of the dialog.

Middleware & Software Packs

FreeRTOS

- Adjust heap :The size heap value should be set at least to 40Kbytes. To optimize the memory, this value could be adjusted during integration and validation tests of the application

The screenshot displays the STM32CubeIDE interface for configuring FreeRTOS. On the left, the 'Middleware and Software Packs' list includes various packs, with 'FREERTOS' selected. The right pane shows the 'FREERTOS Mode and Configuration' dialog, specifically the 'Configuration' tab. A red box highlights the 'TOTAL_HEAP_SIZE' parameter, which is set to '40000 Bytes'.

FREERTOS Mode and Configuration	
Mode	
Configuration	
Reset Configuration	
User Constants	
Tasks and Queues	
Timers and Semaphores	
Mutexes	
Events	
FreeRTOS Heap Usage	
Config parameters	
Include parameters	
Advanced settings	
Configure the below parameters :	
FreeRTOS version	10.3.1
CMSIS-RTOS version	2.00
MPU/FPU	
ENABLE_MPU	Disabled
ENABLE_FPU	Disabled
Kernel settings	
USE_PREEMPTION	Enabled
CPU_CLOCK_HZ	SystemCoreClock
TICK_RATE_HZ	1000
MAX_PRIORITIES	56
MINIMAL_STACK_SIZE	128 Words
MAX_TASK_NAME_LEN	16
USE_16_BIT_TICKS	Disabled
IDLE_SHOULD_YIELD	Enabled
USE_MUTEXES	Enabled
USE_RECURSIVE_MUTEXES	Enabled
USE_COUNTING_SEMAPHORES	Enabled
QUEUE_REGISTRY_SIZE	8
USE_APPLICATION_TASK_TAG	Disabled
ENABLE_BACKWARD_COMPATIBILITY	Enabled
USE_PORT_OPTIMISED_TASK_SELECTION	Disabled
USE_TICKLESS_IDLE	Disabled
USE_TASK_NOTIFICATIONS	Enabled
RECORD_STACK_HIGH_ADDRESS	Disabled
Memory management settings	
Memory Allocation	Dynamic / Static
TOTAL_HEAP_SIZE	40000 Bytes
Memory management scheme	heap_4
Hook function related definitions	
USE_IDLE_HOOK	Disabled
USE_TICK_HOOK	Disabled

NVIC

Categories A-Z

System Core

- ✓ DMA
- ✓ GPIO
- IWDG
- ✓ **NVIC**
- ✓ RCC
- ⚠ SYS
- WWDG

Analog >

Timers >

Connectivity >

Multimedia >

Computing >

Middleware and Software Packs >

Utilities >

NVIC Mode and Configuration

Mode

Configuration

✓ NVIC

✓ Code generation

☐ Sort by Preemption Priority and Sub Priority

☐ Sort by interrupts names

Search

⏪ ⏩

Show available interrupts

☒ Force DMA channels Interrupts

NVIC Interrupt Table	Enabled	Preemption Priority	Uses FreeRTOS functions
Non maskable interrupt	✓	0	☐
Hard fault interrupt	✓	0	☐
System service call via SWI instruction	✓	0	☐
Pendable request for system service	✓	3	☒
System tick timer	✓	3	☒
PVD through EXTI line 16, PVM (monit. VDDIO2) thro...	☐	3	☒
Flash global interrupt	☐	3	☒
RCC global Interrupt	☐	3	☒
EXTI line 2 and line 3 interrupts	☒	3	☒
EXTI line 4 to 15 interrupts	☒	3	☒
DMA1 channel 1 interrupt	☒	3	☒
DMA1 channel 2 and channel 3 interrupts	☒	3	☒
Time base: TIM1 break, update, trigger and commuta...	☒	3	☐
SPI1/I2S1 Interrupt	☒	3	☒
USART2 + LPUART2 Interrupt	☒	3	☒



Middleware & Software Packs

X-CUBE-ST67W61 Software package

The screenshot displays the STM32CubeMX software interface, specifically the 'Pinout & Configuration' tab. The 'Software Packs' section is active, showing a list of available packs. The 'X-CUBE-ST67W61' pack is selected, and its components are listed in the 'Software Packs Component Selector' dialog box.

The 'Software Packs Component Selector' dialog box shows the following components and their selection status:

Pack / Bundle / Component	Status	Version	Selection
STMicroelectronics.X-CUBE-ST60		1.0.0	Install
STMicroelectronics.X-CUBE-ST67W61	✓	1.0.0	
Device Applications	✓	1.0.0	
Application	✓	1.0.0	BLE_Commissioning_v
Freertos_Tickless		1.0.0	☐
Network ST67W6X_Network_Driver	✓	1.0.0	
ServiceShell		1.0.0	☐
ServiceAPI	✓	1.0.0	☒
Driver / W61_at_and_bus	✓	1.0.0	☒
Utilities	✓		
Logging	✓	1.0.0	☒
Shell		1.0.0	☐
Statistics		1.0.0	☐
Debug TraceRecorder		4.10.2	
TraceRecorder		4.10.2	☐
Data Exchange cJSON		1.7.18	
cJSON		1.7.18	☐
File System LittleFS		2.10.1	
LittleFS		2.10.1	☐
Utility Utilities		1.4.2	

An arrow points to the 'BLE_Commissioning_v' selection in the 'Application' row, with the text 'Select BLE_Commissioning_v'.

The 'Utilities' section is highlighted in yellow. The 'Ok' and 'Cancel' buttons are visible at the bottom right of the dialog box.

Middleware & Software Packs

X-CUBE-ST67W61 Software package

The screenshot displays the STM32CubeIDE configuration environment. The left sidebar shows a list of software packs, with **X-CUBE-ST67W61** highlighted. The main workspace is titled "STMicroelectronics.X-CUBE-ST67W61.1.0.0 Mode and Configuration". It features a "Mode" section with checked options for "Device Applications" and "Network ST67W6X Network Driver". Below this is a "Configuration" section with a "Reset Configuration" button and a tabbed interface. The "Platform Settings" tab is selected, showing a "Platform proposal" table. A blue callout box labeled "Platform settings" points to this tab.

Name	IPs or Components	Found Solutions	BSP API
LogSh_UART	USART:Asynchronous	USART2	Unknown
NCP_SPI	SPI:Full-Duplex Master	SPI1	Unknown

Middleware & Software Packs

X-CUBE-ST67W61 Software package

The screenshot shows the STM Studio software configuration interface. The left pane, titled "Pinout & Configuration", displays a list of software packs. The right pane, titled "Clock Configuration", shows the configuration for the selected package, "STMicroelectronics.X-CUBE-ST67W61.1.0.0 Mode and Configuration".

The "Clock Configuration" pane is divided into two sections: "Mode" and "Configuration". The "Mode" section shows the selected mode, "Mode". The "Configuration" section shows the configuration parameters for the selected mode.

The "Configuration" section is further divided into four tabs: "W6X Modules", "Parameter Settings", "User Constants", and "Platform Settings". The "Parameter Settings" tab is selected.

The "Parameter Settings" tab shows a list of parameters to be configured. The parameters are grouped into sections: "Basic Parameters", "Logging-Shell", "W6X Parameters", and "W61 drv Parameters".

The "W6X Parameters" section contains the following parameters:

- LOG_OUTPUT_MODE: LOG_OUTPUT_UART
- FREERTOS_TASK_NOTIF_ARRAY_LEN: 8
- LOG_LEVEL: LOG_DEBUG
- W6X_BLE_HOSTNAME: ST67W61_BLE
- W6X_WIFI_HOSTNAME: ST67W61_WiFi
- W6X_POWER_SAVE_AUTO: No
- W6X_CLOCK_MODE: Internal RC oscillator
- W6X_WIFI_AUTOCONNECT: No
- W6X_WIFI_DHCP_MODE: DHCP STA+SAP
- W6X_WIFI_COUNTRY_CODE: 00
- W6X_WIFI_ADAPTIVE_COUNTRY_CODE: No
- W6X_NET_RECV_TIMEOUT (ms): 5000
- W6X_NET_SEND_TIMEOUT (ms): 5000
- W6X_NET_RECV_BUFFER_SIZE: 9216
- W6X_WIFI_DNS_MANUAL: No
- W6X_WIFI_DNS_IP_1: {208, 67, 222, 222}
- W6X_WIFI_DNS_IP_2: {8, 8, 8, 8}
- W6X_WIFI_DNS_IP_3: {0, 0, 0, 0}

The "W6X_POWER_SAVE_AUTO" parameter is highlighted with a red box, and a callout box points to it with the text "Disable W6X_POWER_SAVE_AUTO".

Project Creation

The screenshot shows the STM32CubeIDE Project Manager window. The left sidebar has a 'Project' tab selected. The main area is divided into 'Pinout & Configuration' and 'Clock Configuration' tabs. The 'Project Settings' section includes fields for Project Name, Project Location, Application Structure, Toolchain Folder Location, and Toolchain / IDE. The 'Linker Settings' section includes Minimum Heap Size and Minimum Stack Size. The 'Thread-safe Settings' section includes Cortex-M0+NS, Enable multi-threaded support, and Thread-safe Locking Strategy. The 'Mcu and Firmware Package' section includes Mcu Reference, Firmware Package Name and Version, and Use Default Firmware Location. The 'Firmware Relative Path' field is also visible.

Project Name: ST67_G0_BLE_Comm

Project Location: C:\ST67_Test

Application Structure: Advanced ☐ Do not generate the main()

Toolchain Folder Location: C:\ST67_Test\ST67_G0_BLE_Comm\

Toolchain / IDE: STM32CubeIDE ☒ Generate Under Root

Linker Settings

Minimum Heap Size: 0x200

Minimum Stack Size: 0x400

Thread-safe Settings

Cortex-M0+NS

☐ Enable multi-threaded support

Thread-safe Locking Strategy: Default - Mapping suitable strategy depending on RTOS selection.

Mcu and Firmware Package

Mcu Reference: STM32G0B1RETx

Firmware Package Name and Version: STM32Cube_FW_G0_V1.6.2 ☒ Use latest available version

☒ Use Default Firmware Location

Firmware Relative Path: C:/Users/kulkams/STM32Cube/Repository/STM32Cube_FW_G0_V1.6.2

1. Enable “Generate Under Root”

Here, we are enabling the Flat directory structure: (i.e. Driver and MW directories copied into your project directory)

2. Enable “ Use Default Firmware Location”

Project Creation

Home > STM32G0B1RETx - NUCLEO-G0B1RE > ST67_G0_BLE_Comm.ioc - Project Manager

Pinout & Configuration | Clock Configuration | Project Manager

Project

Code Generator

Advanced Settings

STM32Cube MCU packages and embedded software packs

- ☐ Copy all used libraries into the project folder
- ☒ Copy only the necessary library files
- ☐ Add necessary library files as reference in the toolchain project configuration file

Generated files

- ☐ Generate peripheral initialization as a pair of '.c/.h' files per peripheral
- ☐ Backup previously generated files when re-generating
- ☒ Keep User Code when re-generating
- ☒ Delete previously generated files when not re-generated

HAL Settings

- ☐ Set all free pins as analog (to optimize the power consumption)
- ☐ Enable Full Assert

3. Enable "Copy only the necessary files"

Generate Code

5. Generate Code

Home > STM32G0B1RETx - NUCLEO-G0B1RE > ST67_G0_BLE_Comm.ioc - Project Manager

GENERATE CODE

Pinout & Configuration | Clock Configuration | Project Manager | Tools

Driver Selector

Search (Ctrl+F)

RCC HAL
GPIO HAL
DMA HAL
SPI HAL
SPI1 HAL
USART HAL
USART2 HAL
STMicroelectronics X-CUBE-ST67W61.1.0.0 HAL

Project

Code Generator

Advanced Settings

Generated Function Calls

Generate Code	Rank	Function Name	Peripheral Instance Name	Do Not Generate Function Call	Visibility (Static)
☒	1	SystemClock_Config	RCC	☐	☐
☒	2	MX_GPIO_Init	GPIO	☐	☒
☒	3	MX_DMA_Init	DMA	☐	☒
☒	4	MX_SPI1_Init	SPI1	☐	☒
☒	5	MX_USART2_UART_Init	USART2	☐	☒
☒	6	MX_App_BLE_Commissioning_Init	STMicroelectronics X-CUBE-ST67W61.1.0.0	☒	☐

Register CallBack

Search (Ctrl+F)

ADC DISABLE
CEC DISABLE
COMP DISABLE
CRYP DISABLE
DAC DISABLE
FDCAN DISABLE
HCD DISABLE
I2C DISABLE
I2S DISABLE
IRDA DISABLE
LPTIM DISABLE
PCD DISABLE
RNG DISABLE
RTC DISABLE
SMBUS DISABLE
SPI DISABLE
TIM DISABLE
UART DISABLE
USART DISABLE
WWDG DISABLE

4. Enable this option
(here: we are not generating this function call)

Folder Structure

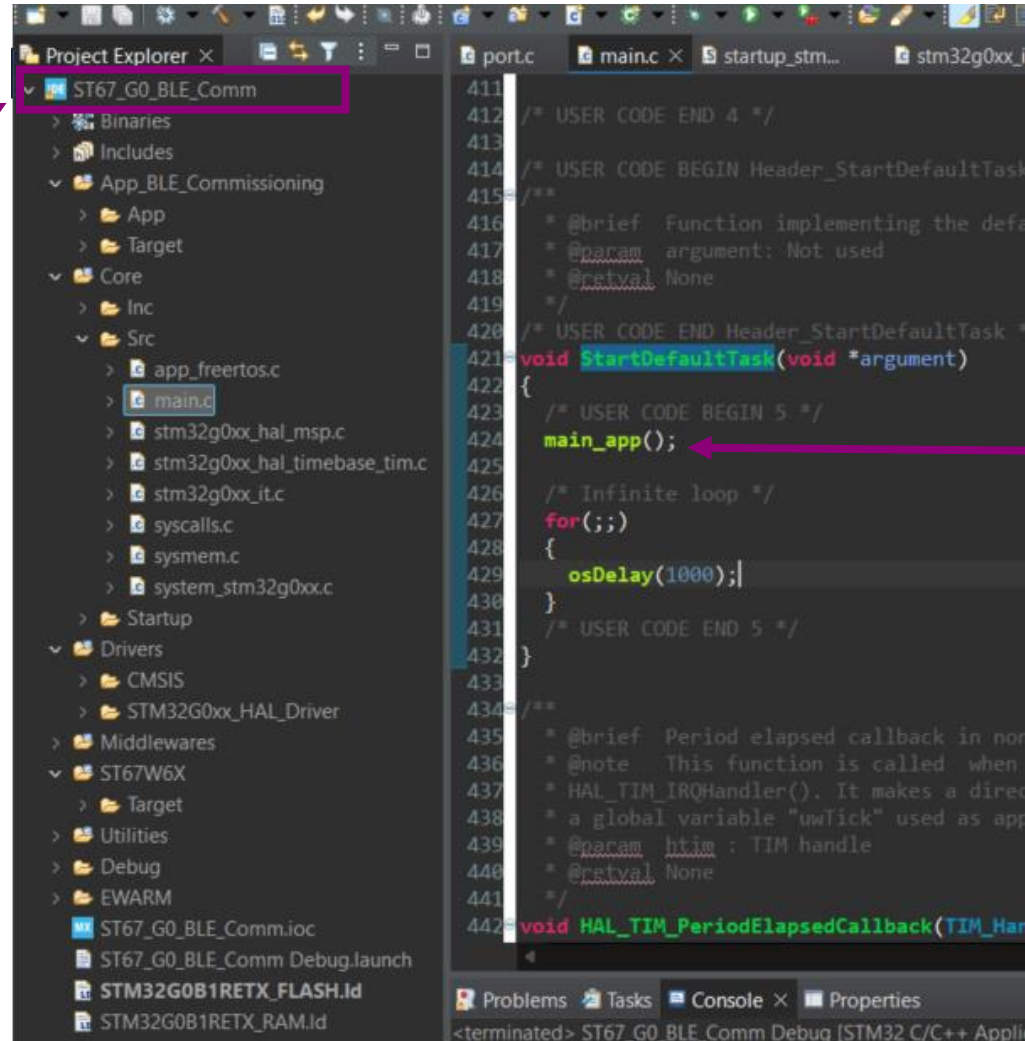
Name	Date modified	Type	Size
.settings	7/24/2025 1:37 PM	File folder	
App_BLE_Commissioning	7/24/2025 7:22 PM	File folder	
Core	7/24/2025 1:35 PM	File folder	
Debug	7/29/2025 6:31 PM	File folder	
Drivers	7/24/2025 1:34 PM	File folder	
EWARM	7/24/2025 6:37 PM	File folder	
Middlewares	7/24/2025 1:34 PM	File folder	
ST67W6X	7/24/2025 1:34 PM	File folder	
Utilities	7/24/2025 11:28 PM	File folder	
.cproject	7/25/2025 7:36 PM	CPROJECT File	34 KB
.mxproject	7/25/2025 7:36 PM	MXPROJECT File	25 KB
.project	7/24/2025 1:35 PM	PROJECT File	2 KB
ST67_G0_BLE_Comm Debug.launch	7/29/2025 9:18 PM	LAUNCH File	10 KB
ST67_G0_BLE_Comm.ioc	7/25/2025 7:36 PM	STM32CubeMX	16 KB
STM32G0B1RETX_FLASH.ld	7/25/2025 7:36 PM	LD File	6 KB
STM32G0B1RETX_RAM.ld	7/24/2025 1:35 PM	LD File	6 KB

Folder structure view after generating the code from CubeMX

- App code
- Core
- Drivers
- Middlewares
- ST67W6X
- Utilities

STM32CubeIDE

Load the project on
the STM32CubeIDE



Call main_app() function under
StartDefaultTask() in main.c file



xPortIsInsideInterrupt()

- If the used device embeds an Arm® Cortex®M0+ (e.g. STM32G0) and its FreeRTOS version is older than of V10.6, definition below **must be added manually after project has been generated**.
- The below function definition must be added in the **portmacro.h** file (Project/Middlewares/Third_Party/ FreeRTOS/ Source/Portable/ARM_CM0)

```
#define portINLINE                __inline
#ifndef portFORCE_INLINE
    #define portFORCE_INLINE    inline __attribute__( ( always_inline ) )
#endif

portFORCE_INLINE static BaseType_t xPortIsInsideInterrupt( void )
{
    uint32_t ulCurrentInterrupt;
    BaseType_t xReturn;
    /* Obtain the number of the currently executing interrupt. */
    __asm volatile ( "mrs %0, ipsr" : "=r" ( ulCurrentInterrupt )::"memory" );
    if( ulCurrentInterrupt == 0 )
    {
        xReturn = pdFALSE;
    }
    else
    {
        xReturn = pdTRUE;
    }
    return xReturn;
}
```

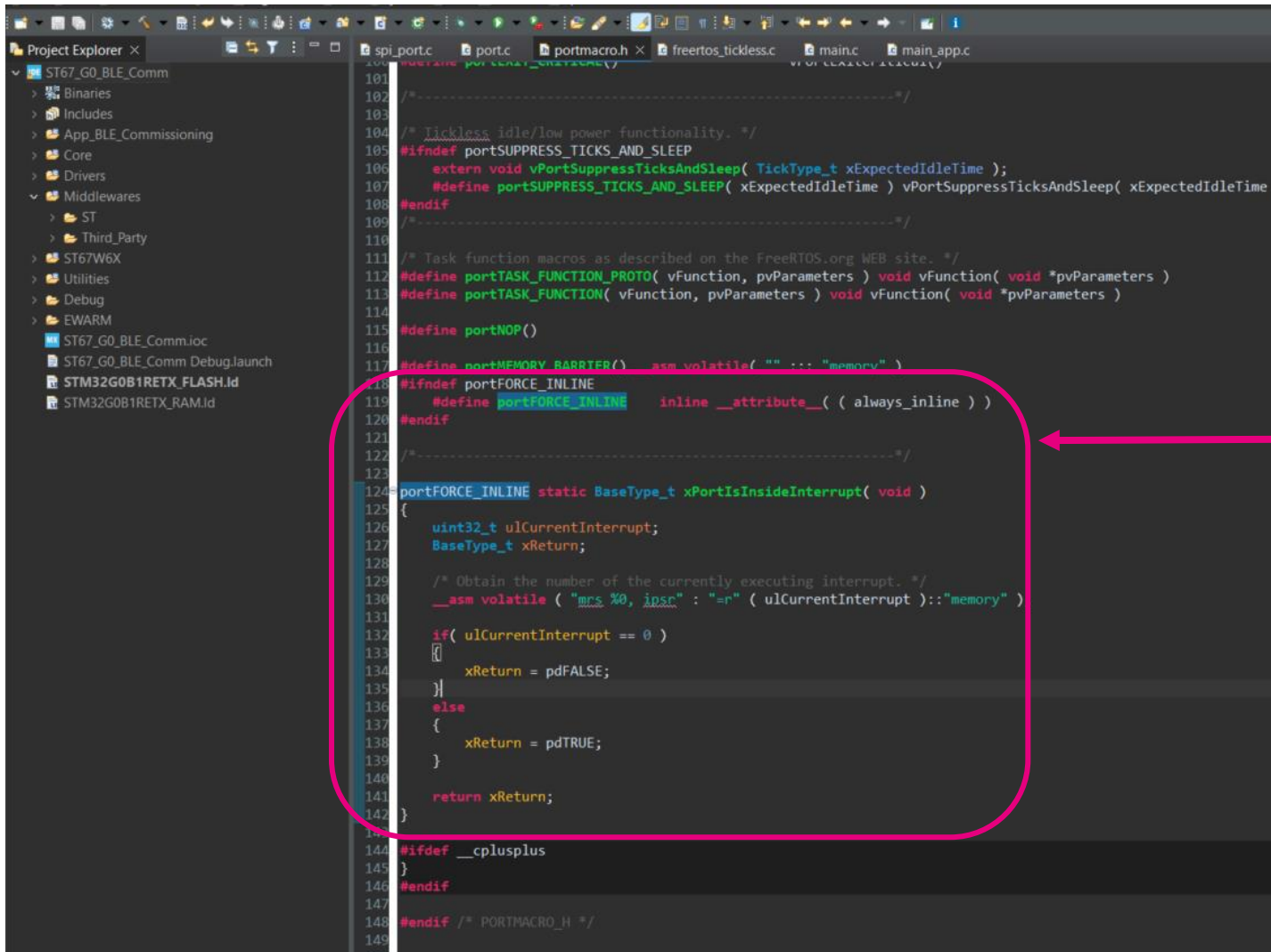
STM32CubeIDE

Call `xPortIsInsideInterrupt()` function under `portmacro.h` file

IMP: As we are using G0 (CortexM0) -> older version of FreeRTOS, so we need to add this function definition manually.

```
#define portINLINE      __inline
#ifndef portFORCE_INLINE
    #define portFORCE_INLINE inline
    __attribute__((always_inline))
#endif
```

```
portFORCE_INLINE static BaseType_t
xPortIsInsideInterrupt( void )
{
    uint32_t ulCurrentInterrupt;
    BaseType_t xReturn;
    /* Obtain the number of the currently executing
    interrupt. */
    __asm volatile ( "mrs %0, ipsr" : "=r" (
    ulCurrentInterrupt )::"memory" );
    if( ulCurrentInterrupt == 0 )
    {
        xReturn = pdFALSE;
    }
    else
    {
        xReturn = pdTRUE;
    }
    return xReturn;
}
```



```
101
102 /*-----*/
103
104 /* Tickless idle/low power functionality. */
105 #ifndef portSUPPRESS_TICKS_AND_SLEEP
106     extern void vPortSuppressTicksAndSleep( TickType_t xExpectedIdleTime );
107     #define portSUPPRESS_TICKS_AND_SLEEP( xExpectedIdleTime ) vPortSuppressTicksAndSleep( xExpectedIdleTime )
108 #endif
109 /*-----*/
110
111 /* Task function macros as described on the FreeRTOS.org WEB site. */
112 #define portTASK_FUNCTION_PROTO( vFunction, pvParameters ) void vFunction( void *pvParameters )
113 #define portTASK_FUNCTION( vFunction, pvParameters ) void vFunction( void *pvParameters )
114
115 #define portNOP()
116
117 #define portMEMORY_BARRIER() __asm volatile( "" ::: "memory" )
118
119 #ifndef portFORCE_INLINE
120     #define portFORCE_INLINE inline __attribute__(( always_inline ))
121 #endif
122 /*-----*/
123
124 portFORCE_INLINE static BaseType_t xPortIsInsideInterrupt( void )
125 {
126     uint32_t ulCurrentInterrupt;
127     BaseType_t xReturn;
128
129     /* Obtain the number of the currently executing interrupt. */
130     __asm volatile ( "mrs %0, ipsr" : "=r" ( ulCurrentInterrupt )::"memory" )
131
132     if( ulCurrentInterrupt == 0 )
133     {
134         xReturn = pdFALSE;
135     }
136     else
137     {
138         xReturn = pdTRUE;
139     }
140
141     return xReturn;
142 }
143
144 #ifdef __cplusplus
145 }
146 #endif
147
148 #endif /* PORTMACRO_H */
149
```

STM32CubeIDE

Add **#ifndef STM32G0B1xx**

Here we are disabling the custom memcpy() definition, as we would be using the memcpy() definition in string.h

Project/ST67W6X/spi_port.c

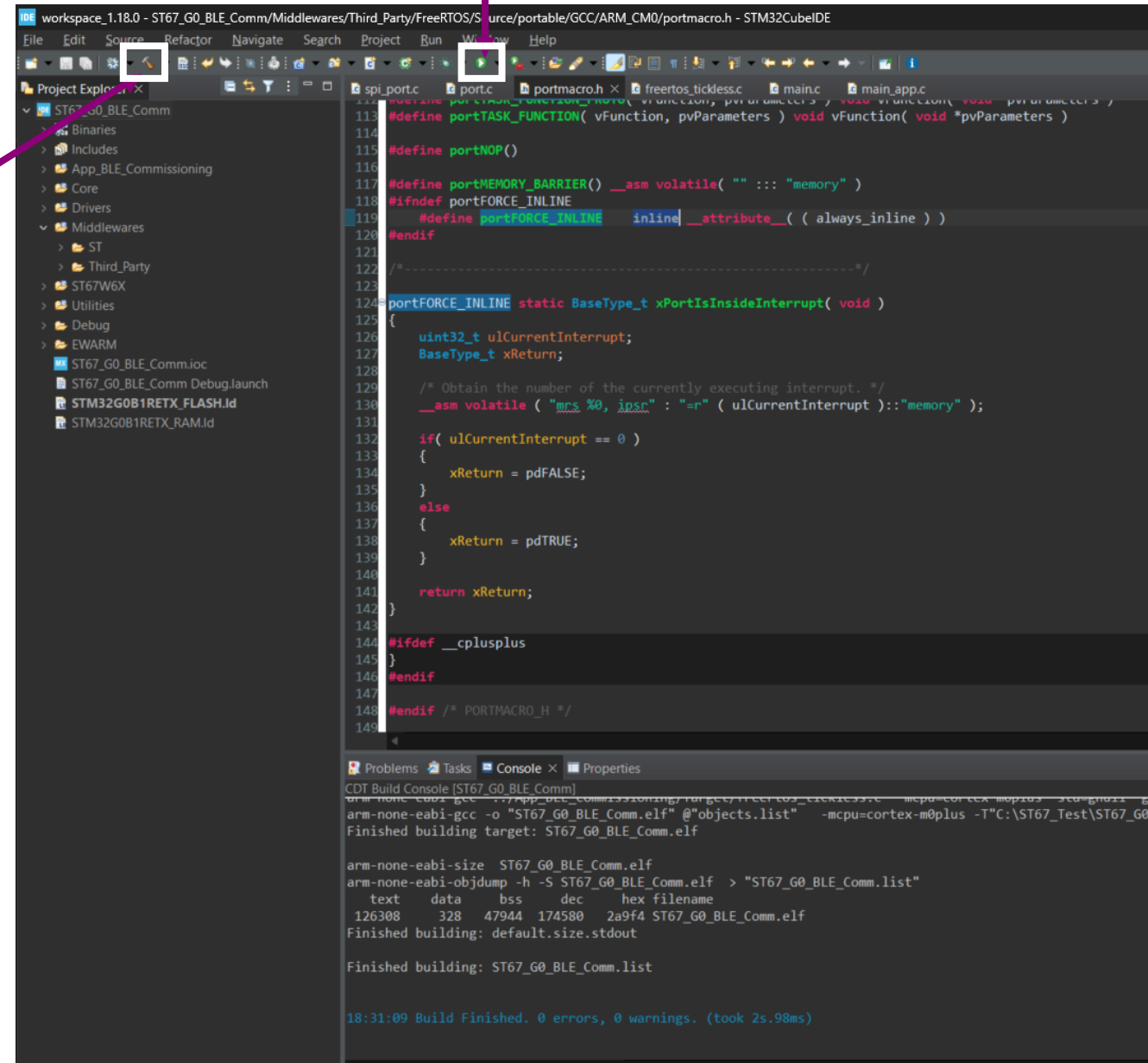
Add **#endif**

```
76 /* USER CODE END PFP */
77
78 /* Functions Definition -----
79 #ifndef STM32G0B1xx
80 #ifdef __ICCARM__
81 void *spi_port_memcpy(void *dest, const void *src, unsigned int len)
82 #endif /* __ICCARM__ */
83 #ifdef __GNUC__
84 void *memcpy(void *dest, const void *src, unsigned int len)
85 #endif /* __ICCARM__ */
86 {
87     /* USER CODE BEGIN memcpy_1 */
88
89     /* USER CODE END memcpy_1 */
90     uint8_t *d = (uint8_t *)dest;
91     const uint8_t *s = (const uint8_t *)src;
92
93     /* Copy bytes until the destination address is aligned to 4 bytes */
94     while (((uint32_t) d % 4 != 0) && len > 0)
95     {
96         *d++ = *s++;
97         len--;
98     }
99
100     /* Copy 4-byte blocks */
101     uint32_t *d32 = (uint32_t *)d;
102     const uint32_t *s32 = (const uint32_t *)s;
103     while (len >= 4)
104     {
105         *d32++ = *s32++;
106         len -= 4;
107     }
108
109     /* Copy remaining bytes */
110     d = (uint8_t *)d32;
111     s = (const uint8_t *)s32;
112     while (len > 0)
113     {
114         *d++ = *s++;
115         len--;
116     }
117
118     return dest;
119     /* USER CODE BEGIN memcpy_End */
120
121     /* USER CODE END memcpy_End */
122 }
123 #endif
```

2. Flash the project

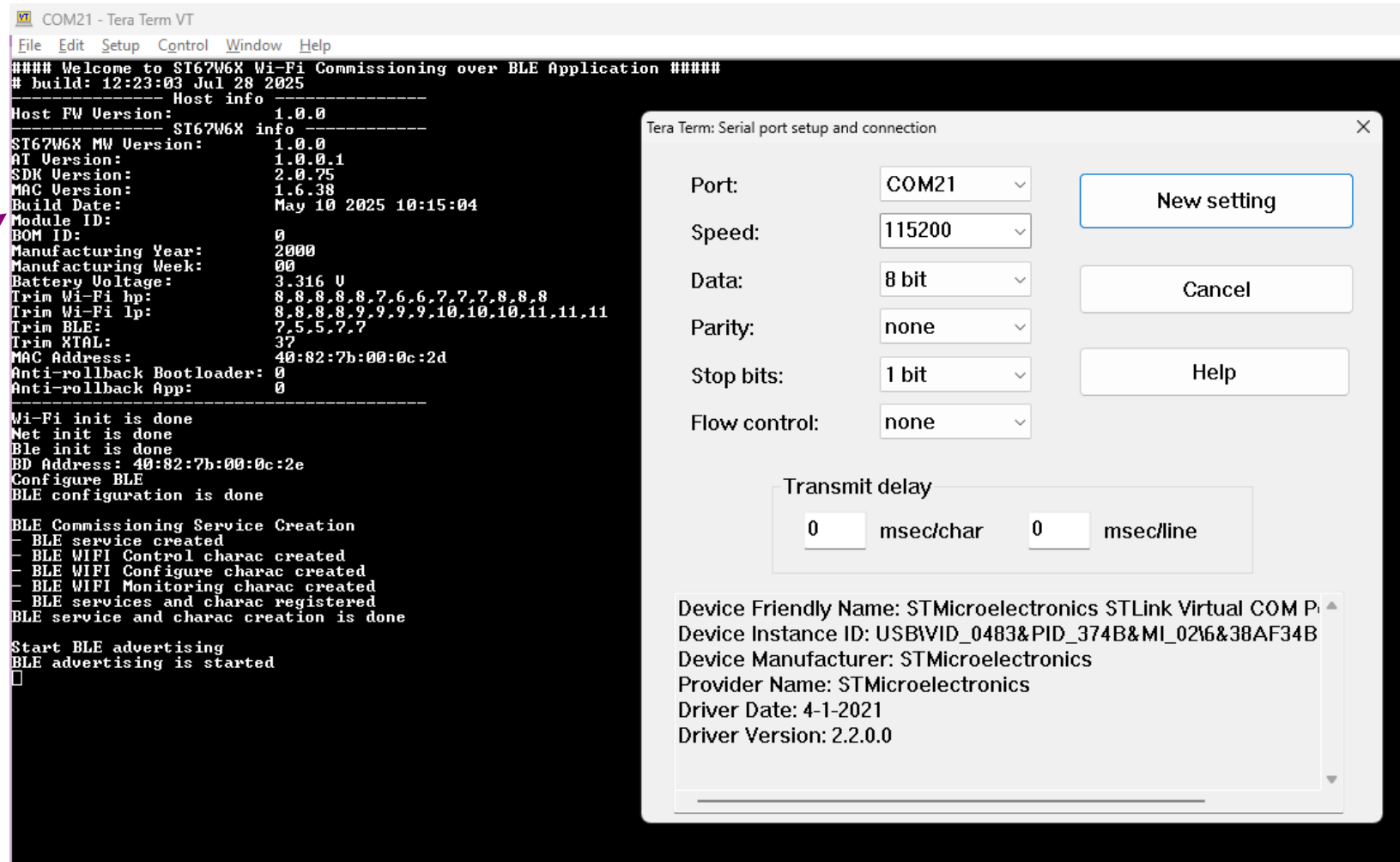
Build and flash

1. Build the project



```
workspace_1.18.0 - ST67_G0_BLE_Comm/Middlewares/Third_Party/FreeRTOS/Source/portable/GCC/ARM_CM0/portmacro.h - STM32CubeIDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
  ST67_G0_BLE_Comm
    Binaries
    Includes
    App_BLE_Commissioning
    Core
    Drivers
    Middlewares
      ST
      Third_Party
      ST67W6X
    Utilities
    Debug
    EWARM
    ST67_G0_BLE_Comm.ioc
    ST67_G0_BLE_Comm.Debug.launch
    STM32G0B1RETX_FLASH.Id
    STM32G0B1RETX_RAM.Id
  spi_port.c
  portmacro.h
  freertos_tickless.c
  main.c
  main_app.c
  113 #define portTASK_FUNCTION( vFunction, pvParameters ) void vFunction( void *pvParameters )
  114
  115 #define portNOP()
  116
  117 #define portMEMORY_BARRIER() __asm volatile( "" ::: "memory" )
  118 #ifndef portFORCE_INLINE
  119 #define portFORCE_INLINE inline __attribute__(( always_inline ))
  120 #endif
  121
  122 /*-----*/
  123
  124 portFORCE_INLINE static BaseType_t xPortIsInsideInterrupt( void )
  125 {
  126     uint32_t ulCurrentInterrupt;
  127     BaseType_t xReturn;
  128
  129     /* Obtain the number of the currently executing interrupt. */
  130     __asm volatile ( "mrs %0, ipsr" : "=r" ( ulCurrentInterrupt ) :: "memory" );
  131
  132     if( ulCurrentInterrupt == 0 )
  133     {
  134         xReturn = pdFALSE;
  135     }
  136     else
  137     {
  138         xReturn = pdTRUE;
  139     }
  140
  141     return xReturn;
  142 }
  143
  144 #ifdef __cplusplus
  145 }
  146 #endif
  147
  148 #endif /* PORTMACRO_H */
  149
  Problems Tasks Console Properties
  CDT Build Console [ST67_G0_BLE_Comm]
  arm-none-eabi-gcc -D__APP_BLE_COMMISSIONING__ -D__TARGET_FREERTOS_TICKLESS__ -D__MCPU_CORTEX_M0PLUS__ -D__ST67_G0_BLE__ -D__ST67_G0_BLE_FLASH__ -D__ST67_G0_BLE_RAM__ -o "ST67_G0_BLE_Comm.elf" @"objects.list" -mcpu=cortex-m0plus -T"C:\ST67_Test\ST67_G0_BLE_Comm.ld"
  Finished building target: ST67_G0_BLE_Comm.elf
  arm-none-eabi-size ST67_G0_BLE_Comm.elf
  arm-none-eabi-objdump -h -S ST67_G0_BLE_Comm.elf > "ST67_G0_BLE_Comm.list"
  text data bss dec hex filename
  126308 328 47944 174580 2a9f4 ST67_G0_BLE_Comm.elf
  Finished building: default.size.stdout
  Finished building: ST67_G0_BLE_Comm.list
  18:31:09 Build Finished. 0 errors, 0 warnings. (took 2s.98ms)
```

Tera Term Logs



The screenshot shows the Tera Term VT window with a menu bar (File, Edit, Setup, Control, Window, Help) and a log of system messages. A magenta arrow points from a text box to the log. A dialog box titled 'Tera Term: Serial port setup and connection' is open, showing configuration for COM21 at 115200 baud, 8-bit data, 1-bit stop bits, and no parity or flow control. The dialog also includes a 'Transmit delay' section and device information.

```
COM21 - Tera Term VT
File Edit Setup Control Window Help
#### Welcome to ST67W6X Wi-Fi Commissioning over BLE Application ####
# build: 12:23:03 Jul 28 2025
----- Host info -----
Host FW Version: 1.0.0
ST67W6X info -----
ST67W6X MW Version: 1.0.0
AT Version: 1.0.0.1
SDK Version: 2.0.75
MAC Version: 1.6.38
Build Date: May 10 2025 10:15:04
Module ID:
BOM ID: 0
Manufacturing Year: 2000
Manufacturing Week: 00
Battery Voltage: 3.316 V
Trim Wi-Fi hp: 8,8,8,8,8,7,6,6,7,7,7,8,8,8
Trim Wi-Fi lp: 8,8,8,8,9,9,9,9,10,10,10,11,11,11
Trim BLE: 7,5,5,7,7
Trim XTAL: 37
MAC Address: 40:82:7b:00:0c:2d
Anti-rollback Bootloader: 0
Anti-rollback App: 0
-----
Wi-Fi init is done
Net init is done
Ble init is done
BD Address: 40:82:7b:00:0c:2e
Configure BLE
BLE configuration is done
BLE Commissioning Service Creation
- BLE service created
- BLE WIFI Control charac created
- BLE WIFI Configure charac created
- BLE WIFI Monitoring charac created
- BLE services and charac registered
BLE service and charac creation is done
Start BLE advertising
BLE advertising is started
□
```

Tera Term: Serial port setup and connection

Port: COM21
Speed: 115200
Data: 8 bit
Parity: none
Stop bits: 1 bit
Flow control: none

New setting
Cancel
Help

Transmit delay
0 msec/char 0 msec/line

Device Friendly Name: STMicroelectronics STLink Virtual COM P
Device Instance ID: USB\VID_0483&PID_374B&MI_02\6&38AF34B
Device Manufacturer: STMicroelectronics
Provider Name: STMicroelectronics
Driver Date: 4-1-2021
Driver Version: 2.2.0.0

Tera Term Logs
after flashing

ST BLEToolbox/ ST BLE Webpage

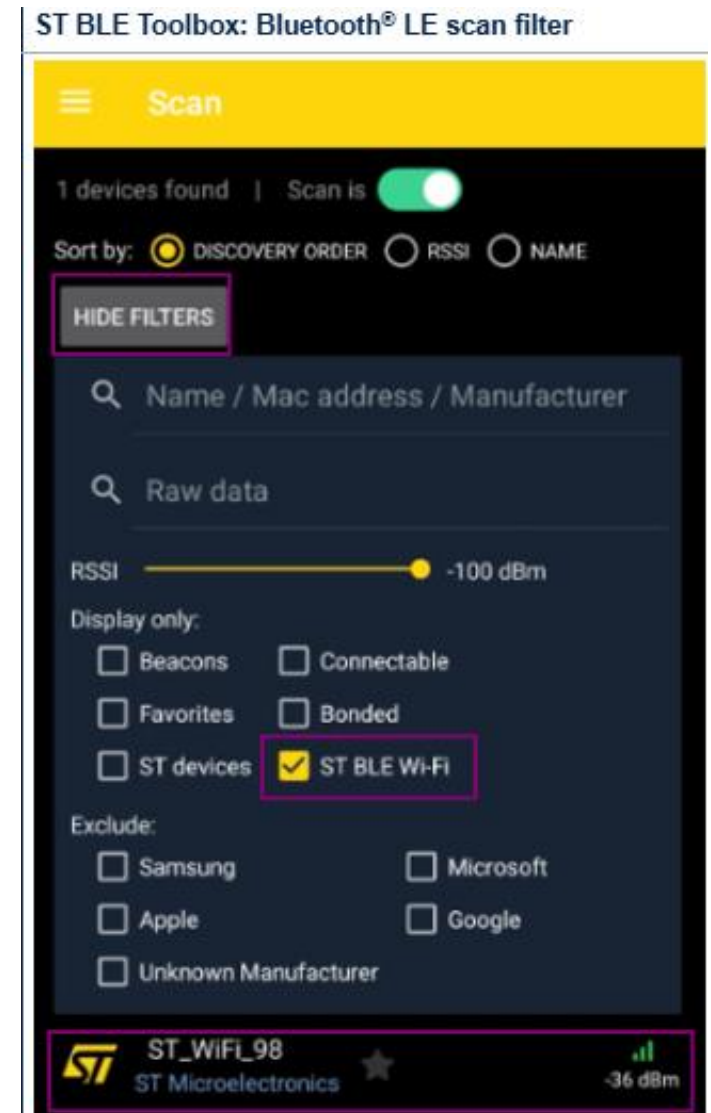
To interface with the Wi-Fi commissioning embedded application, you can use [STBLEToolbox Android/iOS application](#) / [ST Web Bluetooth® Interface](#)

Download [STBLEToolBox](#)

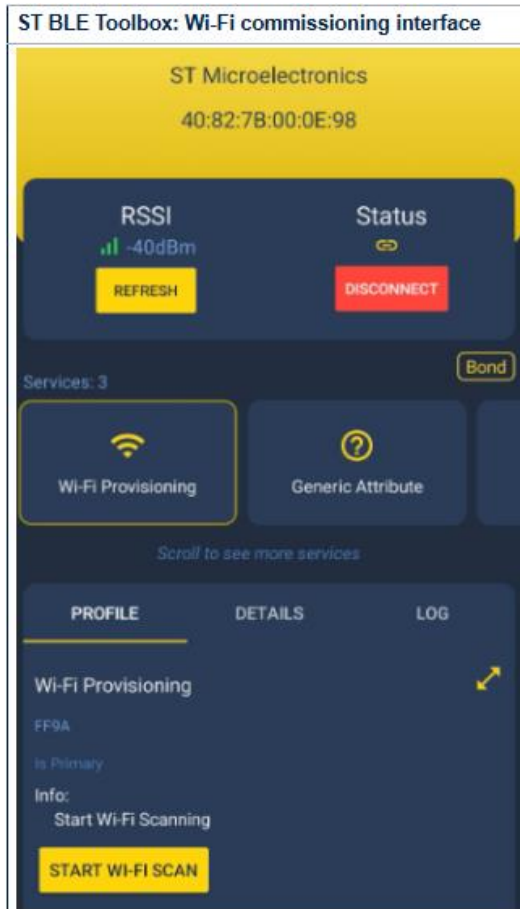
The versions supporting Wi-Fi commissioning service are **v1.4.3 for Android** and **1.2.8 for iOS**

Steps to launch and connect to Wi-Fi AP:

1. Launch the smartphone application and enable the scan button, if it is not already activated.
2. You can filter scanned devices by clicking on the SHOW FILTERS button and select ST BLE Wi-Fi to show only devices with Wi-Fi commissioning service.
3. Select your device, named ST_WiFi_XX where XX represents the last BD address digit, and connect to it.

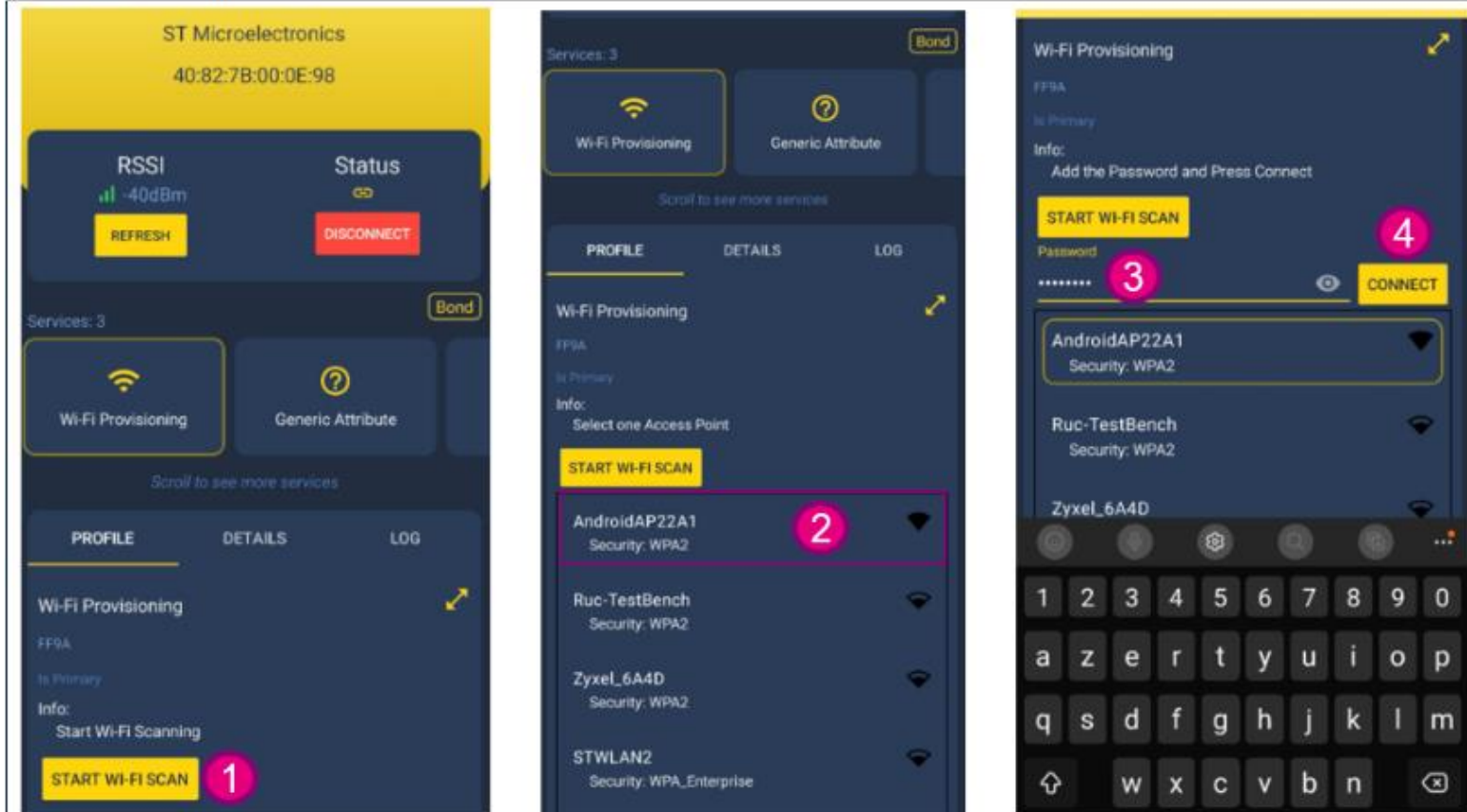


4. Once connected to the Bluetooth® LE device, click on Wi-Fi commissioning button to open the commissioning dedicated interface.

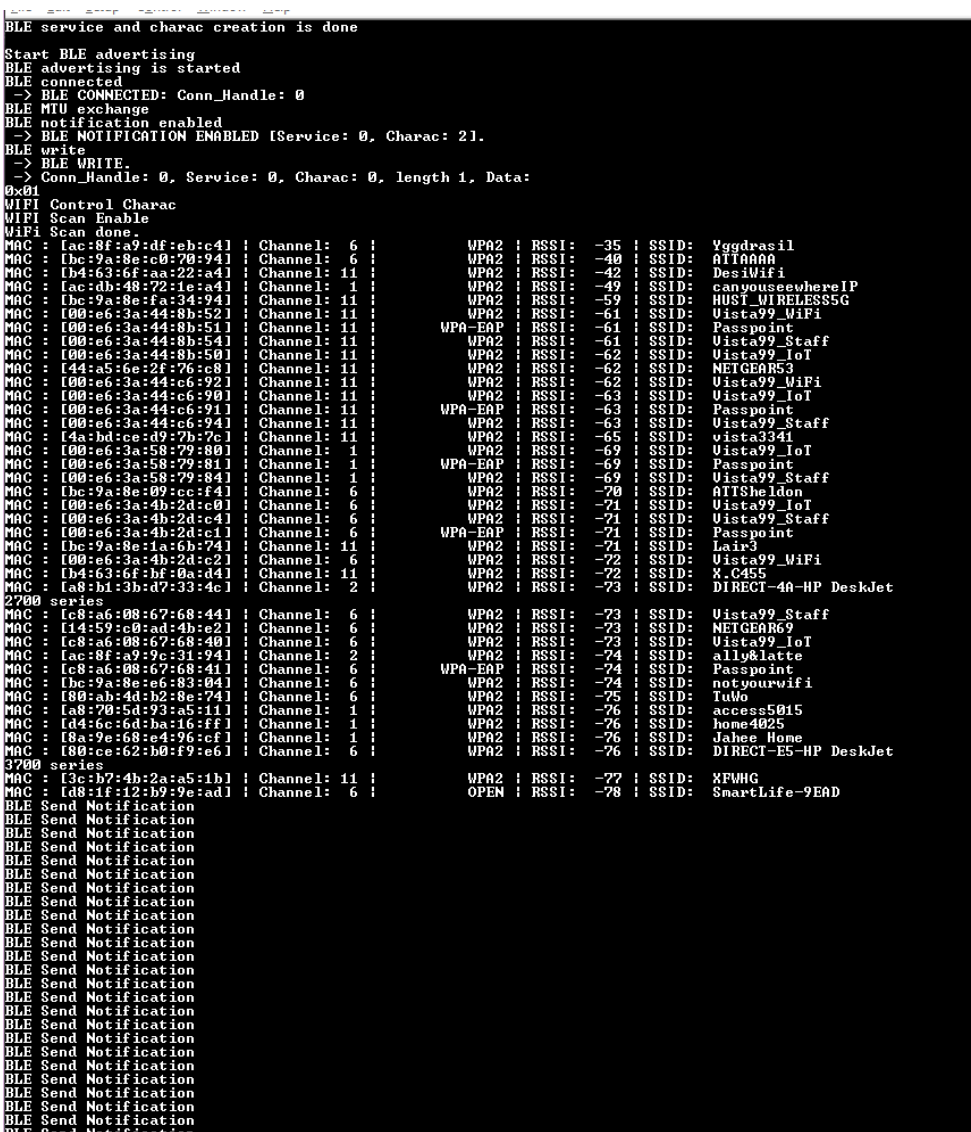
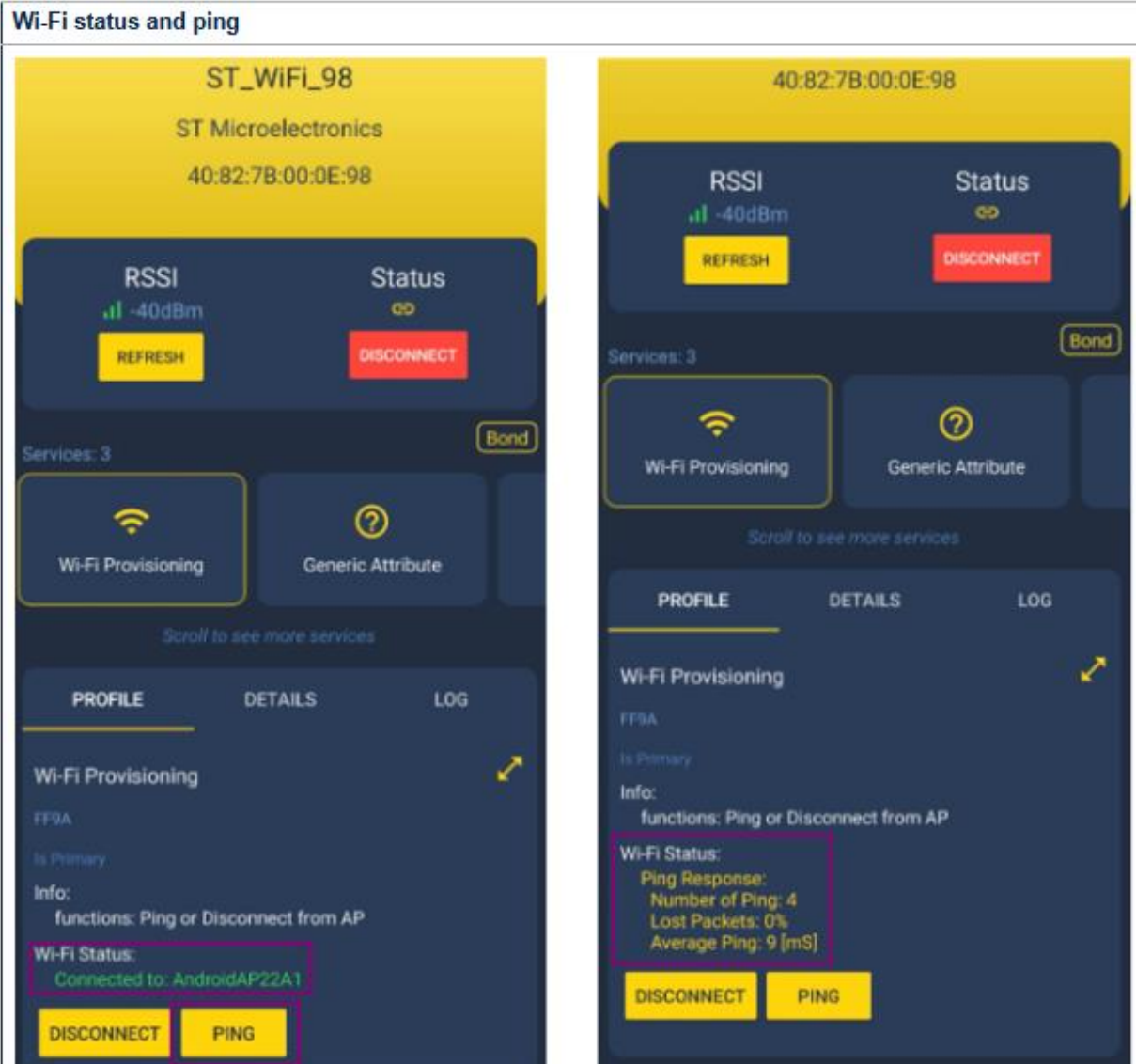


From this interface,
Launch a Wi-Fi scan (1),
To detect and list all available networks(2).
Once the available networks are displayed, select the one you want to connect to (2)
Type a password if needed (3)
Click on CONNECT button (4).

Wi-Fi connection interface



Once the connection request is done, the connection status is displayed in the interface. Clicking on the Ping button start a ping test with the Wi-Fi access point and returns results in the interface.





life.augmented

Walk through of the MQTT application

[ST67W611M Wi-Fi®](#) – [ST67W6X](#) MQTT Demonstration - stm32mcu

ST67W6X MQTT

Overview

- **Description**

The MQTT demonstration available in the X-CUBE-ST67W61 expansion package implements an MQTT client which connects to an MQTT Broker in order to publish environmental sensor data and device local information and subscribe on MQTT topics to receive command.

- In this example, the subscribed topic can be used to receive additional 'control' actions from a remote client publishing to the same topic.

1. Change the LED state on the host board

- To monitor the data sent and received, to/from the topics, a MQTT client can be installed on a PC, smartphone or tablet. Subscribing to the configured topics, allows the client to be notified every time the device publishes data to this topic.

- **Requirements**

- A X-NUCLEO-67W61M1 Wi-Fi Expansion board.
- A NUCLEO-U575ZI-Q Host board.
- A Wi-Fi Access Point with internet access and no firewall blocking the MQTT requests.
- [Optional] A X-NUCLEO-IKS4A1 Environmental sensors expansion board.

ST67W6X MQTT Topology

NUCLEO-U575ZI-Q
X-NUCLEO-67W61M1
X-NUCLEO-IKS4A1

Wi-Fi station

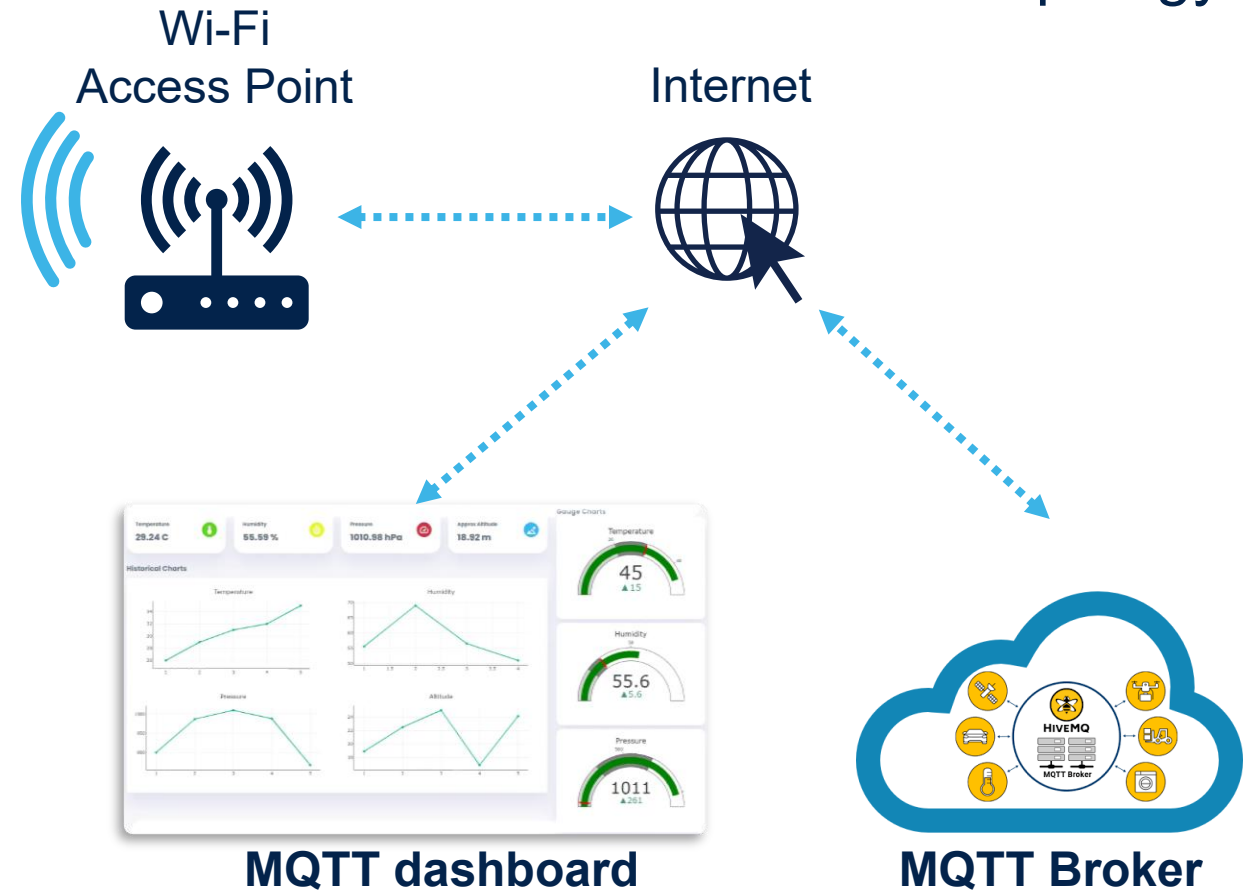


USB serial



PC with Hyper Terminal

Wi-Fi



ST67W6X MQTT

Usage

- Initialization of Wi-Fi, Net and MQTT modules in the Network Coprocessor.
- Wi-Fi connection establishment with the local Access Point.
- Synchronization of local time in using the Network Coprocessor SNTP client.
- Initialization of MEMS expansion board to get environmental sensor values.
- Connection of MQTT Client to a remote MQTT broker.
- MQTT topic subscription to receive messages from cloud.
- Periodic publication of message including:
 - Environmental sensor values**
 - Local time
 - RSSI of Access Point

MQTT Initialization process

Description	Function	File
Initialize the system (HAL, Clocks, SPI, I2C, UART, RTC, GPIO)	main()	Core/Src/mainc.c
Create the default FreeRTOS task containing the main application execution	MX_FREERTOS_Init()	Core/Src/app_feertos.c
Start all configured tasks	osKernelStart()	cmsis_os2.c
Run the main_app() enter application function	StartDefaultTask()	Core/Src/app_freertos.c
Initialize the logging utility to forward all traces on UART peripheral	W6X_Util_Init()	ST67W6X_Network_Driver/Core/w6x_sys.c
Initialize the application callbacks to receive events from ST67W6X_Network_Driver	W6X_RegisterAppCb()	ST67W6X_Network_Driver/Core/w6x_sys.c
Initialize the ST67W6X_Network_Driver Power up the Network Coprocessor Display global identification	W6X_Init()	ST67W6X_Network_Driver/Core/w6x_sys.c
Initialize the ST67W6X_Network_Driver Wi-Fi module Set the STA mode Enable the DHCP Client Configure the Country Code Set the Hostname	W6X_WiFi_Init()	ST67W6X_Network_Driver/Core/w6x_wifi.c
Initialize the ST67W6X_Network_DriverNetwork module	W6X_Net_Init()	ST67W6X_Network_Driver/Core/w6x_net.c
Initialize the ST67W6X_Network_Driver MQTT module Register pre-allocated buffer to receive subscription messages	W6X_MQTT_Init()	ST67W6X_Network_Driver/Core/w6x_mqtt.c

MQTT Application Process

The different steps of the application **Process** are described below:

Description	Function	File
Connect the Wi-Fi station device to an available Access Point	W6X_WiFi_Connect()	ST67W6X_Network_Driver/Core/w6x_wifi.c
Enable the time synchronization through a remote SNTP server	W6X_Net_SetSNTPConfiguration()	ST67W6X_Network_Driver/Core/w6x_net.c
Retrieve the SNTP time to update the RTC time configuration	W6X_Net_GetTime()	ST67W6X_Network_Driver/Core/w6x_net.c
Initialize the sensor expansion board	Sys_Sensors_Init()	X-CUBE-MEMS1/App/sys_sensors.c
Configure the device to a MQTT broker	W6X_MQTT_Configure()	ST67W6X_Network_Driver/Core/w6x_mqtt.c
Connect the device to a MQTT broker	W6X_MQTT_Connect()	ST67W6X_Network_Driver/Core/w6x_mqtt.c
Start a topic subscription. When a new message is received, a callback catches and parses the frame.	W6X_MQTT_Subscribe()	ST67W6X_Network_Driver/Core/w6x_mqtt.c
Publish a new message on the selected topic. The RTC time is always set by calling HAL_RTC_GetTime() and HAL_RTC_GetDate(). If a sensor board is available, the environmental measures are added in the frame by calling Sys_Sensors_Read()	W6X_MQTT_Publish()	ST67W6X_Network_Driver/Core/w6x_mqtt.c

Two topics subscription are started:

- /devices/<MQClientID>/control
- /sensors/<MQClientID>

ST67W6X MQTT

User setup

- Local Access Point credentials: *App_MQTT\App\app_config.h*

```
#define WIFI_SSID          "ST_DEMO"  
#define WIFI_PASSWORD     "stdemo01"
```

- MQTT parameters: *App_MQTT\App\app_config.h*

```
#define MQTT_HOST_NAME "broker.emqx.io"
```

```
#define MQTT_HOST_PORT 1883
```

```
#define MQTT_SECURITY_LEVEL 0
```

```
#define MQTT_CLIENT_ID "mySTM32_772"
```

```
#define MQTT_USERNAME ""
```

```
#define MQTT_USER_PASSWORD ""
```

Notes :

It uses a public broker

Requires at least one device publisher and one subscriber
TLS (secure broker connection)

ST67W6X MQTT

IoT MQTT panel app



- Smartphone client : IoT MQTT Panel
 - Android play store: [IoT MQTT Panel - Apps on Google Play](#)
 - Android APK-pure: <https://apkpure.com/iot-mqtt-panel/snr.lab.iotmqttpanel.prod>
 - Apple store: [IoT MQTT Panel on the App Store](#)
- PC / Web client (no firewall)
 - HiveMQ: [MQTT Client by HiveMQ](#)
 - Mosquitto: [Eclipse Mosquitto](#)

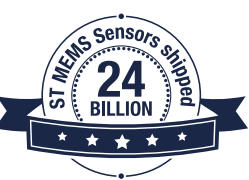


life.augmented

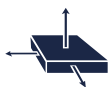
MLC - Machine Learning Core

Agenda

- 1 Introduction to Sensors and MLC
- 2 Demo: MLC Workflow in MEMS Studio
- 3 Hands-On: Integrating MLC into your STM32 MQTT project



ST MEMS and sensors overview



Accelerometers Inclinometers Vibration sensors

LIS2DE12 / LIS2DH12

LIS2DW12 / LIS2DTW12

LIS2DU12

LIS2DUX12 / LIS2DUXS12

H3LIS331DL

IIS2DLPC

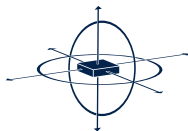
IIS2DULPX

IIS2ICLX

IIS3DWB

AIS2DW12

AIS2IH



6-axis IMUs

LSM6DSO / LSM6DSOX
LSM6DSO32 / LSM6DSO32X

LSM6DSR / LSM6DSRX

LSM6DSV / LSM6DSV16X
LSM6DSV16BX / LSM6DSV32X

LSM6DSV80X / LSM6DSV320X

LSM6DSO16IS

ISM330DHCX

ISM330BX

ISM330IS

ASM330LHH / ASM330LHHX
ASM330LHHXG1

ASM330LHB / ASM330LHBG1



Biosensors

ST1VAFE3BX

ST1VAFE6AX



Magnetometers

LIS2MDL

IIS2MDC

LSM303AGR



Absolute Pressure

LPS22DF

ILPS22QS

LPS28DFW

ILPS28QSW



IR Presence

STHS34PF80



Temperature

STTS22H



Image Sensors

VD55G1

VD55G0

VD56G3

VD66GY



Proximity and Ranging

VL53L4ED

VL53L4CX

VL53L4CD

VL53L7CX / VL53L7CH

VL53L8CX / VL53L8CH

Ambient Light Sensor

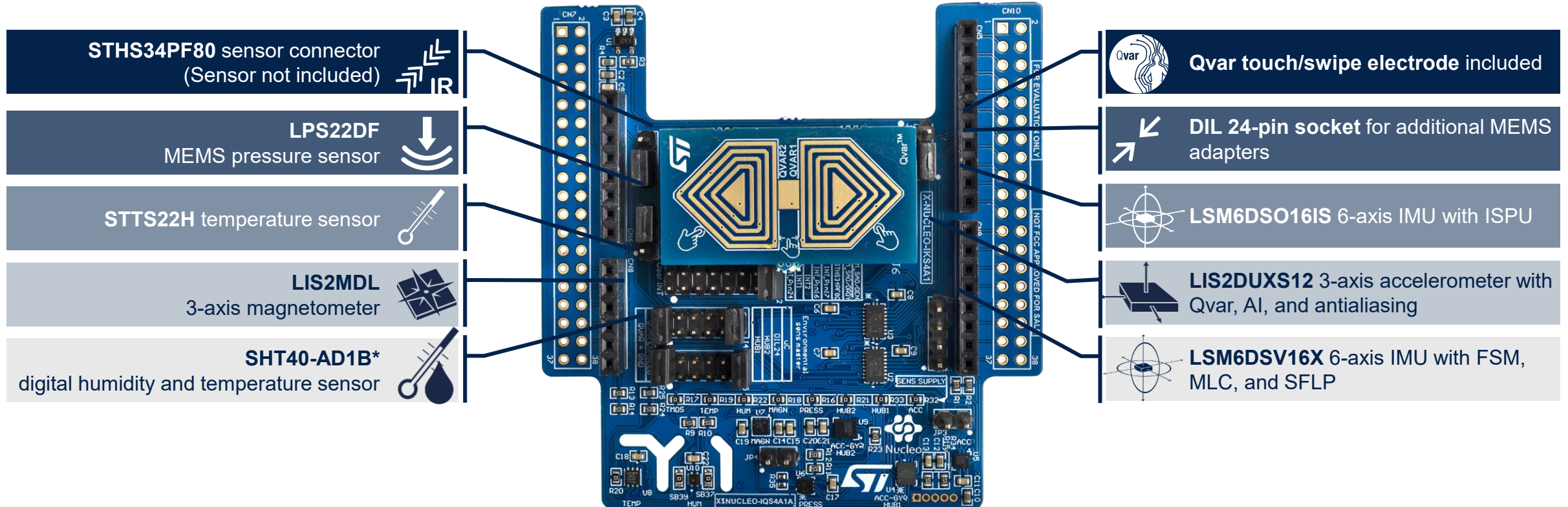
VD6283TX

Sensors on X-NUCLEO-IKS4A1



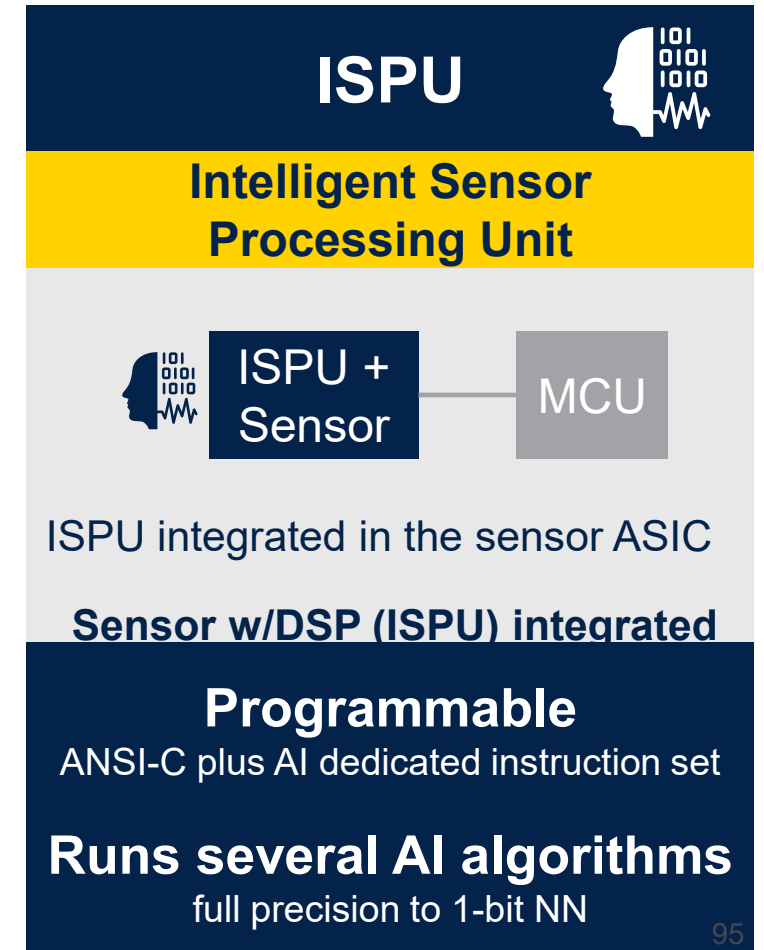
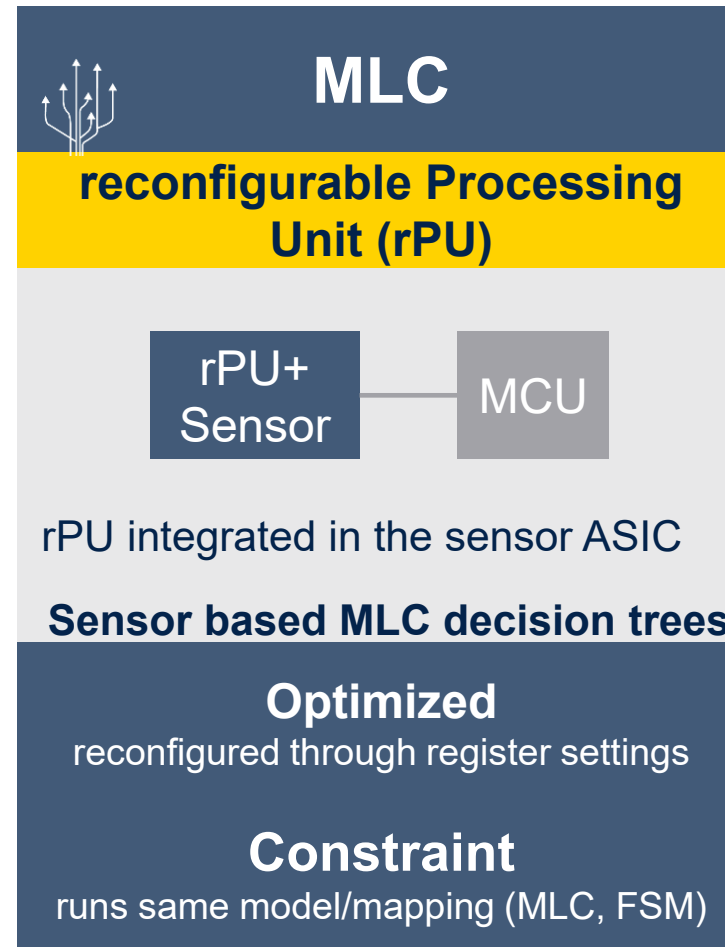
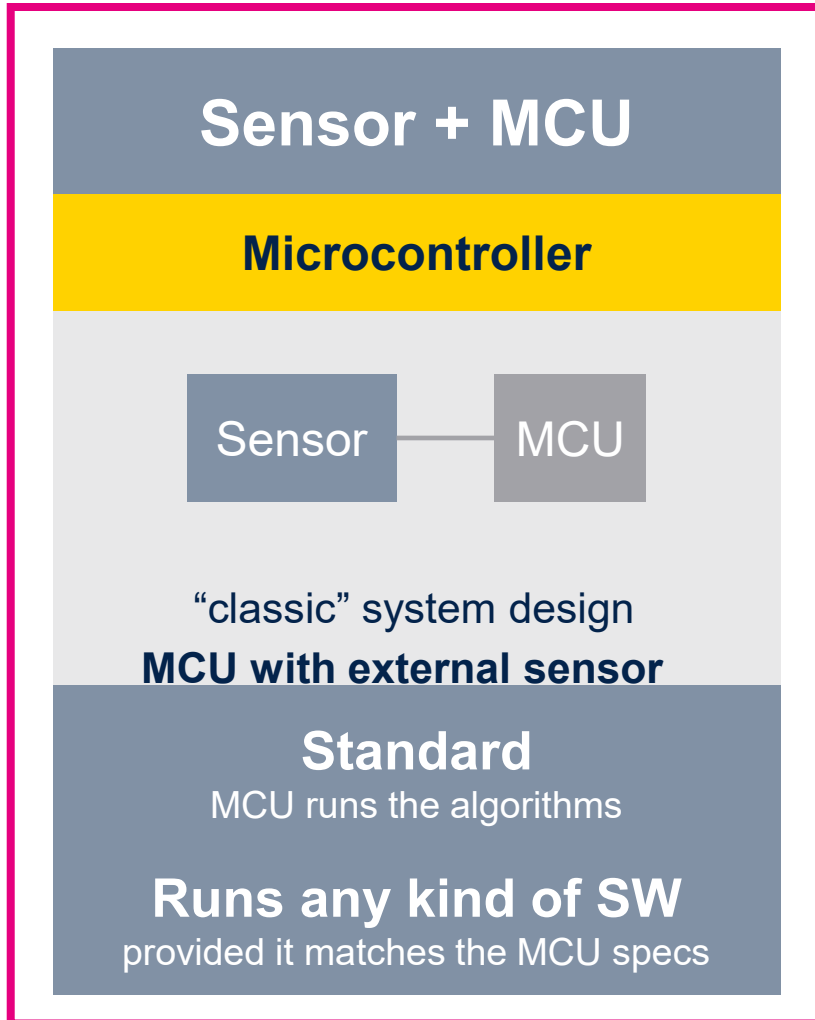
life.augmented

A platform for new applications





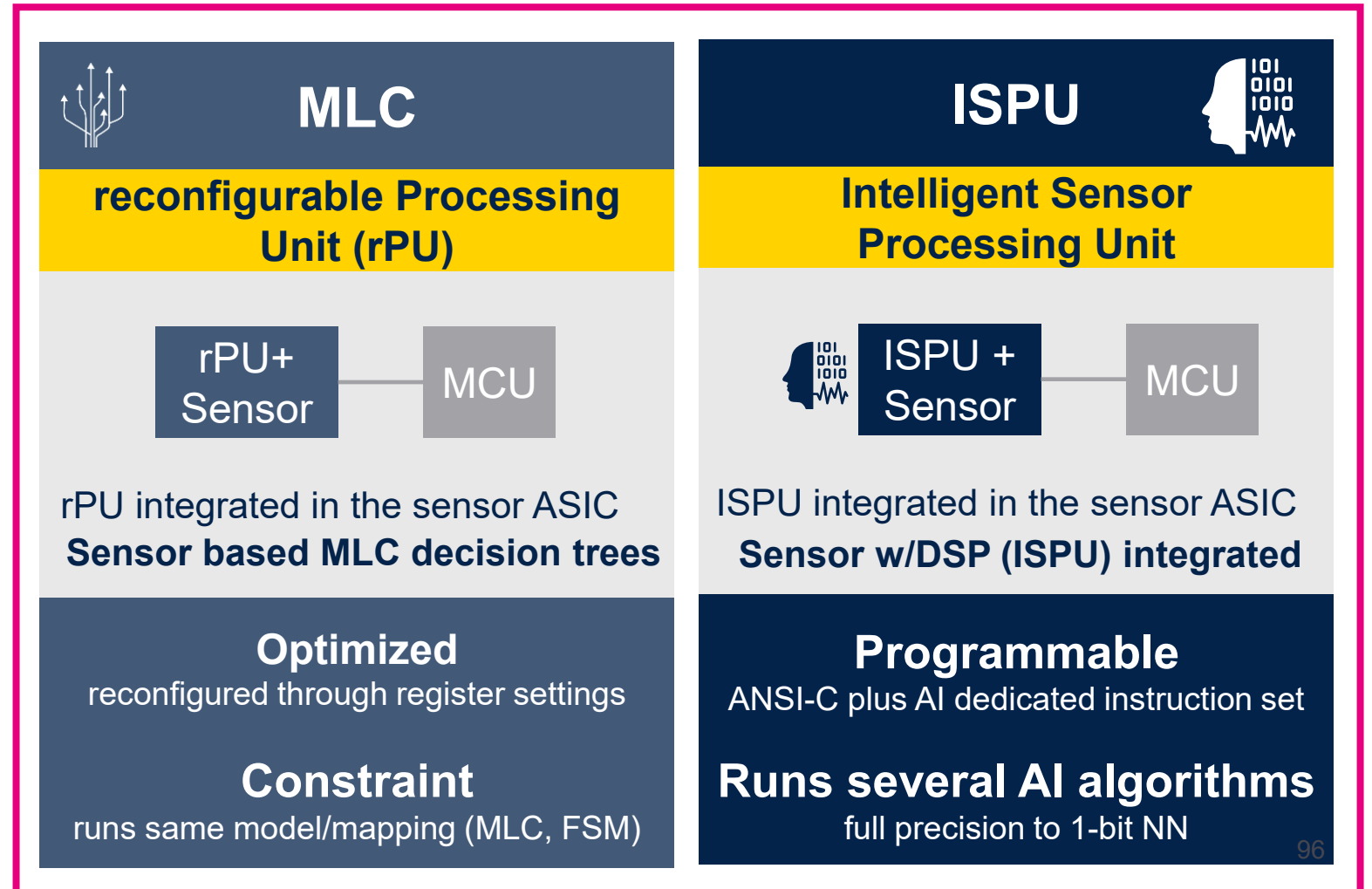
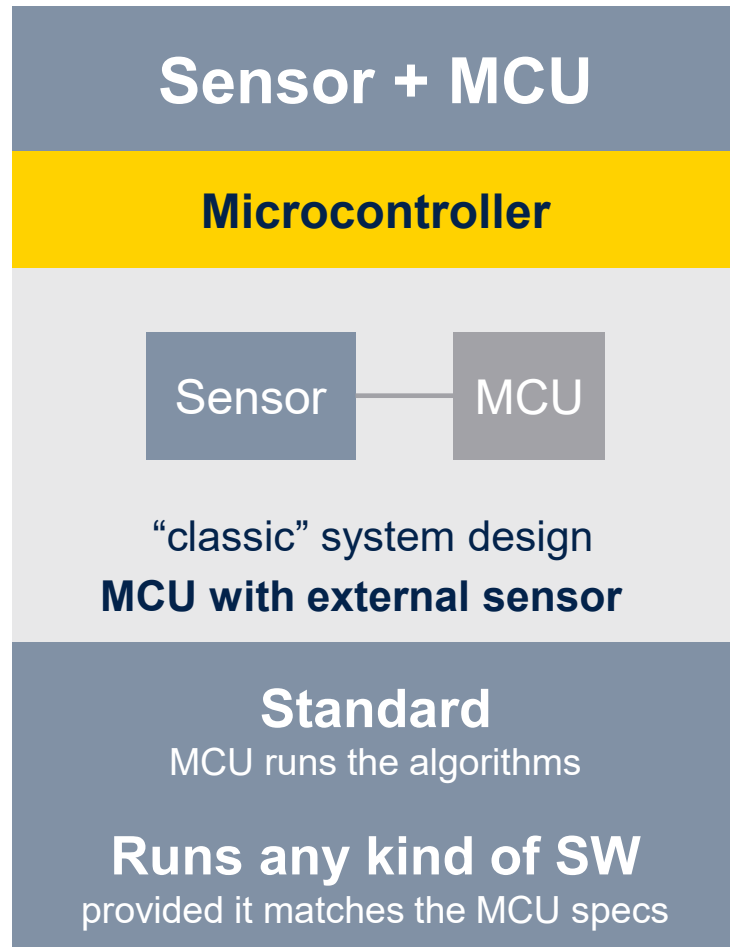
Intelligence ON the Edge



95

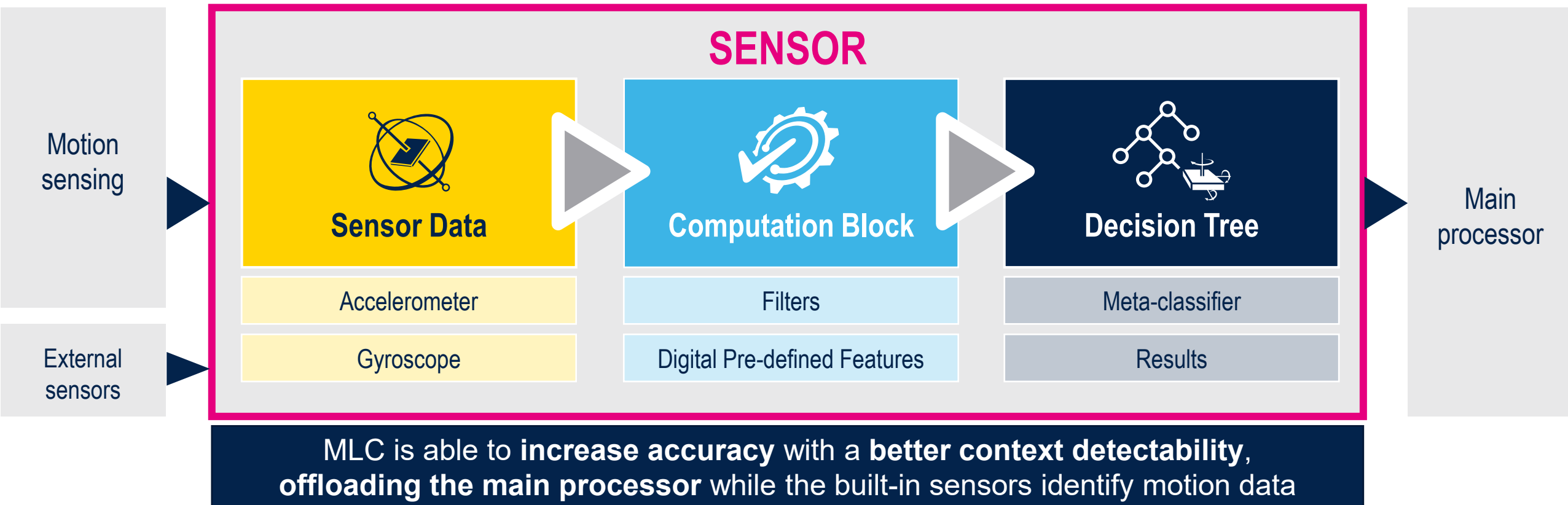


Moving the Intelligence IN the Edge



MLC - Machine Learning Core

MLC is an in-sensor classification engine based on a decision tree logic

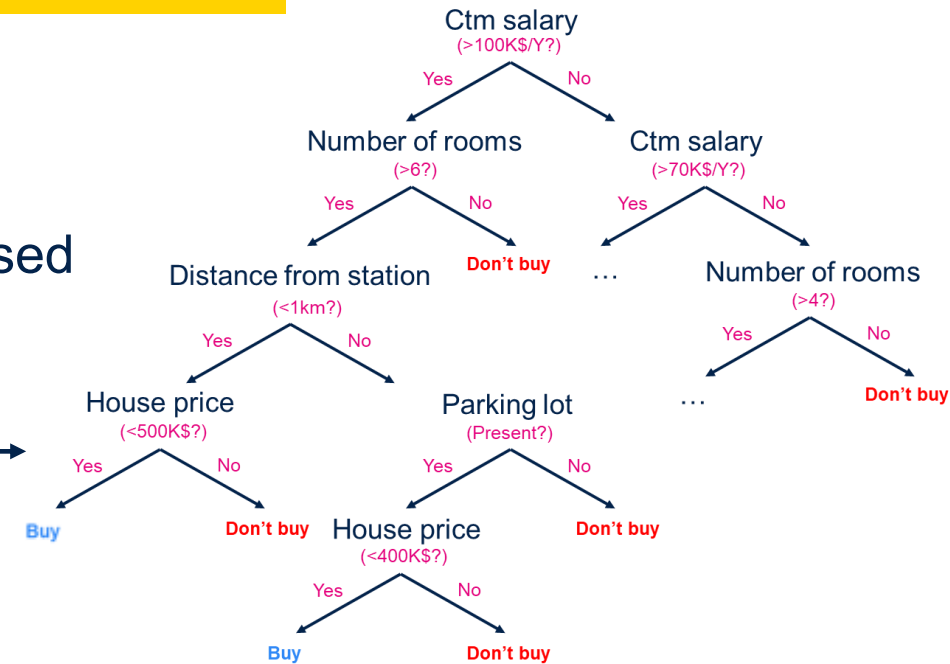


ML model training

What does the ML model learn from the data?



- The **features** that will be used
- In which **decision nodes** the **features** will be used
- The **threshold values** for **decision nodes**
- The results for the **leaf nodes**



Training the same Machine Learning Model with another dataset (i.e. houses in Milan), the resulting decision tree will be different in order to “**best fit**” the new data given for training

ST toolbox for machine learning core (MLC) in the sensor

Machine learning core configuration

Operating mode

Capture data



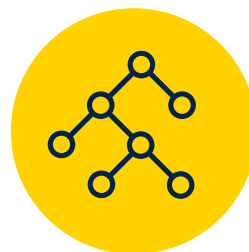
- Accelerometer
- Gyroscope
- External sensors

Label data



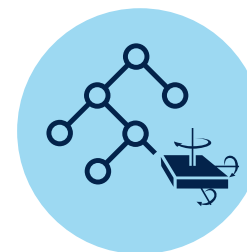
- Filters
- Features

Build decision tree



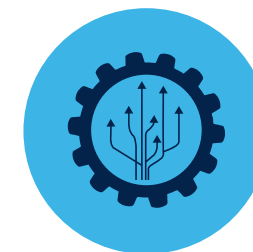
- Classification
- Results

Embed decision tree



- DT implementation

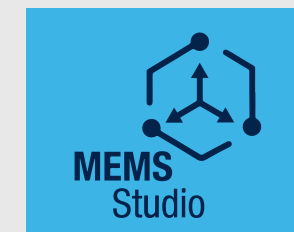
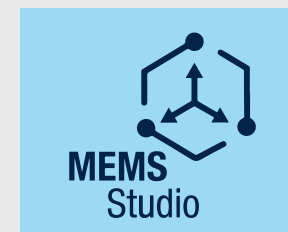
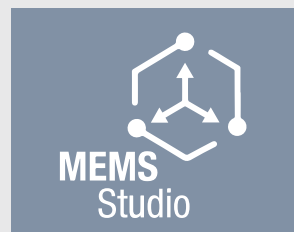
Process new data



- Real time test

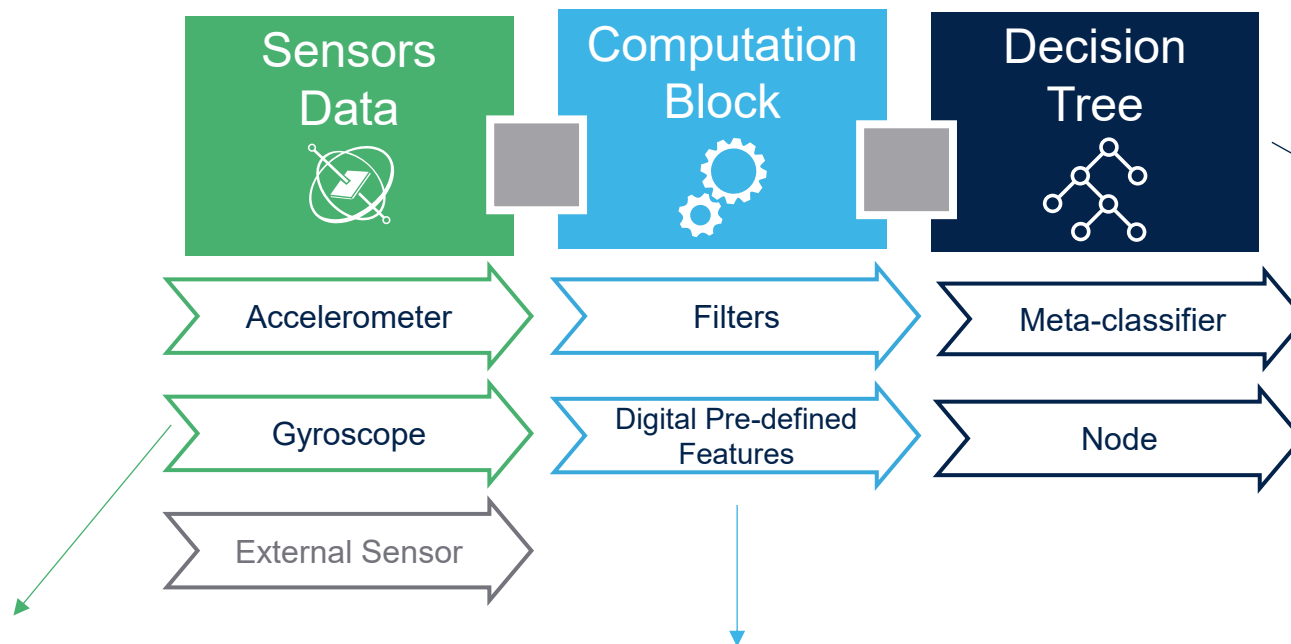
WHAT

HOW





Machine Learning Core - Overview



Sensors Data:

- **Accelerometer** $\rightarrow [a_x \ a_y \ a_z], [a_v], [a^2_v]$
- **Gyroscope** $\rightarrow [g_x \ g_y \ g_z], [g_v], [g^2_v]$
- **External sensor** $\rightarrow [m_x \ m_y \ m_z], [m_v], [m^2_v]$ (i. e. mag)
- **Magnitude** $\rightarrow V = \sqrt{X^2 + Y^2 + Z^2}$

Computation Block

- Sensors data can be filtered with a 2nd order IIR **Filter**.
- **Features** are statistical parameter calculated from: Input Data or Filtered Input Data
- Examples of features are: Mean, Variance, Energy, Peak to Peak, ...

Decision Tree

- is a predictive model built from **training data**. Outputs of the computation blocks are the inputs of the decision tree.
- Each **Node** is characterized by one «if-then-else» condition.
 - Mean on Acc_X < 0.5 g
 - Variance on Gyro_Z < 200 dps
- Decision tree can either generate a result at every sample or filter the results with a **Meta-classifier**, to have a more robust output.



Data collection: Key points

Class definition

Class 1 >> Motor stop...

Class 2 >> Motor running OK

Class 3 >> Motor running NG

Motor vibration example

Class 1 >> Still

Class 2 >> Walking

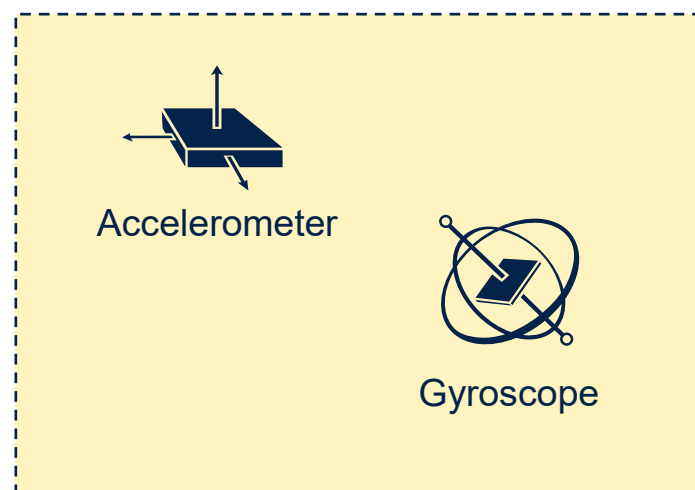
Class 3 >> Running

Class 4 >> Riding bicycle

Class 5 >> Car

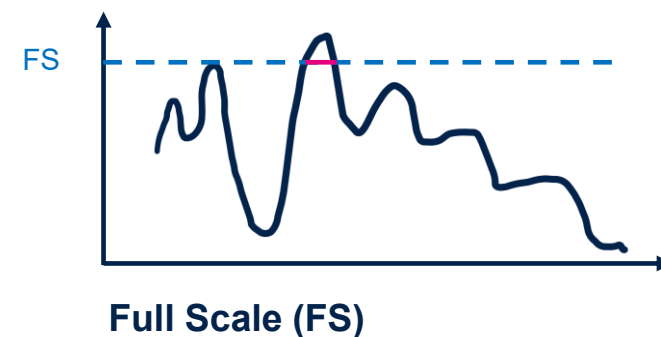
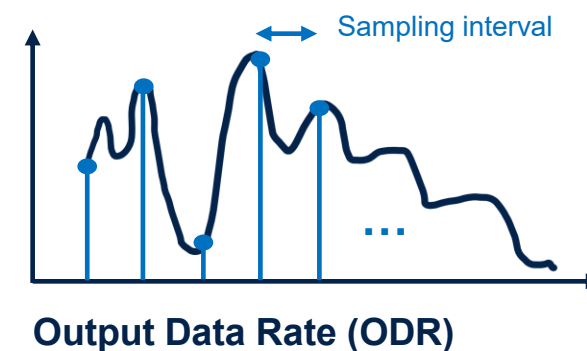
Human Activity Recognition example

Sensor selection



Other external sensors
(Pressure, Compass, ...)

Sensor configuration



Features calculation inside MEMS

Raw data



Acc_X, Acc_Y, Acc_Z
Acc_V, Acc_V2



Gyro_X, Gyro_Y, Gyro_Z
Gyro_V, Gyro_V2



ExtSens_X, ExtSens_Y,
ExtSens_Z
ExtSens_V, Ext_Sens_V2



Filters

Filtered data
(High-pass, Band-pass,
IIR1 and IIR2 filters)

Features

Mean

$$\frac{1}{WL} \sum_{k=0}^{WL-1} I_k$$

Variance

$$\left(\frac{\sum_{k=0}^{WL-1} I_k^2}{WL} \right) - \left(\frac{\sum_{k=0}^{WL-1} I_k}{WL} \right)^2$$

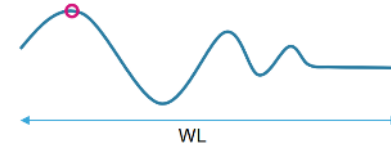
Energy

$$\sum_{k=0}^{WL-1} I_k^2$$

Peak-to-Peak



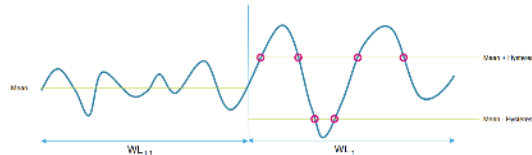
Maximum



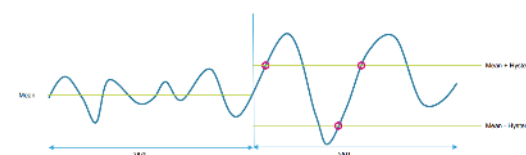
Minimum



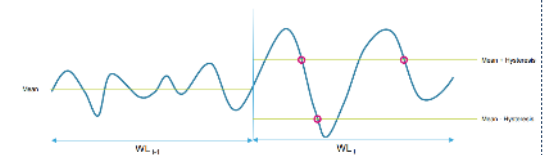
Zero crossing



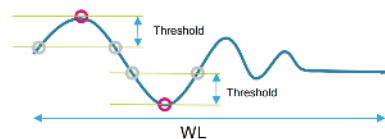
Positive Zero crossing



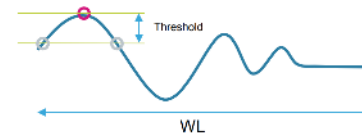
Negative Zero crossing



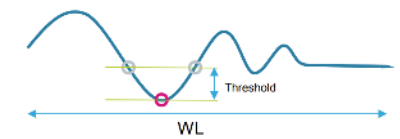
Peak detector



Positive Peak detector



Negative Peak detector

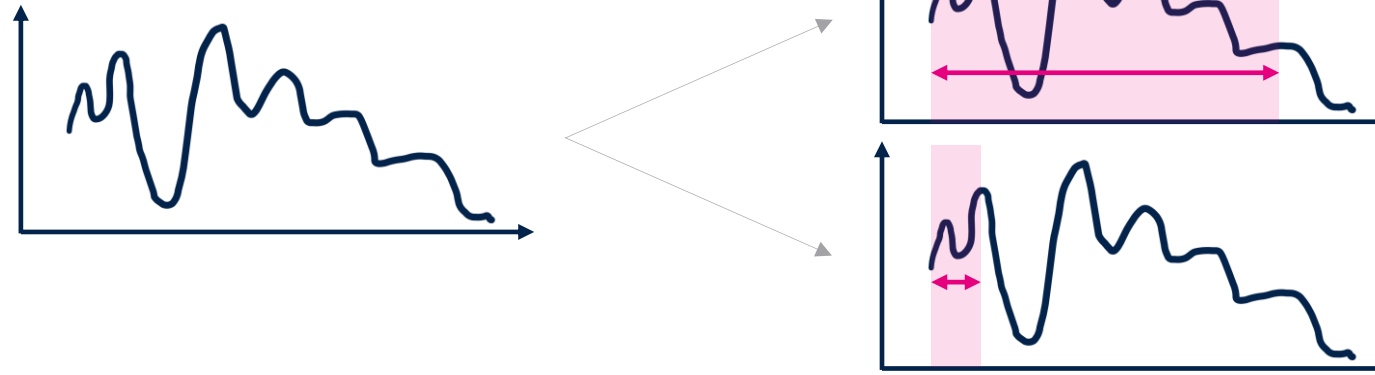


Note: the window for features computation (WL) is configurable (from 1 to 255 samples). It is not a moving window

Window size selection

Window length is strongly dependent on the application

The window should capture all of the information for the decision tree to separate different classes



A few tips for selecting the right size:



- Selection by the user **after data analysis and visualization**
- If the **motion is repetitive** at some frequency, ideal window size should be long enough to capture the entire motion pattern
- Short window length could make the **response quick**
- **Consider the slowest varying signal**



Decision Tree Output

- Decision tree results are accessible through dedicated registers in the sensor
- An interrupt is generated when the decision tree result changes
Interrupt status bits are available in dedicated registers
- Interrupt can be routed to INT1/2 pad through dedicated register

Register	Content
MLC0_SRC (70h)	Result of decision tree 1
MLC1_SRC (71h)	Result of decision tree 2
MLC2_SRC (72h)	Result of decision tree 3
MLC3_SRC (73h)	Result of decision tree 4
MLC4_SRC (74h)	Result of decision tree 5
MLC5_SRC (75h)	Result of decision tree 6
MLC6_SRC (76h)	Result of decision tree 7
MLC7_SRC (77h)	Result of decision tree 8

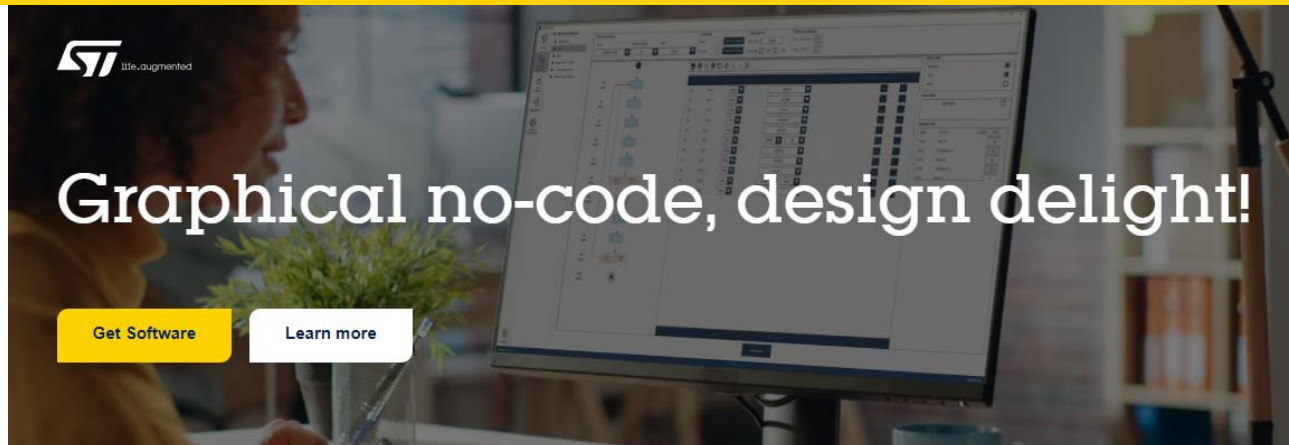
Register	Content
MLC_STATUS_MAINPAGE (38h)	Contains interrupt status bits for changes in the decision tree result
MLC_STATUS (15h)	Contains interrupt status bits for changes in the decision tree result
MLC_INT1 (0Dh)	Allows routing of interrupt status bits for decision trees to INT1 pin
MLC_INT2 (11h)	Allows routing of interrupt status bits for decision trees to INT2 pin

MLC Configuration Demo!



MEMS-Studio

One desktop software solution for a 360-degree MEMS sensor portfolio experience



Analyze data



Develop embedded AI features



Design no-code algorithms



Evaluate embedded libraries

MLC Configuration Demo

For this demo we will use the same sensor and sensor configuration as the motion intensity example available on github for the LIS2DUXS12 3D accelerometer.

https://github.com/STMicroelectronics/st-mems-machine-learning-core/tree/main/examples/motion_intensity/lis2duxs12

The github example classifies 8 motion intensities (0-7), we will limit this demo to 3 levels and reduce the training data to save time!

Sensor: LIS2DUXS12 Accelerometer

FS: 2g

ODR: 12.5Hz

Classes:

Motion Intensity 0 (MI0): board held in hand

Motion Intensity 1 (MI1): board shaken gently

Motion Intensity 2 (MI2): board shaken vigorously

NOTE: This section is intended to be a live demo, showing how the flow to generate mlc config file works using MEM Studio

Step 1 – Capture Data

Prepare Board for Data Capture

- Use Datalog Extended Firmware from X-CUBE-MEMS1 Package
 - **Note:** Firmware is Board Specific – use the right combo for the board stack you have
 - X-CUBE-MEMS1\Projects\NUCLEO-U575ZI-Q\Examples\IKS4A1\DataLogExtended\Binary
- Use STM32CubeProgrammer to flash the DatalogExtended binary to the board
- Disconnect from board
- Reset the board

Step 1 – Capture Data

Configure Sensors for Data Capture

- Open MEMS Studio
- Connect to the board
- Select LIS2DUXS12 as accelerometer and click select
- In Quick Setup tab, select accelerometer FS=2g and ODR 12.5Hz (these match Motion Intensity example on github)
- Click Save to File tab
- Set log file location, e.g. C:\ST67_Workshop\MLC_Demo\
- In Data box, right click then unselect all. Then check Timestamp and Accelerometer.
- Ensure Time base is set to timestamp

Step 1 – Capture Data

Run Data Capture

- Open Line charts tab to show accel data
- Click Play button (top left) to start data capture
- Hold board in hand for 20s – this is motion intensity 0
- Gently shake the board along all 3 axes for 20s – this is motion intensity 1
- Vigorously shake the board along all 3 axes for 20s – this is motion intensity 2
- Stop the capture by pressing stop button in top left corner of MEMS Studio

Step 2 – Label Data

- Open Data Analysis tab
- Load the accel data capture file you created with the 3 types of motion
- Use time domain graph
- Click on graph to select regions and label Motion Intensity 0, 1 and 2: MI0, MI1, MI2
- Once all 3 MI's are labeled, click Save Labeled Data to generate 3 individual motion capture files, save them to same folder as original data capture file, e.g: C:\ST67_Workshop\MLC_Demo
- Use file explorer to show that 3 files have been created with label in file name

Step 3 – Build Decision Tree

Load Data Patterns

- Open Advanced Features tab and select MLC
- Make sure Data Patterns tab is selected
- Set workspace to C:\ST67_Workshop\MLC_Demo
- Select the LIS2DUXS12 sensor
- Using Browse button or file explorer drag & drop to load MI0 file. Assign class label MI0 to it. Click “Load”.
- Repeat above step for MI1 and MI2
- You should have 3 pattern files loaded with classes MI0 through MI2

Step 3 – Build Decision Tree

Configure AFS

- Open AFS tab
- Set Machine Learning Core Settings: ODR 12.5 to match captured data, Window length=25 (2s of data)
- Configure Inputs (to match captured data)
 - Accelerometer_only
 - Full Scale 2g
 - ODR: 12.5
- Leave AFS settings at default and click Run to start automatic feature selection
- Once AFS completes, click Apply to Settings to transfer filters/features to ARFF panel

Step 3 – Build Decision Tree

ARFF (Attribute-Relation File Format) Generation

- Open ARFF Generation tab
- Note settings from AFS have been set, review them
- Click Generate ARFF File to generate the ARFF file used in the next step (DT generation)

Step 3 – Build Decision Tree

Decision Tree Generation

- Open Decision Tree Generation tab
- Set number of decision trees to 1
- Click Generate Decision Tree
- Once DT is generated review results and show generated decision tree (show Dectree button)

Step 3 – Build Decision Tree

Generation of Configuration Files

- Open Config generation tab
- For output values for the classes select 0, 1 and 2 to match MI0,1 and 2
- Leave counters at 0
- Click Generate Config file
- Show json file
- Click Export to show how to generate corresponding header file

Step 3 – Build Decision Tree

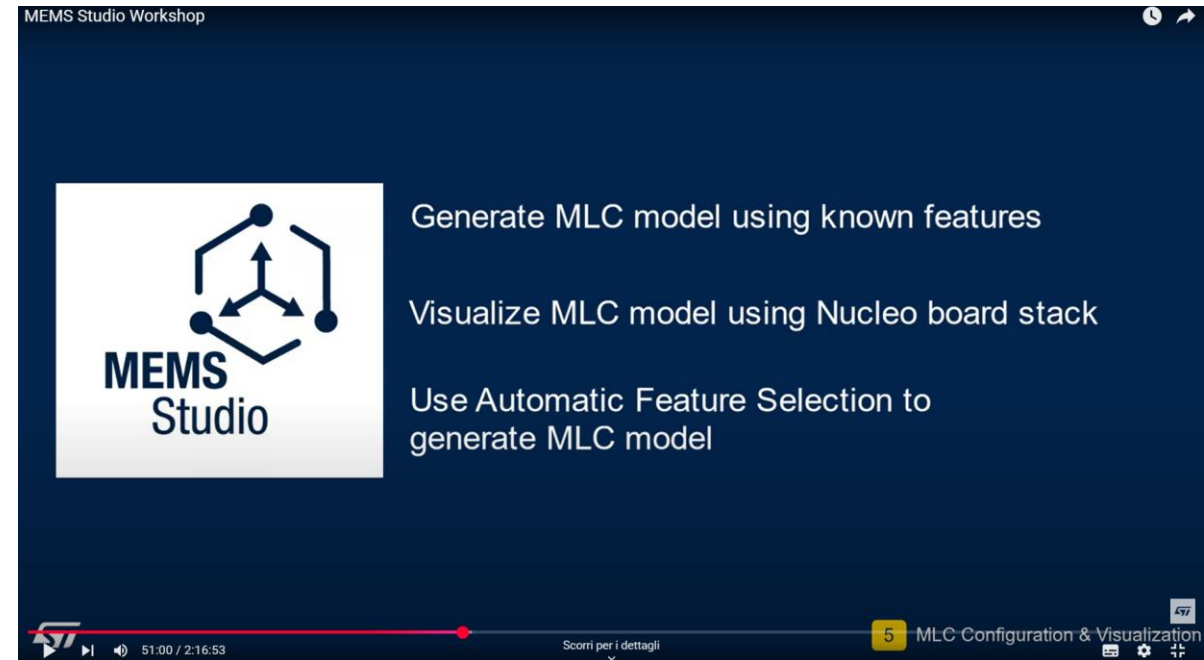
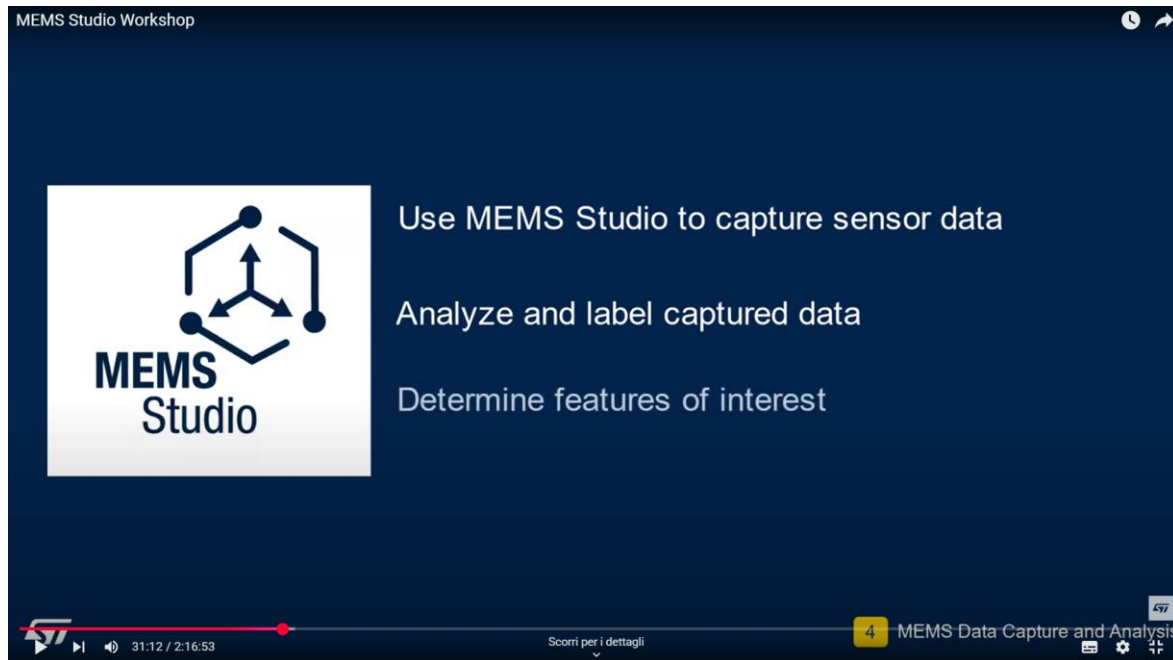
Test Generated Tree

- Click Load Config into Sensor
- Click Decision Tree Output Viewer
- Press Play button to start data capture/testing
- Show Motion Intensities 0, 1 and 2

Resources

MEMS Studio workshop video:

<https://www.youtube.com/watch?v=MbOEQcwzOAAQ> (Module 4 & 5)



Hands-On: Embed MLC into MQTT App



Enabling MLC in Your STM32 Project

- In this hands-on part of the workshop, we'll use a generated MLC example available on ST's github
- Github: <https://github.com/STMicroelectronics/st-mems-machine-learning-core>
- We'll use the LIS2DUXS12 sensor

[st-mems-machine-learning-core](#) / [examples](#) / [motion_intensity](#) / [lis2duxs12](#) / 



lorenzobracco first version 

Name	Last commit message
 ..	
 README.md	first version
 lis2duxs12_motion_intensity.h	first version
 lis2duxs12_motion_intensity.json	first version

Enabling MLC in Your STM32 Project

- The README file provides useful information for integration
- Overview, Sensor configuration, MLC outputs, interrupts

1 - Introduction

Simple example of motion intensity detection implemented using the variance feature on the accelerometer norm. The classes recognized in this example are eight different levels of intensity, from 0 to 7.

For information on how to integrate this algorithm in the target platform, please follow the instructions available in the README file of the [examples](#) folder.

For information on how to create similar algorithms, please follow the instructions provided in the [tutorials](#) folder.

2 - Sensor configuration and orientation

The accelerometer is configured with $\pm 2\text{ g}$ full scale and 12.5 Hz output data rate.

Any sensor orientation is allowed for this algorithm.

3 - Machine Learning Core configuration

Just one feature (variance), has been applied to the norm of the accelerometer data. The MLC runs at 12.5 Hz, computing features on windows of 39 samples (corresponding to almost 3 seconds). One decision tree with 7 nodes has been configured to detect the different classes. A meta-classifier has not been used.

- MLC0_SRC (70h) register values
 - 0 = Intensity_0
 - 1 = Intensity_1
 - 2 = Intensity_2
 - 3 = Intensity_3
 - 4 = Intensity_4
 - 5 = Intensity_5
 - 6 = Intensity_6
 - 7 = Intensity_7

4 - Interrupts

The configuration generates an interrupt (pulsed and active high) on the INT1 pin every time the register MLC0_SRC (70h) is updated with a new value. The duration of the interrupt pulse is 77 ms in this configuration.

Enabling MLC in Your STM32 Project

- Folder contains 3 files:
 - README.md – useful info
 - JSON config file – useful for testing using ST Tools e.g MEMS Studio
 - C header file – useful for integration into STM32 projects

Name
..
README.md
lis2duxs12_motion_intensity.h
lis2duxs12_motion_intensity.json

- We'll use the header file to enable the MLC in our MQTT Project
- We'll publish the detected motion intensity to the MQTT broker and it will be displayed in the mobile app.

Enabling MLC in Your STM32 Project

- Header file contains an array with a sequence of commands to execute to configure the sensor and MLC
- Each array entry contains an opcode and relevant info, such as register addresses and data

```
static const struct mems_conf_op lis2duxs12_motion_intensity_conf_0[] = {  
    { .type = MEMS_CONF_OP_TYPE_WRITE, .address = 0x13, .data = 0x10 },  
    { .type = MEMS_CONF_OP_TYPE_DELAY, .data = 5 },  
    { .type = MEMS_CONF_OP_TYPE_WRITE, .address = 0x14, .data = 0x00 },  
    { .type = MEMS_CONF_OP_TYPE_WRITE, .address = 0x3F, .data = 0x80 },  
    { .type = MEMS_CONF_OP_TYPE_WRITE, .address = 0x04, .data = 0x00 },  
    { .type = MEMS_CONF_OP_TYPE_WRITE, .address = 0x05, .data = 0x00 },  
    { .type = MEMS_CONF_OP_TYPE_WRITE, .address = 0x17, .data = 0x40 },  
    { .type = MEMS_CONF_OP_TYPE_WRITE, .address = 0x02, .data = 0x01 },  
    { .type = MEMS_CONF_OP_TYPE_WRITE, .address = 0x08, .data = 0xB8 },  
    { .type = MEMS_CONF_OP_TYPE_WRITE, .address = 0x09, .data = 0xE6 },  
    { .type = MEMS_CONF_OP_TYPE_WRITE, .address = 0x09, .data = 0x00 },  
    { .type = MEMS_CONF_OP_TYPE_WRITE, .address = 0x09, .data = 0xF0 },  
}
```

Enabling MLC in Your STM32 MQTT Project

- We've added the necessary code to:
 - configure the MLC
 - read MLC outputs
 - publish the motion intensity to the MQTT broker
- Added File:
 - Projects\NUCLEO-U575ZI-Q\Demonstrations\ST67W6X\ST67W6X_MQTT\X-CUBE-MEMS1\App\lis2duxs12_motion_intensity.h
- Modified files:
 - Projects\NUCLEO-U575ZI-Q\Demonstrations\ST67W6X\ST67W6X_MQTT\X-CUBE-MEMS1\App\sys_sensors.h
 - Projects\NUCLEO-U575ZI-Q\Demonstrations\ST67W6X\ST67W6X_MQTT\X-CUBE-MEMS1\App\sys_sensors.c
 - Projects\NUCLEO-U575ZI-Q\Demonstrations\ST67W6X\ST67W6X_MQTT\App_MQTT\App\app_config.h
 - Projects\NUCLEO-U575ZI-Q\Demonstrations\ST67W6X\ST67W6X_MQTT\App_MQTT\App\main_app.c

Enabling MLC in Your STM32 Project

- Modified files: sys_sensors.h

```
26 /* Includes ----- */
27 /* Config ----- */
28 #define SENSOR_ENABLED 1
29 #define MLC_CONFIGURATION lis2duxs12_motion_intensity_conf_0
30
```

```
42 typedef struct
43 {
44     float pressure;      /*!< in mbar */
45     float temperature;   /*!< in degC */
46     float humidity;      /*!< in % */
47     Sys_Sensors_Axes_t acceleration;
48     Sys_Sensors_Axes_t magnetic_field;
49     Sys_Sensors_Axes_t angular_velocity;
50 #ifdef USE_MLC
51     uint8_t motion_intensity; /* motion intensity from mlc, values from 0->7 */
52 #endif
53 } Sys_Sensors_Data_t;
54
```

Enabling MLC in Your STM32 Project

- Modified files: sys_sensors.c

```
#include "logging.h"

#include "iks4a1_motion_sensors_ex.h"
#include "lis2dups12_motion_intensity.h" /* MLC Config file */
```

```
/* Private function prototypes -----*/
static int32_t MLC_Init(void);
```

```
int32_t Sys_Sensors_Read(Sys_Sensors_Data_t *data)
{
...
#ifdef USE_MLC
    /* Get mlc outputs */
    (void)IKS4A1_MOTION_SENSOR_Write_Register(IKS4A1_LIS2DUXS12_0, LIS2DUXS12_FUNC_CFG_ACCESS, 0x80);
    (void)IKS4A1_MOTION_SENSOR_Read_Register(IKS4A1_LIS2DUXS12_0, LIS2DUXS12_MLC1_SRC, &data->motion_intensity);
    (void)IKS4A1_MOTION_SENSOR_Write_Register(IKS4A1_LIS2DUXS12_0, LIS2DUXS12_FUNC_CFG_ACCESS, 0x00);
#endif
...
}
```


Enabling MLC in Your STM32 Project

- Modified files: sys_sensors.c

```
static int32_t Sys_Sensors_IKS4A1_Init(void)
{
...
#ifdef USE_MLC
    /* Initialize the MLC */
    ret = MLC_Init();
#endif
...
}
```

Enabling MLC in Your STM32 Project

- Modified files: sys_sensors.c

```
int32_t MLC_Init(void)
{
    ...
    length = MEMS_CONF_ARRAY_LEN(MLC_CONFIGURATION);
    i = 0;
    while ((i < length) & (ret == BSP_ERROR_NONE))
    {

        if (MLC_CONFIGURATION[i].type == MEMS_CONF_OP_TYPE_DELAY)
        {
            HAL_Delay(MLC_CONFIGURATION[i].data);
        }
        else if (MLC_CONFIGURATION[i].type == MEMS_CONF_OP_TYPE_WRITE)
        {
            ret = IKS4A1_MOTION_SENSOR_Write_Register(IKS4A1_LIS2DUXS12_0, MLC_CONFIGURATION[i].address,
MLC_CONFIGURATION[i].data);
        }
        else
        {
            /* Unsupported opcode */
            ret = BSP_ERROR_FEATURE_NOT_SUPPORTED;
        }
        i++;
    }
    ...
}
```

Enabling MLC in Your STM32 Project

- Modified files: main_app.c

```
static void Subscription_process_task(void *arg)
{
...
/* Process the field 'motionintensity'. Value type: Int. Format: 1 (range 0-7) */
json = cJSON_GetObjectItemCaseSensitive(child, "motionintensity");
if (json != NULL)
{
    if (cJSON_IsNumber(json) == true)
    {
        LogInfo("  %s: %d\n", json->string, json->valueint);
    }
    else
    {
        LogError("JSON parsing error of %s value.\n", json->string);
    }
}
...
}
```

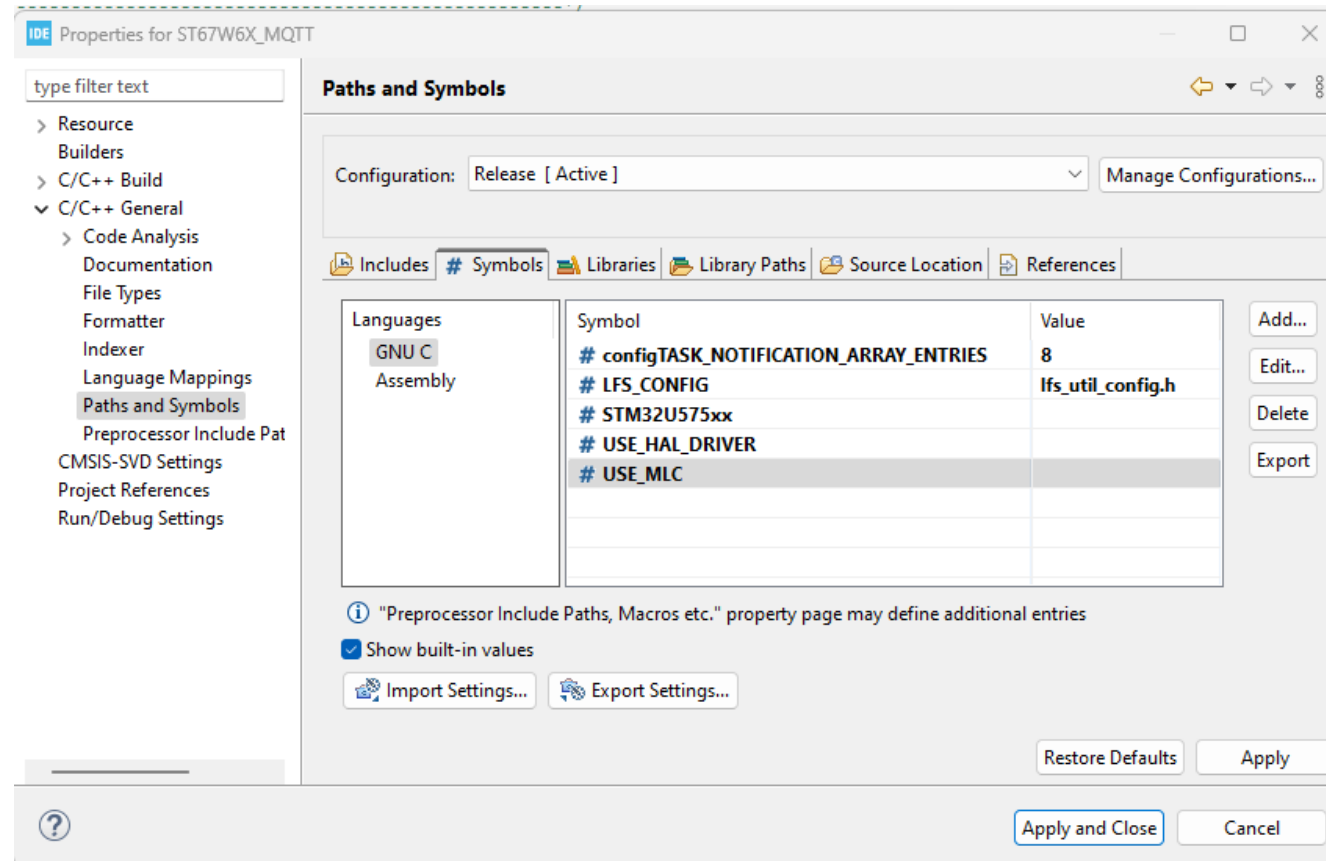
Enabling MLC in Your STM32 Project

- Modified files: main_app.c

```
#ifdef USE_MLC
    len += snprintf((char *)&mqtt_pubmsg[len], MQTT_MSG_BUFFER_SIZE - len,
                    ", \"temperature\": %.2f, \"humidity\": %.2f, \"pressure\": %.2f, \"motionintensity\":%d",
                    (double)sensors_data.temperature, (double)sensors_data.humidity,
                    (double)sensors_data.pressure, (int)sensors_data.motion_intensity);
#else
    len += snprintf((char *)&mqtt_pubmsg[len], MQTT_MSG_BUFFER_SIZE - len,
                    ", \"temperature\": %.2f, \"humidity\": %.2f, \"pressure\": %.2f",
                    (double)sensors_data.temperature, (double)sensors_data.humidity,
                    (double)sensors_data.pressure);
#endif
```

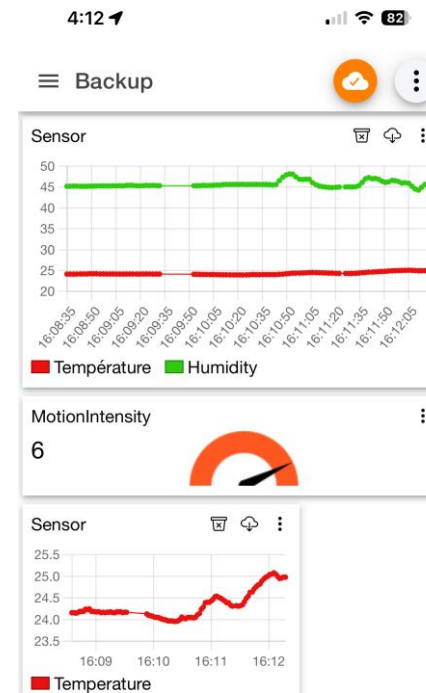
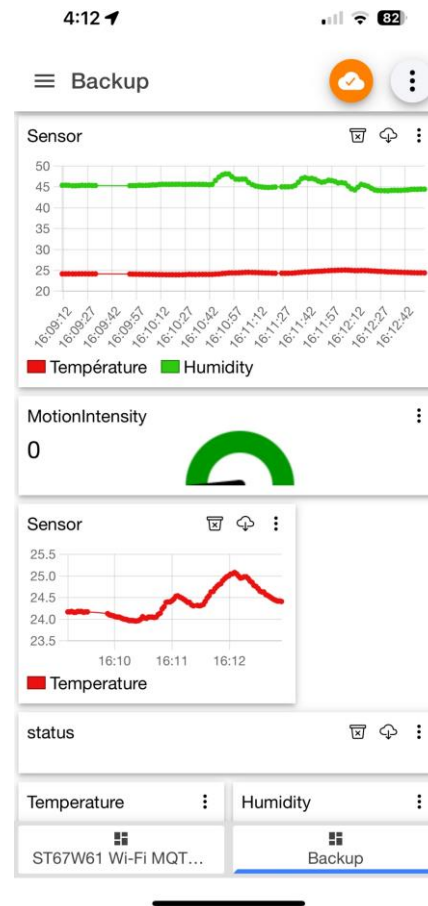
Enabling MLC in Your STM32 Project

- To enable the MLC code, add USE_MLC to project properties

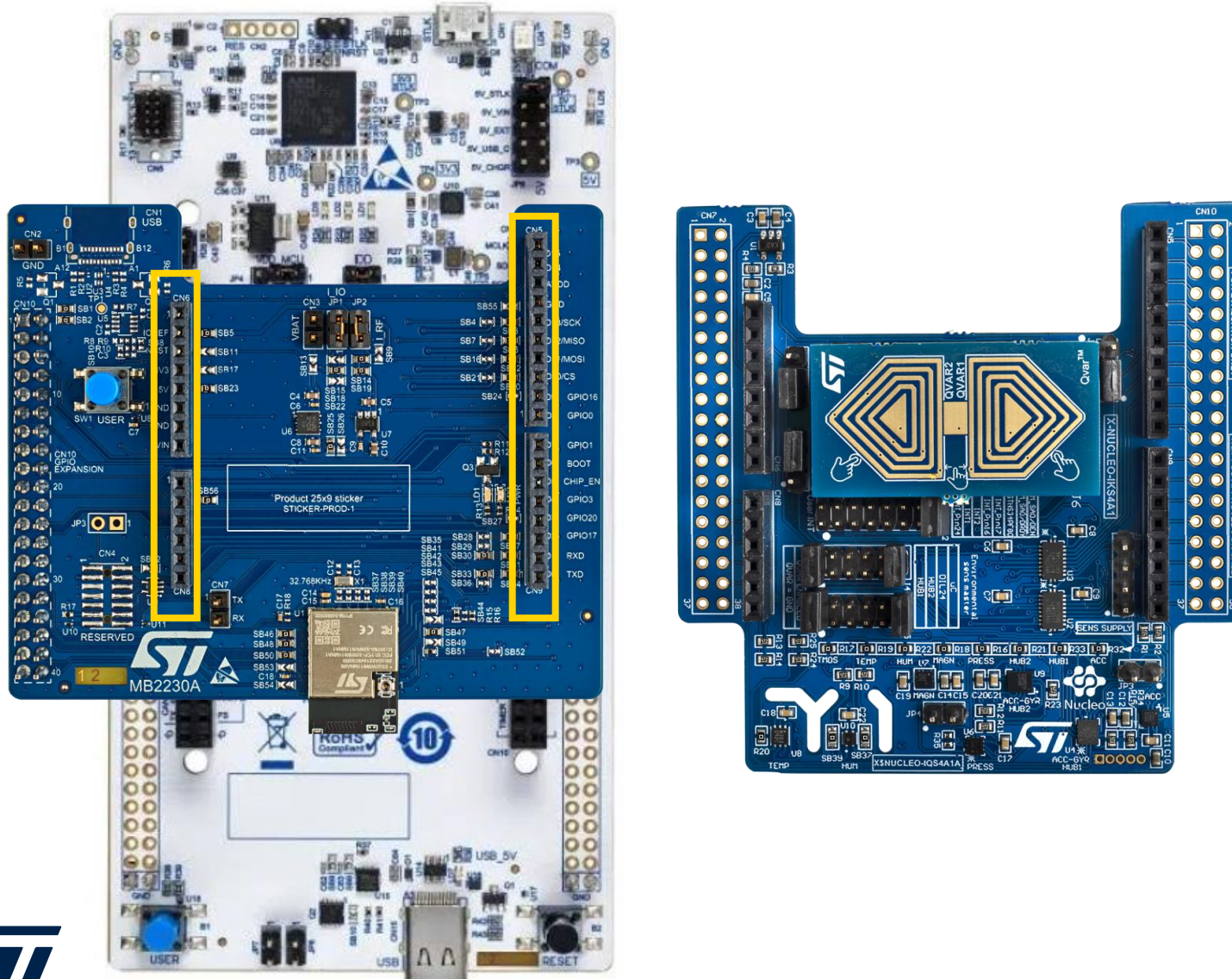


Enabling MLC in Your STM32 Project

- Open the IoTMQTTPanel app and load the configuration file you generated
- Move/shake the board to see motion intensity value change in the Backup panel



Add X-NUCLEO-IKS4A1 to Board Stack



- Align the Arduino connectors
- Press boards together firmly
- Ensure User button on WiFi board is not held down by stacked X-NUCLEO-IKS4A1 board

MEMS Sensors Resources

- Website: <https://www.st.com/mems>
- Github: github.com/STMicroelectronics
- MEMS Studio workshop video:
<https://www.youtube.com/watch?v=MbOEQcwzOAA> (Module 4 & 5)
- Local Marketing contact: alexandra.gogonea@st.com
- Online Support:
 - https://www.st.com/content/st_com/en/support/support-home.html
 - ols.st.com



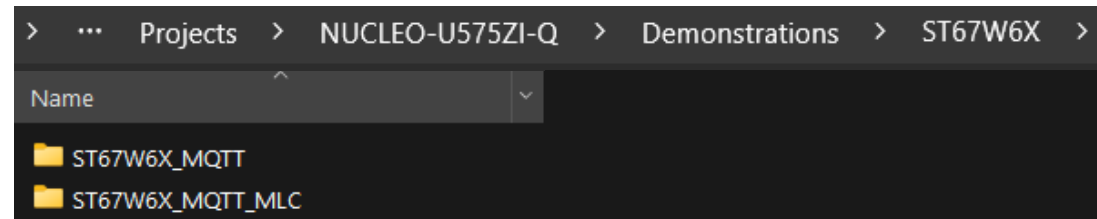
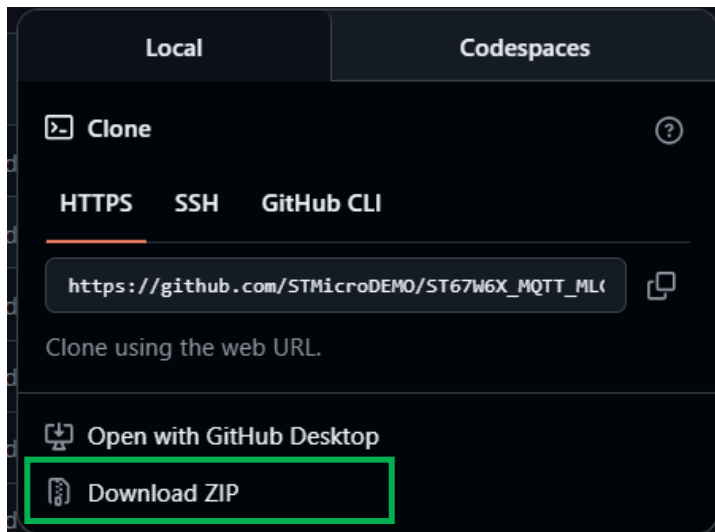
life.augmented

Hands-On: Integrating Embed MLC into MQTTS

ST67W6X MQTT

Hands-On

- In this hands-on we will use a modified MQTT project that includes the MLC portion as well as additional resources located on GitHub
- [STMicroDEMO/ST67W6X_MQTT_MLC](https://github.com/STMicroDEMO/ST67W6X_MQTT_MLC)
- Download the project from GitHub and save the file uncompressed within the X-Cube package:
 - C:\X_CUBE_ST67W61_V1.0.0\Projects\NUCLEO-U575ZI-Q\Demonstrations\ST67W6X



***Note: GitHub may save the project as “ST67W6X_MQTT_MLC-main”*

ST67W6X MQTT

Higher security schemes

- Higher level security schemes can be enabled to utilize TLS to prevent third party interception of data when publishing data to the cloud.
- Security Schemes:
 - 0 - MQTT over TCP with no authentication and no encryption
 - 1 - MQTT over TLS with username authentication
 - 2 - MQTT over TLS with server certificate
 - 3 - MQTT over TLS with client certificate
 - 4 - MQTT over TLS with both certificates
- The different configurations must be followed when configuring the broker and client.

```
127  /* MQTT Broker connection */
128  static W6X_MQTT_Connect_t mqtt_connect =
129  {
130      .HostName = MQTT_HOST_NAME,          /*!< Host name of remote MQTT Broker */
131      .HostPort = MQTT_HOST_PORT,          /*!< Port of remote MQTT Broker */
132      .Scheme = MQTT_SECURITY_LEVEL,        /*!< Security level. only non-secure is currently available */
133      .MQClientId = MQTT_CLIENT_ID,         /*!< MQTT Client ID to be identified on MQTT Broker */
134      .MQUserName = MQTT_USERNAME,          /*!< MQTT Username to be identified on MQTT Broker. Not used in non-secure */
135      .MQUserPwd = MQTT_USER_PASSWORD,      /*!< MQTT Password to be identified on MQTT Broker. Not used in non-secure */
136      .Certificate = MQTT_Certificate,       /*!< Client Certificate. Required when the scheme is greater or equal to 2 */
137      .PrivateKey = MQTT_Key,               /*!< Client Private key. Required when the scheme is greater or equal to 2 */
138      .CACertificate = MQTT_CA_CERTIFICATE, /*!< CA certificate. Required when the scheme is greater or equal to 2 */
139      .SNI_enabled = 0
140  };
```

mqtt_connect structure that must be configured for each MQTT use case.

ST67W6X MQTT

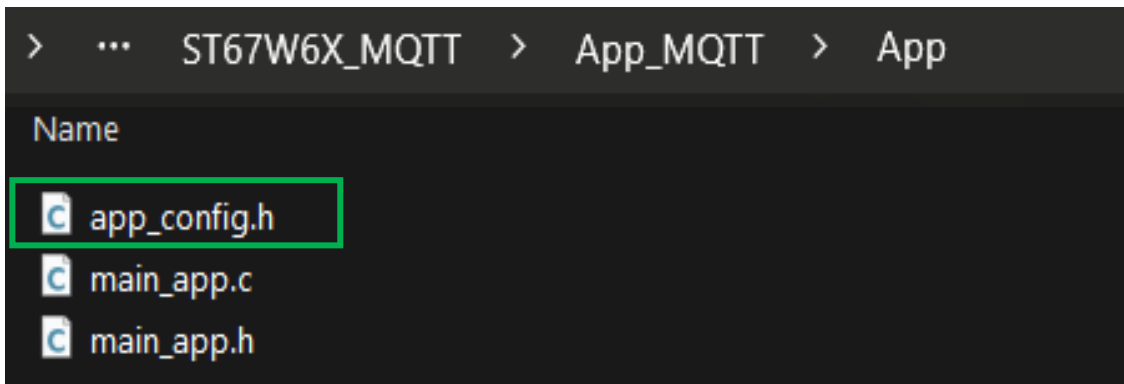
Enabling Higher security schemes

- When enabling security schemes 2 or higher, certificates and keys must be provided to verify security credentials.
- The NCP utilizes littlefs to read certificates and keys from the host MCU that are then stored internally.
- To load certificates and keys onto the host MCU, desired certificates and keys must be placed within the lfs folder in the MQTT demonstration folder:
 - *Projects\NUCLEO-U575ZI-Q\Demonstrations\ST67W6X\ST67W6X_MQTT\littlefs\lfs*
- The desired certificates and keys are loaded onto the host MCU as a littlefs.bin using linker input specified in the project settings.
- This binary can be built from the user specified certificates and keys using the build.bat script once the certificates and keys are placed in the lfs folder.

ST67W6X MQTT

ST67 Client Setup - Security Scheme 1

- In this hands-on portion of the work-shop TLS will be utilized within the MQTT demo using security scheme 1.
- Security scheme 1 - MQTT over TLS with username and password
- MQTT Local Client configuration – Setup on ST67 within CubeIDE in app_config.h

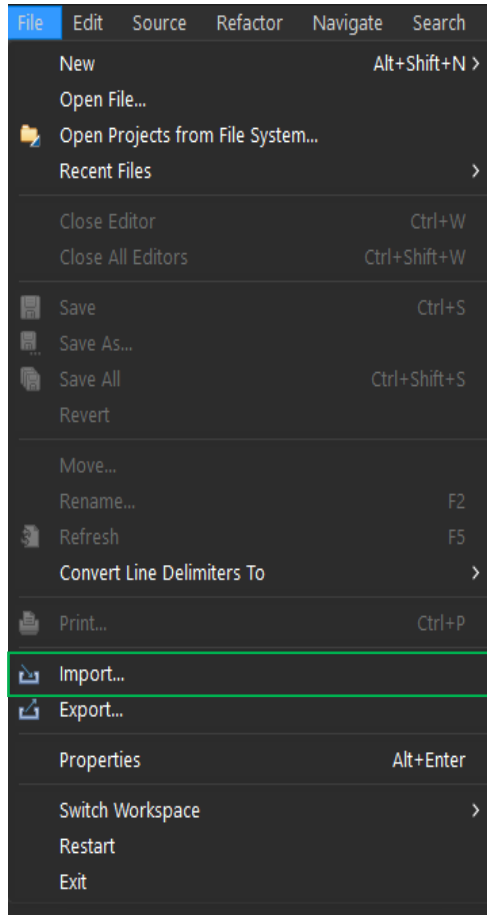


ST67W6X MQTT

Import Project into STM32CubeIDE

Import the modified MQTT application into CubeIDE

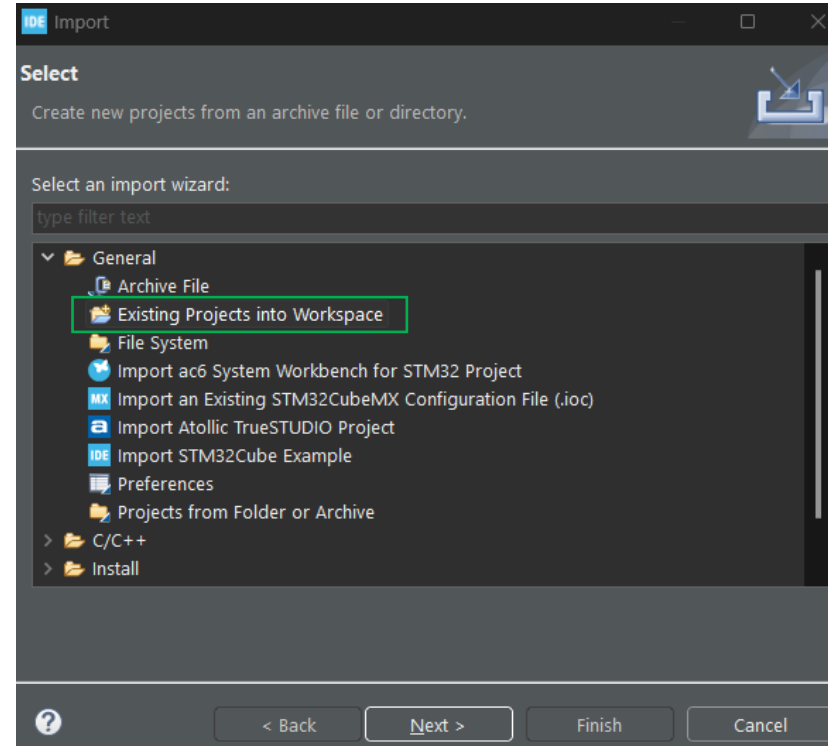
1. Select “File”
2. Select “Import...”



ST67W6X MQTT

Import Project into STM32CubeIDE

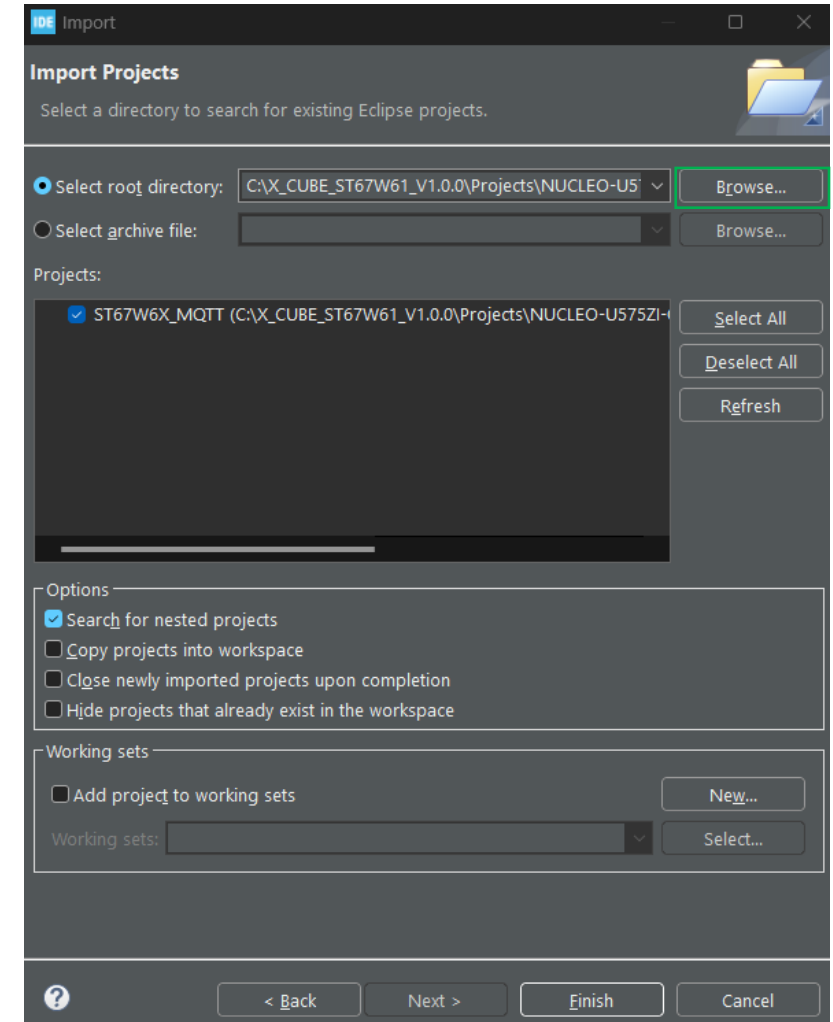
3. Select “Existing Projects into Workspace”
4. Select “Next”



ST67W6X MQTT

Import Project into STM32CubeIDE

5. Select “Browse...”
6. Navigate to “C:\x-cube-st67w61-v1.0.0\Projects\NUCLEO-U575ZI-Q\Demonstrations\ST67W6X\ST67W6X_MQTT_MLC”
7. Select “Finish”

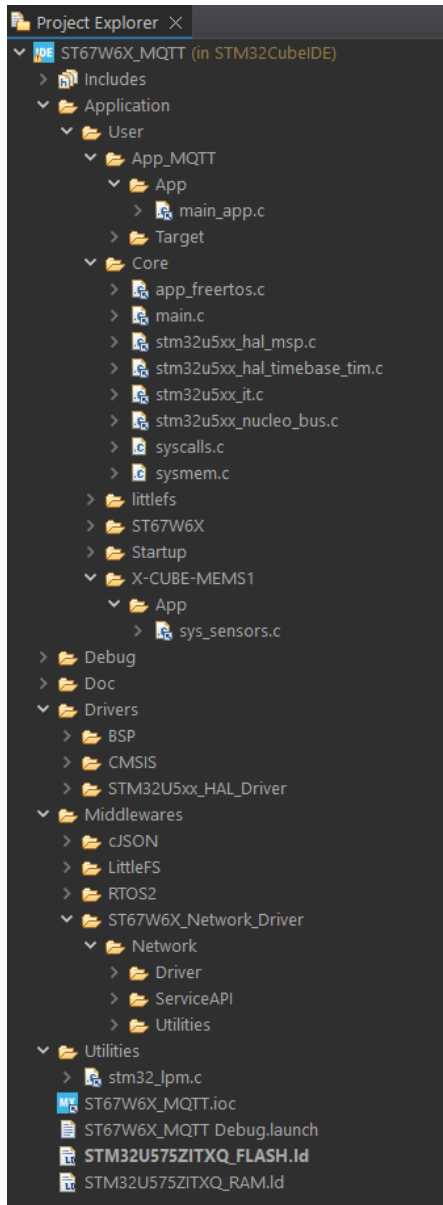


ST67W6X MQTT

Project into STM32CubeIDE

Project Structure:

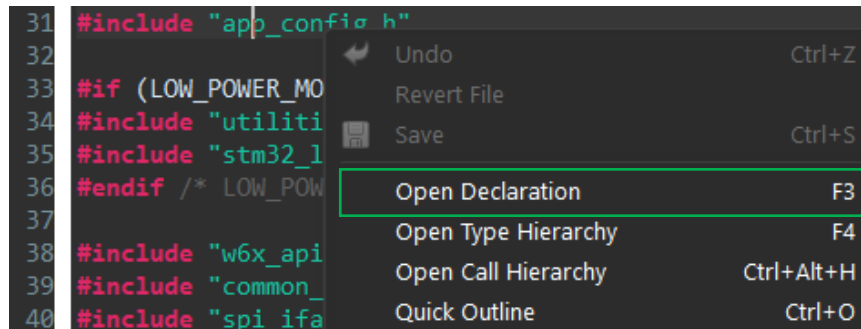
- Application – Contains the files for the application functionality (i.e. main.c, main_app.c, sys_sensors.c, etc.)
- Drivers – Contains the HAL driver, CMSIS, and BSP libraries for STM32U575 and IKS4A1.
- Middlewares – Contains the network driver libraries for the ST67 modules.
- Utilities – Contains the stm32_lpm.c file.



Open app_config.h:

- Open the file main_app.c found in the Application library under User/App_MQTT/App.
- With main_app.c open, to open app_config.h, “right-click” and select “Open Declaration” on app_config.h or hold CTRL and “left-click” on app_config.h.**

```
31 #include "app_config.h"
32
33 #if (LOW_POWER_MODE)
34 #include "utilities.h"
35 #include "stm32_l1.h"
36 #endif /* LOW_POWER_MODE */
37
38 #include "w6x_api.h"
39 #include "common.h"
40 #include "spi_if.h"
```



```
28 /* Application */
29 #include "main.h"
30 #include "main_app.h"
31 #include "app_config.h"
```


ST67W6X MQTT

ST67 Client Setup - Security Scheme 1

- Modified app_config.h to enable security scheme 1

```
83 /** Port of remote MQTT Broker */
84 #define MQTT_HOST_PORT      8883
85
86 /** Security level. only non-secure is currently available */
87 #define MQTT_SECURITY_LEVEL  1
88
89 /** MQTT Client ID to be identified on MQTT Broker */
90 #define MQTT_CLIENT_ID      "hands-on"
91
92 /** MQTT Username to be identified on MQTT Broker. Not used in non-secure */
93 #define MQTT_USERNAME       "st67workshop"
94
95 /** MQTT Password to be identified on MQTT Broker. Not used in non-secure */
96 #define MQTT_USER_PASSWORD  "12345678"
97
98 /** MQTT Client Certificate. Required when the scheme is greater or equal to 3 */
99 #define MQTT_CERTIFICATE     "client_1.crt"
100
101 /** MQTTClient Private key. Required when the scheme is greater or equal to 3 */
102 #define MQTT_KEY             "client_1.key"
103
104 /** MQTT Client CA certificate. Required when the scheme is greater or equal to 2 */
105 #define MQTT_CA_CERTIFICATE  "ca_1.crt"
106
107 /** MQTT Server Name Indication (SNI) enabled */
108 #define MQTT_SNI_ENABLED     1
```

- Modifying the port to 8883 corresponding to TLS rather than TCP
- Security scheme set to 1 corresponding to use username and password fields
- Specified to unique client ID (not necessary for when enabling security)
- Specified username, please avoid username with spaces
- Specified password



ST67W6X MQTT

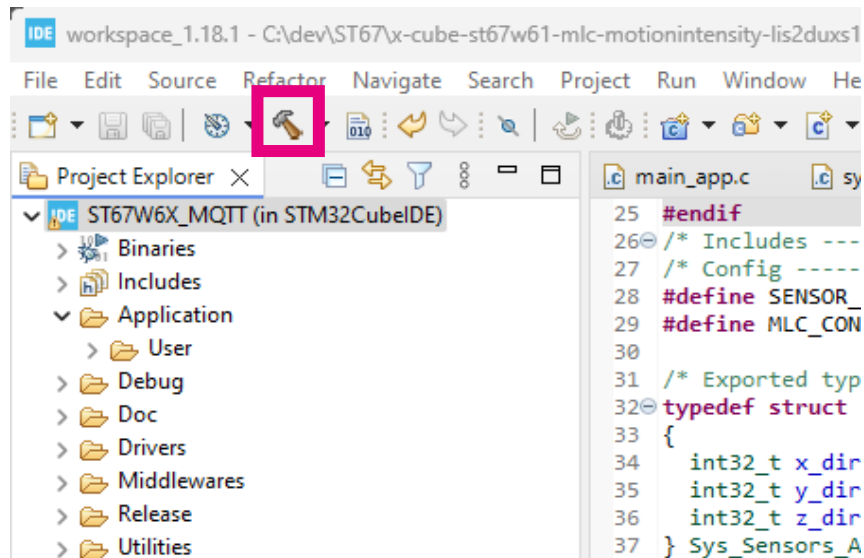
ST67 Client Setup - Security Scheme 1

- Modified app_config.h to provide access point credentials

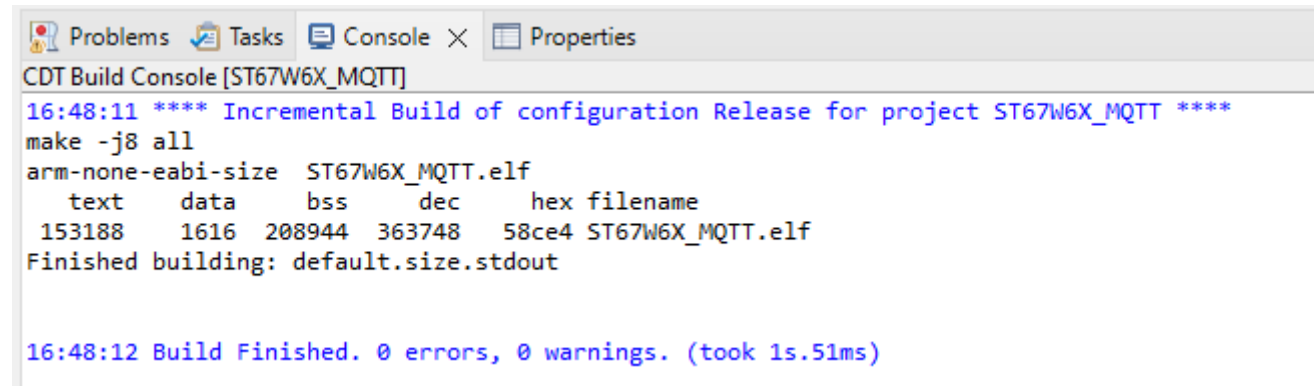
```
61 /* Local Access Point (e.g. gateway, hotspot, etc) connection parameters */
62 #define WIFI_SSID "SSID"
63
64 #define WIFI_PASSWORD "PSWD"
```

*Note: SSID **cannot** have special characters, and both SSID and password should **not** have spaces*

- Once modified with these security changes and with the changes made to enable to MLC portion, save the project and build the application.



- Make sure project builds successfully



```
#### Welcome to ST67W6X MQTT Application ####
# build: 14:31:36 Jul 17 2025
----- Host info -----
Host FW Version: 1.0.0
----- ST67W6X info -----
ST67W6X MW Version: 1.0.0
AT Version: 1.0.0.1
SDK Version: 2.0.75
MAC Version: 1.6.38
Build Date: May 10 2025 10:15:04
Module ID:
BOM ID: 0
Manufacturing Year: 2000
Manufacturing Week: 00
Battery Voltage: 3.340 V
Trim Wi-Fi hp: 6,6,6,6,6,6,5,5,5,5,5,5,4,5
Trim Wi-Fi lp: 5,5,6,6,6,6,7,7,7,7,7,8,8,8,8
Trim BLE: 4,4,4,3,4
Trim XTAL: 38
MAC Address: 40:82:7b:00:0e:c1
Anti-rollback Bootloader: 0
Anti-rollback App: 0
-----
mount success
Wi-Fi init is done
Net init is done
MQTT init is done
```

```
MQTT Publish OK
MQTT Subscription Received on topic "/sensors/hands-on"
time: 25-07-23 18:37:22
rssi: -47
mac: 40:82:7b:00:0e:c1
temperature: 23.55
pressure: 1005.77
humidity: 63.05
motionintensity: 0
MQTT Publish OK
MQTT Subscription Received on topic "/sensors/hands-on"
time: 25-07-23 18:37:24
rssi: -48
mac: 40:82:7b:00:0e:c1
temperature: 23.57
pressure: 1005.78
humidity: 63.16
motionintensity: 0
MQTT Publish OK
MQTT Subscription Received on topic "/sensors/hands-on"
time: 25-07-23 18:37:26
rssi: -48
mac: 40:82:7b:00:0e:c1
temperature: 23.56
pressure: 1005.78
humidity: 63.06
motionintensity: 0
```

ST67W6X MQTT

Nucleo-U575ZI-Q Serial Terminal Output

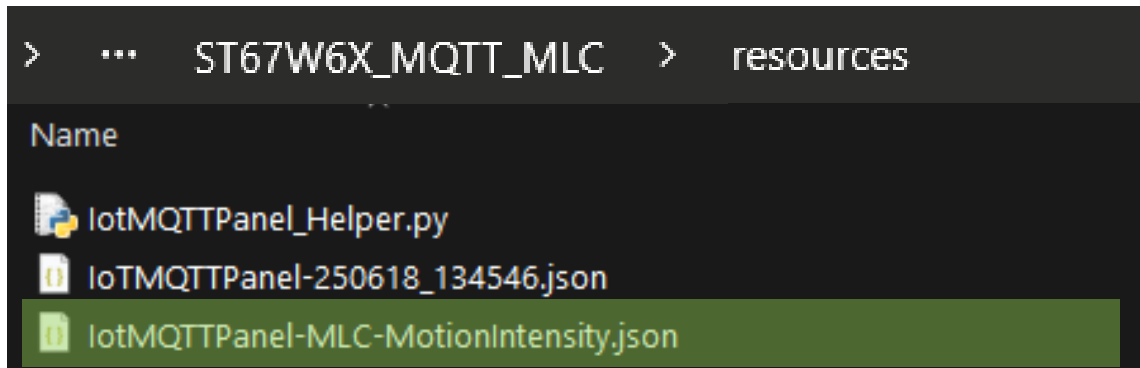
Application running and displaying messages on serial terminal and expected output:

- See FW versions for both host MCU and ST67W6X
- Module initializations (i.e. Wi-Fi, Net, MQTT)
- Scan and connection to AP
- MEMS Init
- MQTT Configure
- MQTT Publish and Subscription received

ST67W6X MQTT

IoT MQTT Panel Client Setup - Security Scheme 1

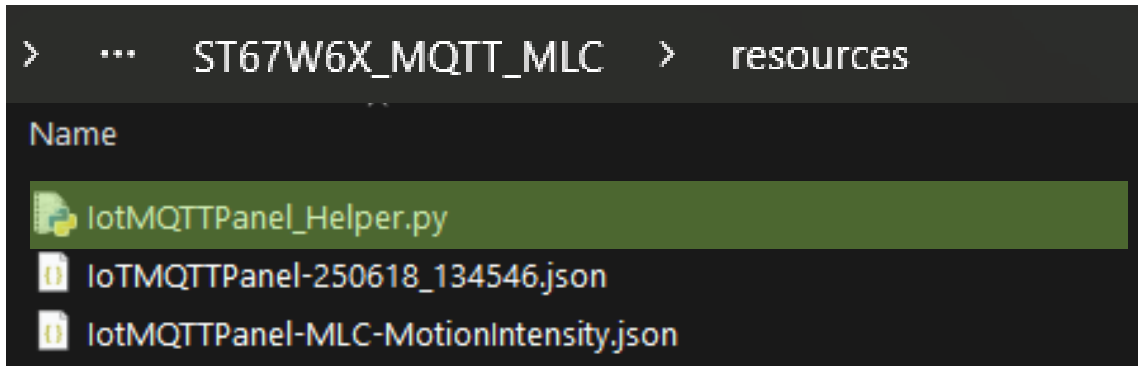
- The IoT MQTT Panel application must be modified to use a username and password as well as match user specifications.
- The IoTMQTTPanel-MLC-MotionIntensity.json is used to configure the IoT MQTT Panel application.



ST67W6X MQTT

IoT MQTT Panel Client Setup - Security Scheme 1

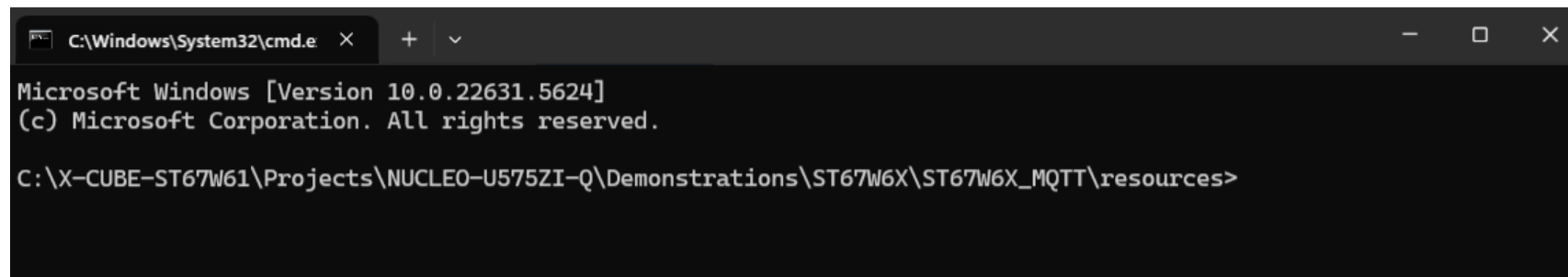
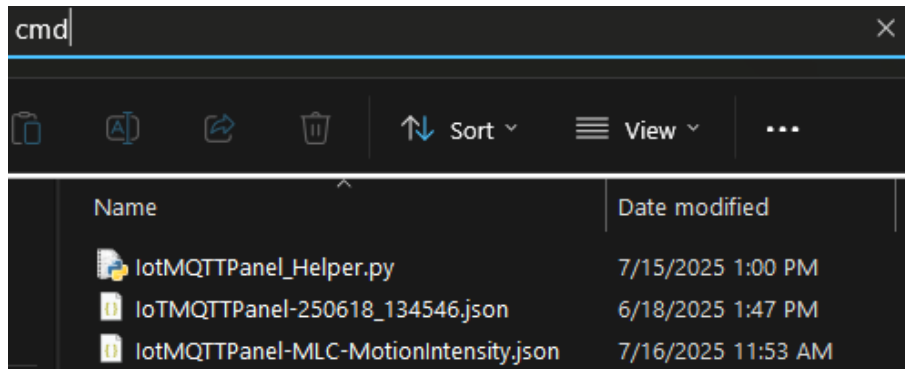
- The .json file can be modified quickly to match the user's specifications using the IoTMQTTPanel_Helper.py script.
- The Python script is located within the project directory at:
 - Projects\NUCLEO-U575ZI-Q\Demonstrations\ST67W6X\ST67W6X_MQTT_MLC\resources



ST67W6X MQTT

IoT MQTT Panel Client Setup - Security Scheme 1

- A command window can be opened at the directory containing the python script by typing in “cmd” in the address bar of the file explorer window.



ST67W6X MQTT

IoT MQTT Panel Client Setup - Security Scheme 1

- To see the available parameters that can be edited within the IoTMQTTPanel_Helper.py provide the option '-h' when calling the script

```
C:\X_CUBE_ST67W61_V1.0.0\Projects\NUCLEO-U575ZI-Q\Demonstrations\ST67W6X\ST67W6X_MQTT_MLC\resources>python IotMQTTPanel_Helper.py -h
usage: IoTMQTTPanel_Helper [-h] [-o OUT] [-b BROKER] [-c CLIENTID] [-u USERNAME] [-p PASSWORD] [--version] JSONin

Pretty print JSON converter and ClientID injection

positional arguments:
  JSONin                Input JSON File

options:
  -h, --help            show this help message and exit
  -o OUT, --out OUT     Output JSON to file
  -b BROKER, --broker BROKER
                        MQTT broker
  -c CLIENTID, --clientid CLIENTID
                        MQTT Client ID to inject
  -u USERNAME, --username USERNAME
                        MQTT Username credentials
  -p PASSWORD, --password PASSWORD
                        MQTT Password credentials
  --version             show program's version number and exit
```

- When calling the script the JSONin file **must** be specified
- For this hands-on we will be using the lotMQTTPanel-MLC-MotionIntensity.json
- We also recommend specifying the “out” file which will save any changes specified in the python script to another .json file

ST67W6X MQTT

IoT MQTT Panel Client Setup - Security Scheme 1

- Configure the .json file with the configurations specified in app_config.h

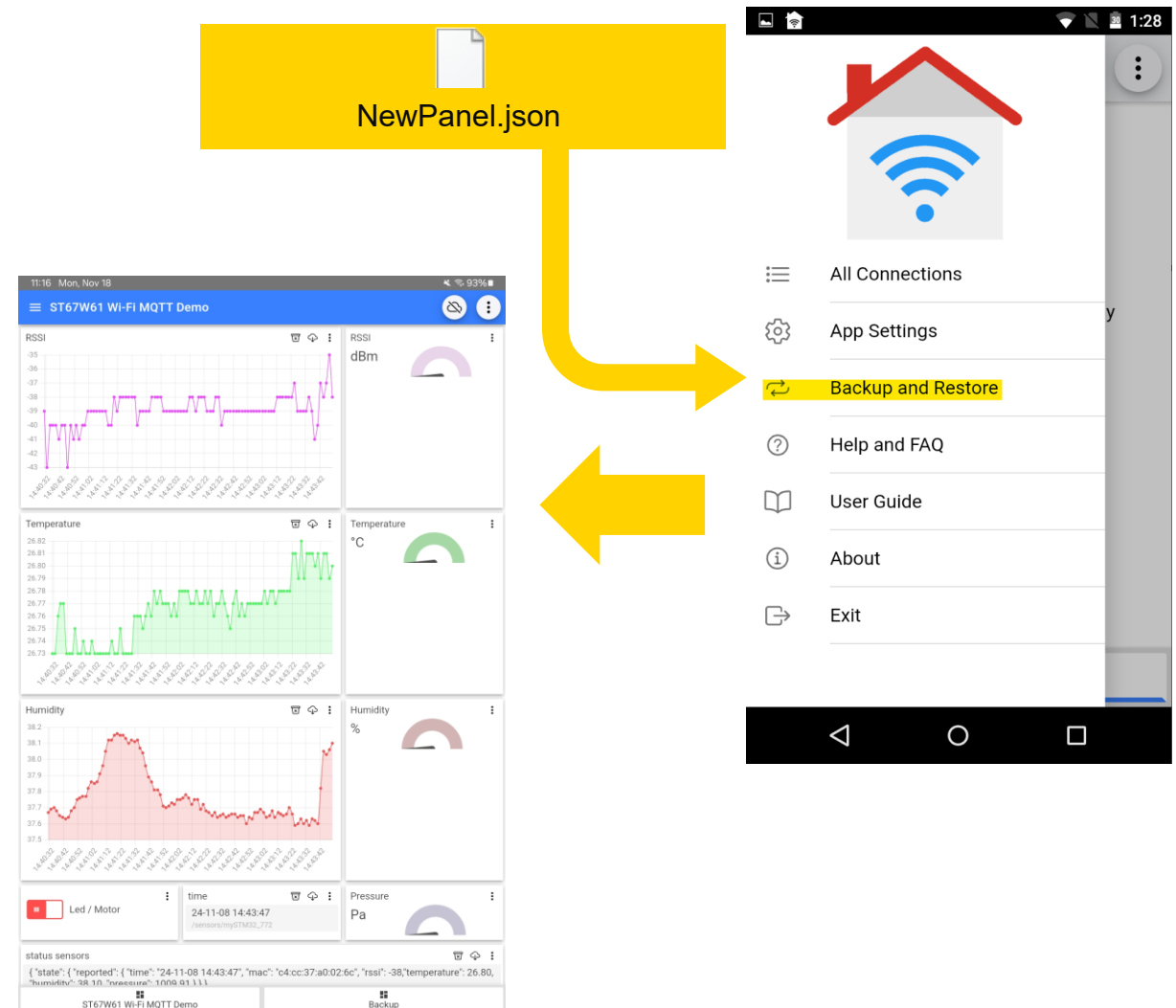
```
C:\X_CUBE_ST67W61_V1.0.0\Projects\NUCLEO-U575ZI-Q\Demonstrations\ST67W6X\ST67W6X_MQTT_MLC\resources>python IotMQTTPanel_Helper.py IotMQTTPanel-MLC-MotionIntensity.json -o NewPanel.json -c hands-on -u st67workshop -p 12345678

IoTMQTTPanel_Helper
JSON in: IotMQTTPanel-MLC-MotionIntensity.json
Inserted ClientID "hands-on" 16 times
Inserted Username "st67workshop" 1 times
Inserted Password "12345678" 1 times
Saving output to: NewPanel.json
Done
```

ST67W6X MQTT

IoT MQTT Panel Client Setup - Security Scheme 1

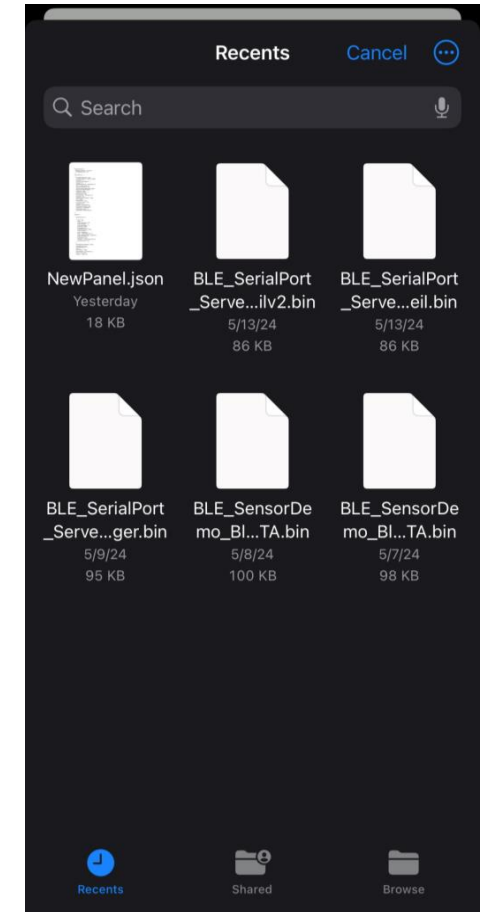
- Setup MQTT Client on IoT MQTT Panel Application
 1. Load modified .json file onto smart device
 2. Restore connection based on configurations set in .json file
 3. Verify connection and data in app with application running on STM32U5+ST67



ST67W6X MQTT

IoT MQTT Panel Client Setup - Security Scheme 1

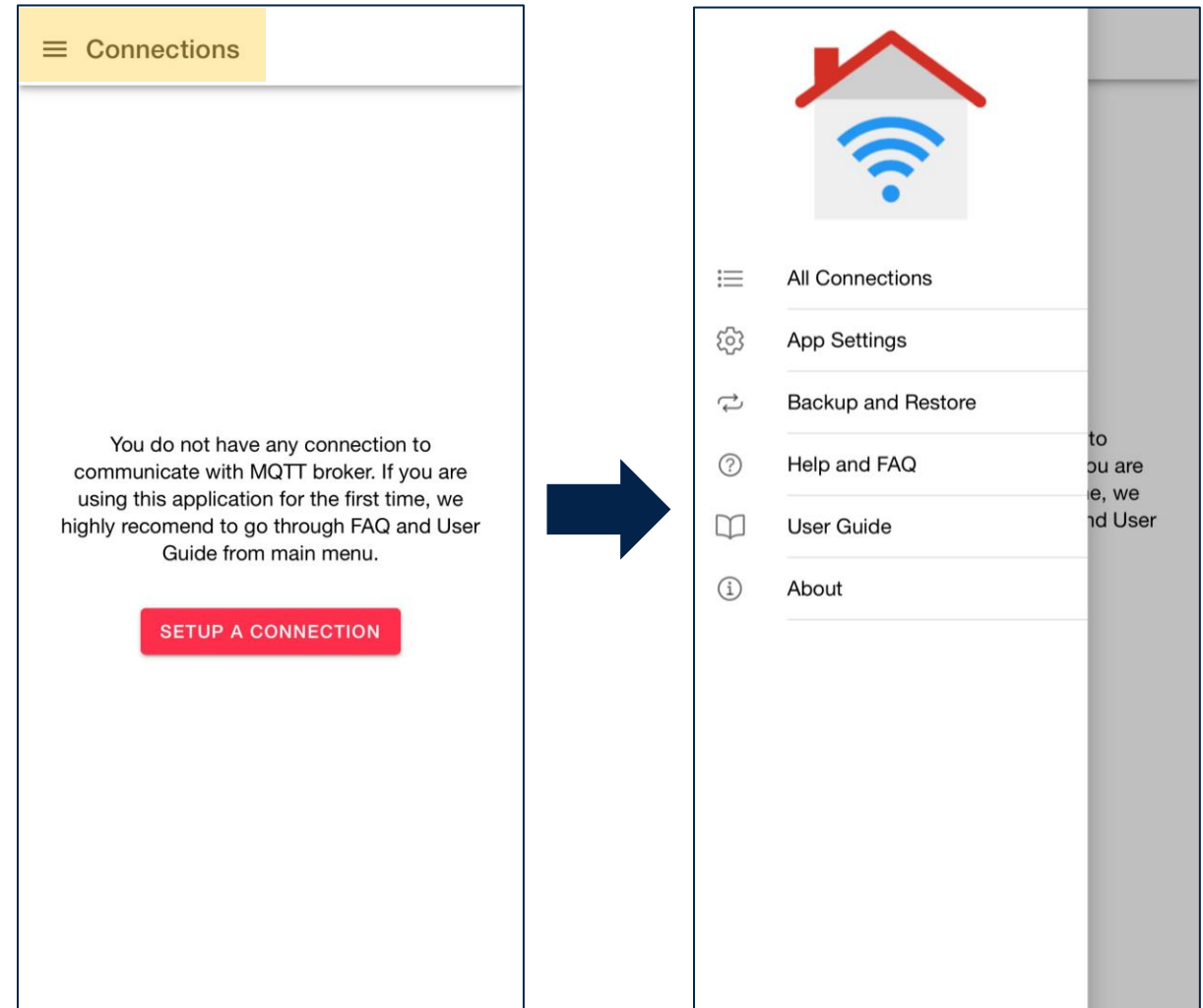
- Modified .json file must be provided to a mobile device with IoT MQTT Panel installed
 - Recommended method is to use a personal email to send the file to your mobile device.



ST67W6X MQTT

IoT MQTT Panel Client Setup – Connection Setup

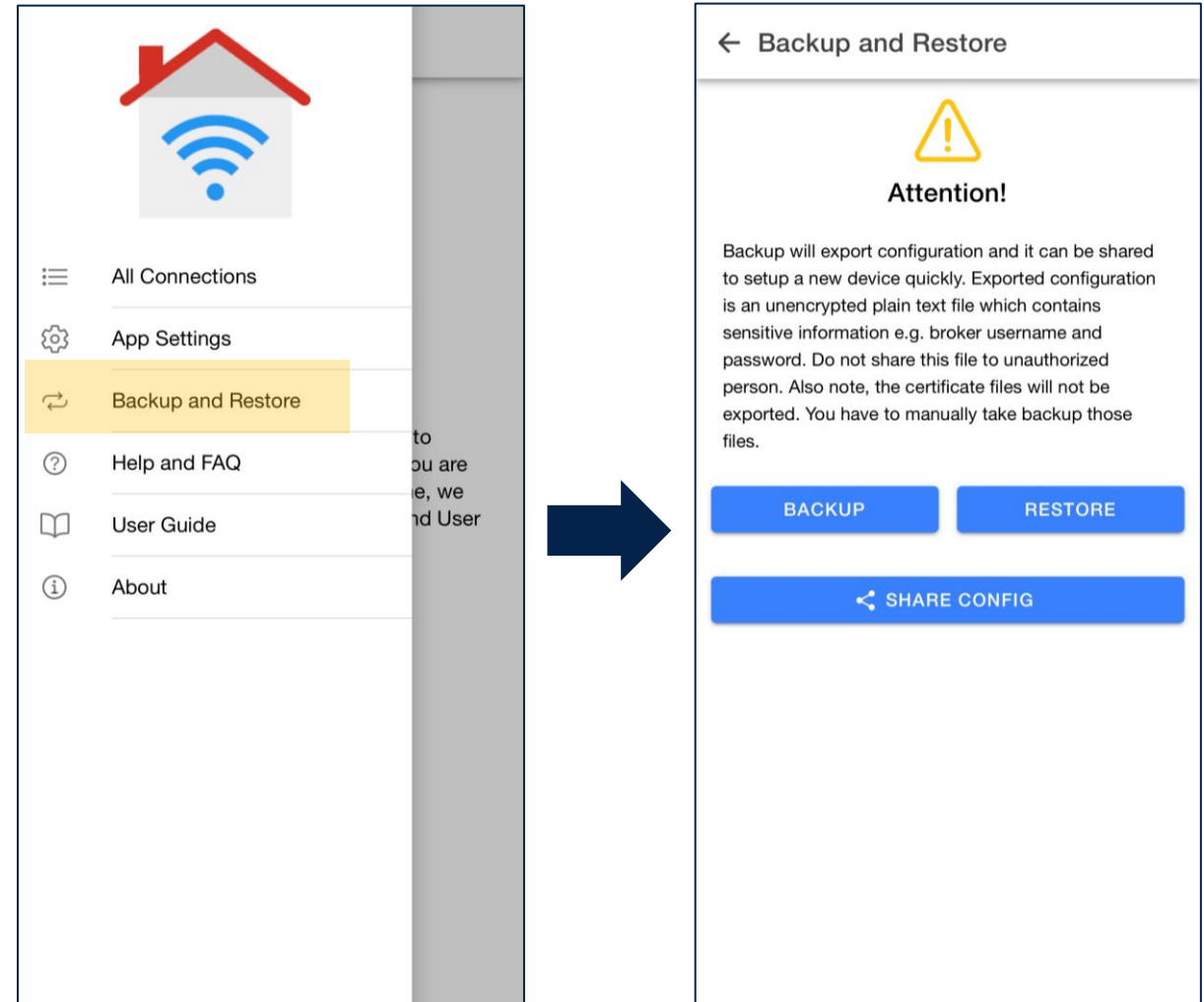
- With IoT MQTT Panel Application open select “Connections” to bring up MQTT Panel application menu.



ST67W6X MQTT

IoT MQTT Panel Client Setup – Connection Setup

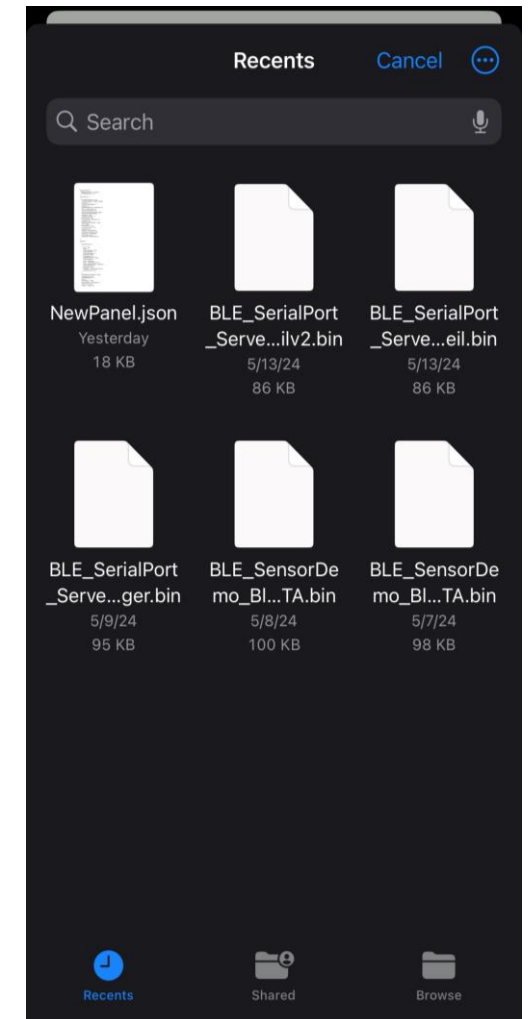
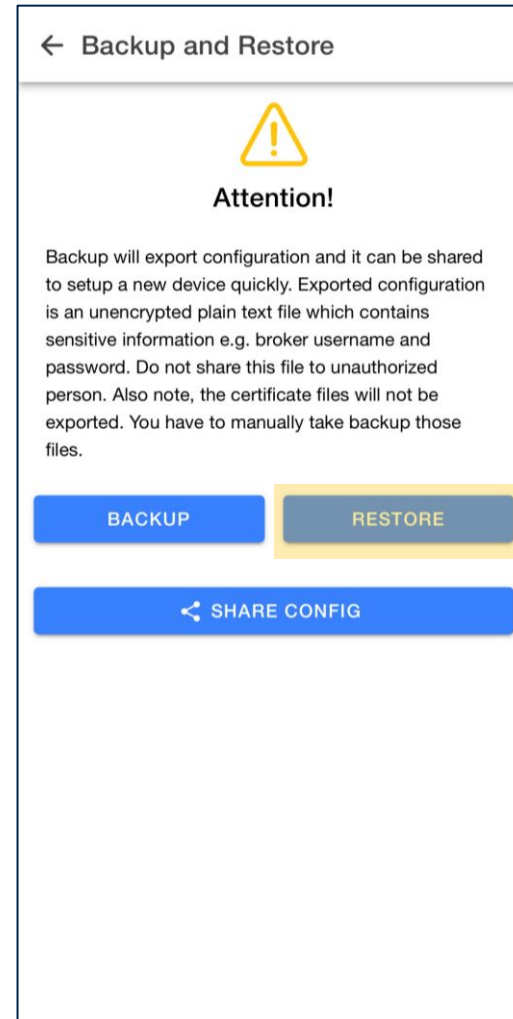
- Select “Backup and Restore” to “Backup” an existing connection or “Restore” a preconfigured connection.



ST67W6X MQTT

IoT MQTT Panel Client Setup – Connection Setup

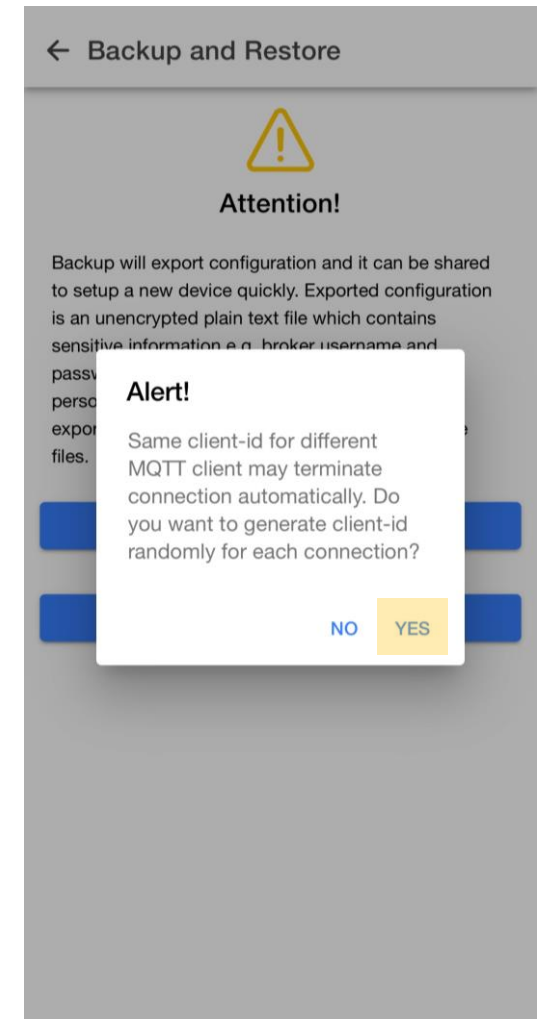
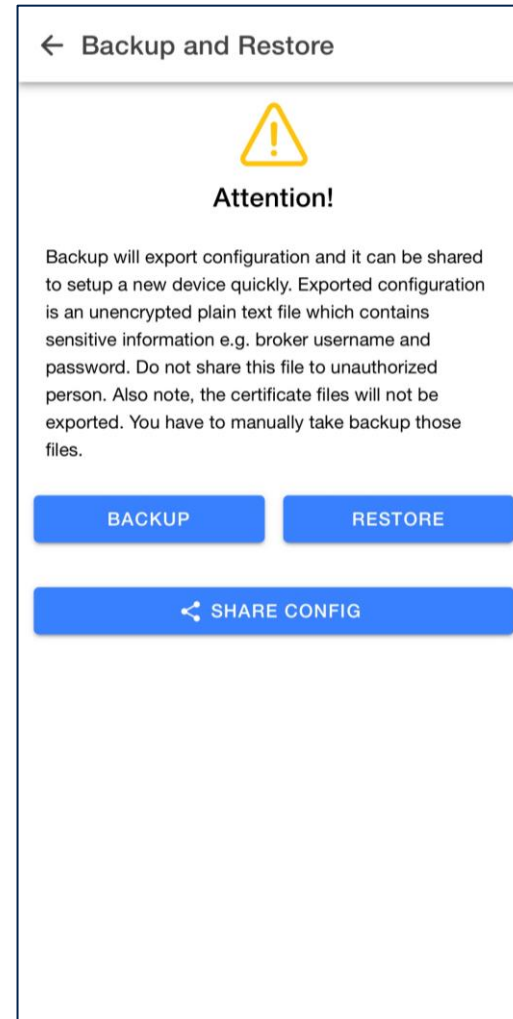
- Select “Restore” to bringup file storage and select preconfigured .json file stored within smart device’s memory



ST67W6X MQTT

IoT MQTT Panel Client Setup – Connection Setup

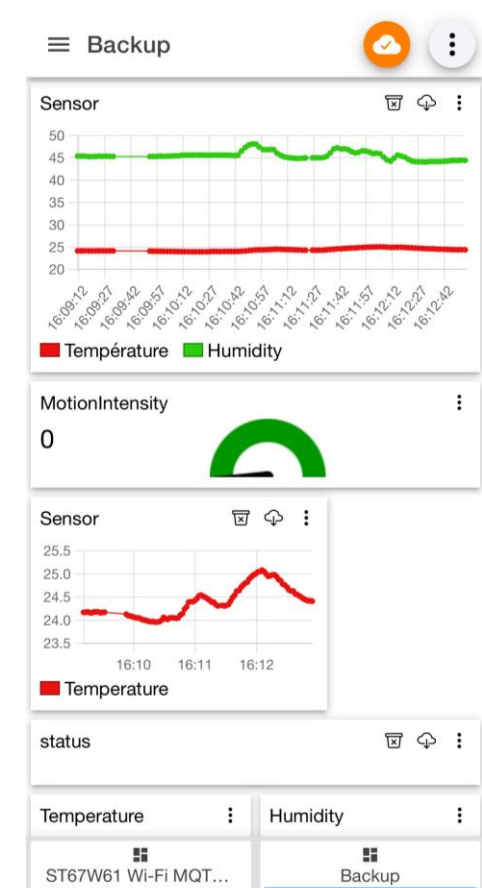
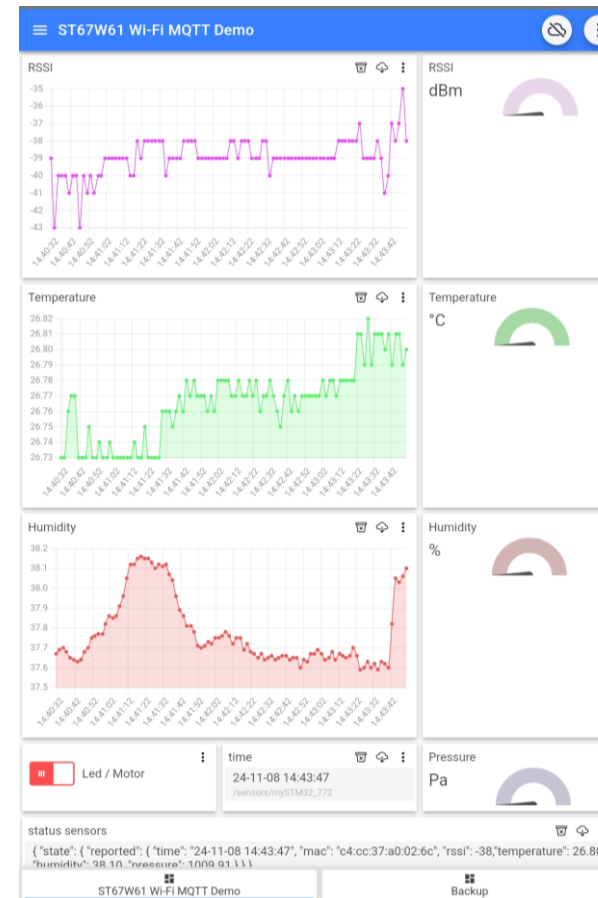
- With preconfigured .json file selected, the IoT MQTT Panel application will “Alert!” the user regarding same client-id.
- Select “Yes”.
 - Client-ID referred to here is correlated to the connection. Client-ID configured previously in the .json and app_config.h is correlated to the topic data is published under.



ST67W6X MQTT

IoT MQTT Panel Client Setup - Security Scheme 1

- The main dashboard:
 - Display the graphical values of RSSI, Temperature, Humidity and Pressure from device published message
 - The Green LED statues on the NUCLEO-U575 Host board can be managed through Button panel
- The backup dashboard:
 - Display the graphical values of Humidity, Temperature and Motion Intensity





life.augmented

Documentation



Access:

The wifi wiki is opened to all



Wiki address of the main page

https://wiki.st.com/stm32mcu/wiki/Connectivity:Introduction_to_Wi-Fi



We will be happy to get your feedback!

Wiki Wi-Fi®: pages breakdown

About Wi-Fi

1 [Wi-Fi® overview](#)

1.1 [What is Wi-Fi®](#)

1.2 [Wi-Fi® network architecture](#)

1.3 [Wi-Fi® features details](#)

Details about
ST67W611M1

2 [ST67W611M1](#)

2.1 [ST67W611M1 overview](#)

2.2 [X-CUBE-ST67W61](#)

2.3 [ST67W611M1: Hardware and board description](#)

2.4 [ST67W611M1 Security overview](#)

2.5 [ST67W611M1-based customer end-Product](#)

2.6 [ST67W611M1 evaluation](#)

X-CUBE-ST67W61

3 [X-CUBE-ST67W61 software application notes and user manuals](#)

3.1 [Getting started with ST67W611M1](#)

Host boards
supported

4 [ST67W611M1 associated hosts boards](#)

5 [Specific tools](#)

Tools & References

6 [Terms and definitions](#)

7 [References](#)

Other documentation

- Getting started with X-CUBE-ST67W61 [here](#)

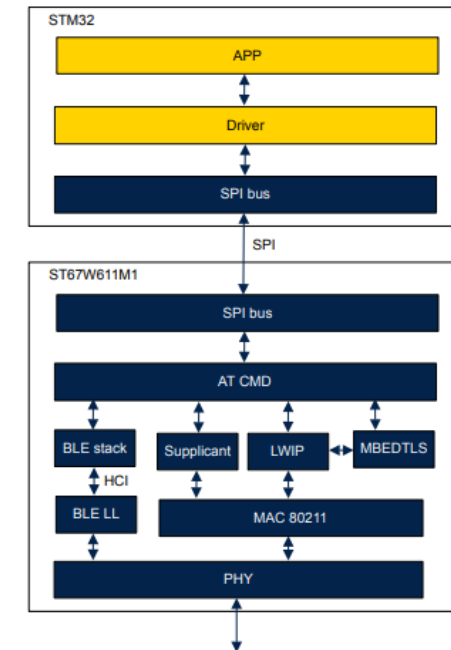
2

X-CUBE-ST67W61 architecture overview

The X-CUBE-ST67W61 is a set of software components implementing host applications driving a Wi-Fi® and Bluetooth® LE coprocessor. The coprocessor is controlled via AT command over SPI interface.

The figure below illustrates the architecture of the solution:

Figure 1. X-CUBE-ST67W61 architecture view



- MW API documentation is available:
 - X-CUBE-ST67W61_V1.0.0\Middlewares\ST\ST67W6X_Network_Driver\Doc\ST67W6X_Network_Driver.chm

ST Customer Support

- [ST Support Home](#)
 - Tickets can be submitted under the part number X-CUBE-ST67W61 or ST67W611M1

Our technology starts with You



Find out more at www.st.com

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



life.augmented

Backup Slides